TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes

Benedikt Bünz
bb@nyu.edu
NYU, Espresso Systems

Giacomo Fenzi
giacomo.fenzi@epfl.ch
EPFL

Ron D. Rothblum
rothblum@gmail.com
Succinct

William Wang
ww@priv.pub
NYU

June 8, 2026

Abstract

A polynomial commitment scheme (PCS) enables a prover to succinctly commit to a large polynomial and later generate evaluation proofs that can be efficiently verified. In recent years, PCSs have emerged as a central focus of succinct non-interactive argument (SNARG) design.

We present TensorSwitch, a hash-based PCS for multilinear polynomials that improves the state-of-the-art in two fundamental bottlenecks: prover time and proof size.

We frame our results as an *interactive oracle PCS*, which can be compiled into a cryptographic PCS using standard techniques. The protocol uses any linear code with rate ρ, list-decoding and correlated agreement up to δ, and encoding time $\tau \cdot \ell$, where ℓ is the block length. For a size n polynomial, security parameter λ, and sufficiently large field, it has the following efficiency measures, up to lower order terms:

- Commitment time: $(\tau/\rho^2 + \tau/\rho + 3) \cdot n$ field multiplications.
- Opening time: $6n$ field multiplications.
- Query complexity: $\frac{1}{-\log(1-\delta^2)} \cdot \lambda$.
- Verification time: $O(\lambda \log n)$.

Moreover, the evaluation proof only contains $O(\log \log n)$ oracles of total size $(\lambda n)^{0.5+o(1)}$.

With a Reed-Solomon code of rate $1/2$, the query complexity is 2.41λ and commitment time is dominated by $(6 \log n + 3) \cdot n$ field multiplications. With an RAA code of rate $1/4$ and distance 0.19, the query complexity is 19λ and the commitment time is $42n$ field additions and $3n$ field multiplications. For both instantiations, the opening time is dominated by $6n$ field multiplications.

Contents

1 Introduction **3**
- 1.1 Our results . . . 4
- 1.2 Related work . . . 7

2 Techniques **10**
- 2.1 Detecting malformed commitments . . . 10
- 2.2 Checking evaluation claims . . . 12
- 2.3 Beyond the unique decoding radius . . . 14
- 2.4 Compiling linear queries to point queries . . . 15
- 2.5 Round-by-round security . . . 16

3 Preliminaries **19**
- 3.1 Multilinear extensions . . . 19
- 3.2 Interactive oracle proofs . . . 20
- 3.3 Codes . . . 22
- 3.4 Out-of-domain sampling . . . 23
- 3.5 Mutual correlated agreement . . . 23

4 Functional commitments with oracles **24**

5 Linear commitments with linear oracles **26**
- 5.1 Definitions . . . 27
- 5.2 Committing with tensor codes . . . 28
- 5.3 Proving linear evaluations . . . 30
- 5.4 Batch proving linear evaluations . . . 35

6 Multilinear polynomial commitments with linear oracles **40**

7 Right-extensions of linear codes **41**
- 7.1 Succinct right-extensions . . . 41
- 7.2 Delegating right-extension computations . . . 42

8 Multilinear polynomial commitments with point oracles **44**
- 8.1 Linear commitments with point oracles . . . 44
- 8.2 Multilinear polynomial commitments with point oracles . . . 47
- 8.3 Main theorem . . . 50

Acknowledgements **53**

References **53**

A Composition lemmas **58**
- A.1 Proof composition . . . 58
- A.2 Commitment composition . . . 63

1 Introduction

Succinct non-interactive arguments (SNARGs) [Mic00] have emerged as a fundamental cryptographic primitive. SNARGs enable a prover to produce a small certificate that it executed a computation correctly. Surprisingly, this certificate can be exponentially smaller, and more efficient to verify, than the complexity of the original computation.

One of the leading approaches to SNARG design is via hash functions [Kil92; Mic00] (see also the recent textbook [CY24]). Such hash-based SNARGs are of particular theoretical and practical interest for several reasons.

1. In the random oracle model (ROM), hash-based SNARGs can be constructed unconditionally (by using the random oracle as an ideal hash function). As there are significant barriers to building SNARGs from falsifiable assumptions in the plain model [GW11], unconditional security in the ROM may be the best one can hope for.[1] Furthermore, as interactive arguments [Kil92; CDGS23], they can be proven secure from just collision resistant hashing, a very mild symmetric-key assumption.

2. Hash-based SNARGs tend to have the fastest and most scalable provers. This is due to a variety of reasons: hash functions can be very lightweight, and they offer much more flexibility in terms of arithmetization (see, e.g., [HLP24; DP25]).

3. Unlike many SNARGs built with group-based cryptography [GGPR13; Gro16; CHMMVW20; GWC19], hash-based SNARGs do not require a trusted setup.

4. Hash-based SNARGs are plausibly secure against quantum adversaries. In particular, they can be constructed unconditionally in the quantum random oracle model [CMS19].

Due to the above facts, most industry deployments of SNARGs today are hash-based.[2]

The typical recipe for constructing SNARGs (which generalizes Kilian's original construction [Kil92]) is to combine an information-theoretically secure polynomial interactive oracle proof (PIOP) [RRR21; CHMMVW20; BFS20] with a polynomial commitment scheme (PCS) [KZG10]. The resulting SNARG's prover time, proof size and verifier time directly depend on the complexity of the PCS. In fact, since we have essentially optimal PIOPs for circuits [Set19; CBBZ23], the efficiency of the underlying PCS is often a key bottleneck in SNARG implementations.

Polynomial commitment schemes. Informally, a PCS consists of two phases. In the *commitment phase*, the prover outputs a short commitment cm to a polynomial p. In the *opening phase*, the prover outputs a short proof π attesting to an evaluation claim of the form $p(x) = y$. For efficiency reasons, the size of the commitment and proof should be much smaller than the size of the polynomial. A verifier, given the commitment cm and proof π, can check the evaluation claim. A PCS must satisfy two properties.

- *Completeness.* Assuming the commitment cm and proof π are computed correctly, the verifier will always accept.

[1] Constructions from falsifiable assumption are possible but would require a non-black-box reduction, establish only non-adaptive soundness, or utilize an exponential-time security reduction as in the Sahai–Waters SNARG [SW14].

[2] See, e.g., `https://ethproofs.org`.

- *Binding.* Once the prover sends to the verifier a commitment cm, a polynomial p is uniquely determined. If the verifier accepts proofs $\pi_1, \ldots, \pi_\ell$ attesting to evaluation claims $(x_1, y_1), \ldots, (x_\ell, y_\ell)$, then $p(x_i) = y_i$ for all $i \in [\ell]$. This is regardless of how cm and π_i are computed, assuming the prover is computationally bounded.

Limitations of hash-based PCSs. One limitation of existing hash-based PCSs, and therefore hash-based SNARGs, is that they have relatively large proofs. This is especially true for those with the fastest, linear-time provers [GLSTW23; BCFRRZ25]. Their proofs are megabytes in size, even for moderate sized polynomials (e.g., 1.5MB for a 2^{25} sized input). This is a problem, especially when the SNARG is distributed to a large set of verifiers or posted on a bandwidth-constrained blockchain. To circumvent this issue, most practical deployments compress a hash-based SNARG proof by "wrapping" it in a group-based SNARG [Sta21].[3] While this can significantly reduce the verifier time and proof size, it adds latency to proof generation, incurs non-native field arithmetic emulation, removes any plausible post-quantum security, and typically requires a trusted setup for the outer SNARG. For this reason, the Ethereum Foundation recently put out a call to build proof systems that are solely hash-based and do not require wrapping [Gol25].

A second key bottleneck in SNARGs is the prover time, especially when proving statements in small (non-exponentially sized) fields. In this case, the verifier's randomness is sampled from an exponentially sized extension field (in practice, often $4\times$ larger than the base field over which the computation is defined). For all prior hash-based PCSs with polylogarithmic proof size, e.g., [BBHR18; ACFY24; ACFY25; BCFRRZ25], the prover needs to encode/commit to data, over the extension field, whose size is linear in the input length. This step is often the most expensive in practice, even compared to the original input (which is over the base field).

1.1 Our results

We present TensorSwitch, a hash-based PCS which addresses both of these issues by achieving close-to-optimal performance characteristics. Combining this PCS with existing PIOPs immediately yields similar improvements to the corresponding hash-based SNARGs.

TensorSwitch can be instantiated with any linear code with distance δ and rate ρ. Under plausible conjectures on the proximity gaps and list decoding radius of this code, it achieves a close to optimal proof size consisting of $(1/\delta^2 + o(1)) \cdot \lambda$ Merkle paths, where λ is the security parameter.[4] The verifier is dominated by verifying these hashes.

To get a sense of what is optimal, note that hash-based PCSs essentially always encode the polynomial with a code, then commit to the codeword with a Merkle tree. Yet even just to distinguish[5] two valid codewords from each other with soundness $2^{-\lambda}$, a verifier has to check λ/δ queries; this results in λ/δ Merkle paths that need to be sent. Thus, our proof size and verifier time is essentially optimal up to a $(1/\delta + o(1))$ factor, and asymptotically better than all prior work. This additional factor stems from the use of tensor codes, which have distance δ^2 when using a

[3] The outer SNARG proof certifies that the inner SNARG verifier would have accepted. The proof size and verifier time largely depends on the outer SNARG, while the prover time is dominated by generating the inner SNARG proof.

[4] More precisely, $(1/-\log(1-\delta^2) + o(1)) \cdot \lambda$. For convenience, we approximate $-\log(1-x) \approx x$ for small $x > 0$ throughout the introduction.

[5] If a verifier cannot distinguish between codewords, then the commitment cannot be binding. This seems to violate the basic security requirement of a PCS.

base code of distance δ. It seems that no protocol using tensor codes to encode the polynomial can do better than our protocol (up to additive factors).

TensorSwitch's commitment phase consists of encoding the polynomial with a two-dimensional tensor code, committing to the codeword with a Merkle tree, $3 \cdot n$ field multiplications, and $3 \cdot n$ additions. Given that n multiplications are required to even just evaluate the polynomial, this is optimal up to a multiplicative factor of 6. The opening phase is dominated by $6 \cdot n$ field multiplications. The hashing/encoding work is $(\lambda n)^{0.5+o(1)}$, as compared to $\tilde{\Theta}(n)$ for all prior work with polylogarithmic proof size. Finally, the prover only computes $O(\log \log n)$ Merkle tree commitments, significantly less than in prior practical hash-based PCSs, e.g., $\Omega(\log n)$ in [BBHR18; ACFY24; ACFY25; BCFRRZ25].

The information-theoretic core. Hash-based PCSs are typically attained by constructing an interactive oracle proof (IOP) of proximity for a linear code [RRR21; BCGRS17; BBHR18]. This is an interactive proof where the input and prover's messages are allowed to be long, but the verifier should only read few of their symbols. Loosely speaking, an IOP of proximity is used to convince the verifier that its input is close to the code (and moreover, it is close to a codeword satisfying certain constraints).

Any IOP of proximity implies an "information-theoretic PCS" protocol that works as follows. The prover encodes the polynomial's coefficients with a linear code, and this codeword serves as the commitment. In the (interactive) opening phase, the prover and verifier jointly run an IOP of proximity on the commtiment. This protocol can then be compiled into a hash-based PCS via standard transformations [Kil92; Mic00; BCS16].

However, TensorSwitch cannot be formulated as an IOP of proximity.[6] In order to capture it, we introduce and formally define a new abstraction, which we call an interactive oracle PCS (IOPCS).[7] The main novelty is that an IOPCS's commitment phase is interactive, and hence the prover can send multiple oracles. We require information-theoretic binding, i.e., with high probability (over the verifier's randomness) the commitment phase's transcript uniquely defines a polynomial, and the opening phase only accepts evaluation claims consistent with this polynomial.

IOPCSs generalize the "IOP of proximity recipe" and are the information-theoretic abstraction underlying essentially all hash-based PCS constructions. After compiling, the PCS works as follows:

- The prover runs the IOPCS prover and commits to each oracle with a Merkle tree.
- The proof size is dominated by q Merkle paths, where q is the number of symbols read by the IOPCS verifier (we refer to q as the query complexity).
- The verifier runs the IOPCS verifier and validates q Merkle paths.

Therefore, in addition to the IOPCS's prover and verifier time, it is important to minimize the total oracle length and query complexity; these correspond to the hashing cost of the prover and verifier, respectively.

Below we give an informal but precise (up to additive constants) analysis of TensorSwitch's efficiency measures as an IOPCS; see Theorem 8.5 for the formal statement.

[6] It is possible to use our techniques to construct an IOP of proximity for tensor codes. However, the query complexity has worse constants compared to TensorSwitch, which is a "direct" construction of a PCS.

[7] In earlier work, Diamond and Posen [DP24b] consider a similar abstraction, under the same name, with important conceptual differences; see Remark 1.4.

Theorem 1.1 (informal)**.** *Let $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ be a linear code of rate $\rho = k/\ell$, with mutual correlated agreement (Definition 3.20) and list-decoding (Definition 3.11) up to distance $\delta \in (0,1)$, that is encodable by an arithmetic circuit of size $\tau \cdot \ell$. Let $\mathcal{C}^2\colon \mathbb{F}^{k\times k} \to \mathbb{F}^{\ell\times\ell}$ be the corresponding tensor code and let $n = k^2$ be the input size.*

For any security parameter $\lambda \in \mathbb{N}$, let $\mathbb{F}$ be a sufficiently large field, e.g., $|\mathbb{F}| = 2^\lambda \cdot \mathrm{poly}(n)$. There exists an IOPCS for multilinear polynomials of size $n = k^2$ with soundness error $2^{-\lambda}$ and the following efficiency measures.

- *Commitment time:* $(\tau \cdot (1/\rho^2 + 1/\rho) + 3) \cdot n + (\lambda n)^{0.5+o(1)} \approx (\tau/\rho^2) \cdot n$ *$\mathbb{F}$-multiplications.*
- *Opening time:* $6n + (\lambda n)^{0.5+o(1)} \approx 6n$ *$\mathbb{F}$-multiplications.*
- *Commitment oracle length:* $(1/\rho^2) \cdot n + (\lambda n)^{0.5+o(1)} \approx (1/\rho^2) \cdot n$ *$\mathbb{F}$-elements.*
- *Proof oracle length:* $(\lambda n)^{0.5+o(1)} \ll n$ *$\mathbb{F}$-elements.*
- *Round complexity:* $O(\log \lambda)$.
- *Number of oracles:* $O(\log \log n)$.
- *Query complexity:* $\left(\frac{1}{-\log(1-\delta^2)} + o(1)\right) \cdot \lambda \approx \lambda/\delta^2$.
- *Verification time:* $O(\lambda \log n)$.
- *Non-oracle communication length:* $O(\lambda + \log n)$ *$\mathbb{F}$-elements.*

Remark 1.2. In the technical sections, we consider tensor products of possibly different codes. To achieve the exact efficiency measures reported in Theorem 8.5, one must employ a slightly skewed tensor. Otherwise, all $(\lambda n)^{0.5+o(1)}$ terms become $\lambda \cdot n^{0.5+o(1)}$.

Remark 1.3 (extension field operations)**.** As discussed above, we may distinguish between the base field, which the polynomial is defined over, and the extension field, from which the verifier samples randomness. Let us focus on the most expensive operations: field multiplications, encoding field elements, and committing to field elements with Merkle trees. Compared to the base field, it is significantly more expensive to do these operations over the extension field.

In TensorSwitch, the prover's extension field multiplications are dominated by $6n$. Moreover, extension field encoding/committing is only performed on data of total size $(\lambda n)^{0.5+o(1)}$. In contrast, all prior hash-based PCSs with polylogarithmic proof size require extension field encoding/committing on data of total size $\tilde{\Omega}(n)$.

New techniques. In a nutshell, our key technical contribution is an interactive reduction from claims over a message of size n, encoded with a tensor code, to claims over one of size $O_\lambda(\sqrt{n})$, encoded with a different code. Moreover, the verifier only issues $O(\lambda)$ queries (treating δ as a constant). In order to construct TensorSwitch, we choose this to be a smaller tensor code, repeating the reduction $O(\log \log n)$ times. In contrast, all prior hash-based PCSs with polylogarithmic proof size are constructed by repeating one of the following reductions (see Section 1.2 for a more comprehensive comparison):

- Arrange the message as a $c \times n/c$ matrix for a (near) constant c and encode with an interleaved code. The verifier samples $O(\lambda)$ columns, reading all c entries off of each column. This approach has $c \cdot O(\lambda)$ complexity, but it only shrinks the statement by a factor of c.
- Arrange the message as a $\sqrt{n} \times \sqrt{n}$ matrix and encode with a tensor code. The verifier samples $O(\lambda)$ columns, reading $O(\lambda)$ entries off of each column. This approach sufficiently shrinks the statement by a factor of $O_\lambda(\sqrt{n})$, but it requires the verifier to issue $O(\lambda^2)$ queries.

Our approach resembles the latter, except the verifier only reads *one* entry per column.

IVC with sublinear proofs. As a secondary application, our techniques immediately imply efficient constructions of incrementally verifiable computation [Val08] and proof-carrying data [CT12] with $O_\lambda(\sqrt{n})$-sized proofs, via accumulation schemes [BCLMS21]. We briefly sketch the construction. Instead of reducing to a smaller tensor code, suppose we reduce to a linear code with good distance, e.g., a low-rate Reed–Solomon code. Composing with one of [BMNW25; BCFW25] yields a hash-based accumulation scheme (for multilinear evaluations) with $O_\lambda(\sqrt{n})$-sized accumulators. Moreover, the efficiency measures are dominated by that of the initial reduction.

1.2 Related work

For reasons mentioned above, we restrict our focus to *hash-based* PCSs. Below we review prior works, roughly categorized based on their techniques. Throughout, recall that TensorSwitch has linear prover time (with nearly optimal overhead), $O(\lambda)$ query complexity, and $(\lambda n)^{0.5+o(1)}$ extension field encoding/committing cost.

IOPs for Reed–Solomon codes. FRI [BBHR18] and subsequent work [ZCF24; ACFY24; ACFY25] construct IOPs of proximity for Reed–Solomon codes (and, more generally, foldable codes in [ZCF24]). The start-of-the-art query complexity is $O(\lambda \log\log n)$ [ACFY24; ACFY25]. In terms of prover cost, there are two downsides. First, the prover time is quasilinear due to the use of Reed–Solomon codes. Second, the extension field encoding/committing cost is $\tilde{\Omega}(n)$. This stems from the use of "folding" reductions, which reduce a statement of size n to a statement, over the extension field, of size at most $n/\operatorname{polylog}(n)$.

IOPs for interleaved codes. Ligero [AHIV23] and subsequent work [GLSTW23] construct IOPs of proximity for interleaved codes. Using a linear-time encodable code, the prover time is linear. Moreover, the PCS prover does not encode or commit to *any* extension field elements. The main downside is that the PCS evaluation proof size (and verifier time) is $O(\sqrt{\lambda n})$, which is not polylogarithmic.

Building on this, Blaze [BCFRRZ25] and subsequent work [BMMS25a; NA25] employ code switching [RR24] to achieve polylogarithmic proof sizes while maintaining linear prover time. The state-of-the-art query complexity is $O(\lambda \log\log n)$ [BMMS25a]. However, these techniques lead to $\tilde{\Omega}(n)$ extension field encoding/committing costs.

Finally, we note that it is always possible to "wrap" Ligero PCS evaluation proofs in a hash-based SNARG with polylogarithmic proofs; see [XZS22] (and [CBBZ23; HS24], which address soundness issues in the original construction) for a concrete example. This approach leads to $O(\sqrt{\lambda n})$ extension field encoding/committing cost. However, it introduces high concrete overheads and implementation complexity. Note that wrapping is not provably secure in the ROM, since it requires non-black-box use of the underlying hash function.

IOPs for tensor codes. Most relevant to this work, Ligero++ [BFHVXZ20] constructs an IOPCS as follows. In the commitment phase, the prover encodes the polynomial with an interleaved code. The interleaving factor is chosen to be $\operatorname{polylog}(n)$, so that each symbol of the codeword is of size $n/\operatorname{polylog}(n)$. Instead of directly sending these symbols to the verifier, the prover sends their *encodings* under a "column" code. Equivalently, the prover encodes the polynomial with a tensor code. The opening phase proceeds as in Ligero, except the protocol uses an IOP of proximity

(for the column code) to prove computations that the Ligero verifier would have performed on the interleaved codeword. This leads to the following efficiency measures:

- The prover time is quasilinear, since Ligero++ (crucially) uses a tensor of Reed–Solomon codes.
- The extension field encoding/committing cost is $\tilde{\Omega}(n)$ due to the use of a heavily skewed tensor; this is required for the verifier to be efficient.
- The query complexity is $\Omega(\lambda^2 \cdot \log(n))$ (and at least $\Omega(\lambda^2)$ seems inherent to their approach), due to invoking the IOP of proximity $O(\lambda)$ times.

TensorSwitch takes a similar approach, but with several key differences. We use (almost) square tensors of arbitrary linear codes, which enables linear commitment time and $(\lambda n)^{0.5+o(1)}$ extension field encoding/committing cost. Moreover, our query complexity is $O(\lambda)$.

Finally, we mention some works that are more theoretical in nature. Arnon, Chiesa, and Yogev [ACY23] construct an IOP of proximity for Reed–Solomon codes (and Reed–Muller codes) with $O(\log \log n)$ rounds and $O(\log \log n)$ queries, but only for inverse polynomial soundness error. Recently, Minzer and Zheng [MZ25] improved the soundness error to be negligible with effectively $O(\lambda)$ query complexity. However, the construction is only instantiable with large statements, due to large constants in its efficiency measures. These IOPs have quasilinear prover time due to the use of Reed–Solomon codes.

	commitment time	opening time	proof size
Brakedown	$25.5n$ $\mathbb{F}$, $2n$ H	$O(n)$ $\mathbb{F}$	$O(\sqrt{\lambda n})$ $\mathbb{F}$
Blaze	$8n$ $\mathbb{F}_+$, $4n$ H	$O(n)$ $\mathbb{F}$, $O(n)$ H	$O_\delta(\lambda \log^2(n))$ MP
FRI	$\log(n)/\rho \cdot n$ $\mathbb{F}$, $1/\rho \cdot n$ H	$O(n)$ $\mathbb{F}$, $O(n)$ H	$O_\delta(\lambda \log n)$ MP
WHIR	$\log(n)/\rho \cdot n$ $\mathbb{F}$, $1/\rho \cdot n$ H	$\tilde{O}(n)$ $\mathbb{F}$, $O(n)$ H	$O_\delta(\lambda \log \log n)$ MP
TensorSwitch (RS)	$\log(n)/\rho^2 \cdot n$ $\mathbb{F}$, $1/\rho \cdot n$ H	$6n$ $\mathbb{F}$, $(\lambda n)^{0.51}$ H	$\frac{1}{-\log(1-\delta^2)} \cdot \lambda$ MP
TensorSwitch (Blaze)	$42n$ $\mathbb{F}_+$, $3n$ $\mathbb{F}$, $16n$ H	$6n$ $\mathbb{F}$, $(\lambda n)^{0.51}$ H	19λ MP

Table 1: Comparison of hash-based PCS efficiency measures, up to lower order terms. The verifier times are all linear in the proof size. $\mathbb{F}$ denotes field multiplications, $\mathbb{F}_+$ denotes field additions, H denotes hashes, and MP denotes Merkle paths. Brakedown is instantiated with $\tau = 14.8$ and rate $\rho = 0.58$; its proof size is dominated by field elements. Blaze is instantiated with $\tau = 2$ and rate $\rho = 1/4$; we assume correlated agreement up to distance $\delta = 0.19$. FRI, WHIR, and TensorSwitch (RS) use a Reed–Solomon code (over a suitable field) with $\tau = \log n$ and rate ρ; we assume correlated agreement up to distance δ.

Remark 1.4 (IOPCS models)**.** Diamond and Posen [DP24b] also propose an IOPCS abstraction; the main conceptual difference between theirs and ours is that their commitment phase is non-interactive. On the other hand, in order to capture TensorSwitch (and, more generally, protocols that work beyond the unique decoding radius of codes, e.g., [BGKS20; ACFY24; ACFY25]), we consider interactive commitment phases. Furthermore, establishing (Fiat–Shamir) security of such protocols requires new notions of information-theoretic binding, which we introduce in Section 4.

Remark 1.5 (Concurrent work)**.** Baweja, Mishra, Mopuri, and Shtepel [BMMS25b] construct an IOP of proximity for tensor codes with linear prover time and $O(\lambda)$ query complexity. Their result

is asymptotically similar to ours, up to the leading constants in prover time and query complexity. However, these constants are crucial for obtaining efficient proof systems. As an example, although their construction has $O(\lambda)$ query complexity, the multiplicative constant is larger than that of TensorSwitch by a factor of $2/(1-\sqrt{1-\delta^2})^2$. For $\delta = 0.19$ (the Blaze code distance), this is greater than 6000. Targeting $\lambda = 128$ bits of security, TensorSwitch requires 2414 queries, whereas [BMMS25b] requires over 14 million.

This disparity stems from the fact that our techniques, while superficially similar to theirs, are actually quite different. For example, we use two-dimensional tensor codes, whereas they require three-dimensional tensors, which are concretely more expensive. Moreover, their core reduction only reduces a statement of size n to a statement of size $O_\lambda(n^{2/3})$, whereas ours reduces to a statement of size $O_\lambda(\sqrt{n})$.

2 Techniques

Recall that our goal is to construct an interactive oracle PCS. Specifically, we will target a commitment scheme for *multilinear polynomials*, which consists of the following phases:

- *Commitment phase.* The prover and verifier interact over multiple rounds, and with high probability the resulting transcript acts as a (binding) commitment to some underlying vector $m \in \mathbb{F}^n$.
- *Opening phase.* The prover and verifier interact over multiple rounds to check evaluation claims of the multilinear extension $\hat{m} \colon \mathbb{F}^{\log n} \to \mathbb{F}$ (see Section 3.1).

Both phases are executed in the point query model: the prover is allowed to send large oracles, from which the verifier only queries a few symbols. In Section 4 we formally define interactive oracle PCSs (and, more generally, interactive oracle *functional* commitments).

Our protocol employs tensor codes. In this overview, we focus on a square tensor code $\mathcal{C}^2$, where the underlying base code $\mathcal{C} \colon \mathbb{F}^k \to \mathbb{F}^\ell$ has message length $k = \sqrt{n}$; the protocol generalizes in a straightforward way to (skewed) tensors of different codes. Recall that $\mathcal{C}^2$ encodes a message, viewed as a matrix $M \in \mathbb{F}^{k \times k}$, as follows. The codeword $F \in \mathbb{F}^{\ell \times \ell}$ is obtained by encoding each row of M with $\mathcal{C}$ and then encoding each resulting column with $\mathcal{C}$ (see Section 3.3).

We denote the minimal relative distance of $\mathcal{C}$ by δ. The unique decoding radius of $\mathcal{C}$ is $\delta_{\mathsf{UD}} \coloneqq \delta/2$.

Commitment phase. The prover commits to M by sending, as an oracle, its encoding $F \coloneqq \mathcal{C}^2(M)$ under the tensor code. Naturally, a dishonest prover might send a matrix $F \in \mathbb{F}^{\ell \times \ell}$ that is not a tensor codeword, and the verifier needs to handle this scenario. Define the *column-decoding* G to be the $k \times n$ matrix obtained by decoding each column of F under $\mathcal{C}$, correcting less than a δ_{UD}-fraction of errors in each column (if a column of F is not decodable, then the corresponding column of G is defined to be $\perp$).

If the prover is honest, then G is the "row-wise encoding" of M; equivalently, G is the encoding of M under the *k-fold interleaving* of $\mathcal{C}$. On the other hand, a dishonest prover can choose F in one of the two following ways.

- The matrix G (uniquely determined by F) is δ_{UD}-close to the k-fold interleaving of $\mathcal{C}$. Equivalently, there exists a (unique) matrix $M \in \mathbb{F}^{k \times k}$ whose row-wise encoding disagrees with G at less than a δ_{UD}-fraction of columns. In this case we say the commitment is *well-formed*, and the opening phase will check evaluation claims against (the multilinear extension of) M.
- No such matrix M exists. In this case, we say that the commitment is *malformed*.

We will first show how the verifier can reject a malformed commitment during the opening phase. Afterwards, we will show how to check evaluation claims against a well-formed commitment.

2.1 Detecting malformed commitments

Suppose that the commitment is malformed, i.e., G is δ_{UD}-far from the k-fold interleaving of $\mathcal{C}$. Following [AHIV23] (and many follow-up works, e.g., [GLSTW23; BCFRRZ25]), we will attempt to detect this case by considering a random linear combination of the rows of G.

In the opening phase, the verifier samples coefficients $r \leftarrow \mathbb{F}^k$ and the prover sends $\mathsf{combo} \in \mathbb{F}^k$, which is purported to be the decoding of $r^T G$. Recall that if the prover had been honest, then

it could have chosen $\mathsf{combo} \coloneqq r^T M$, and indeed it would hold that $\mathcal{C}(\mathsf{combo}) = r^T G$. But what happens when the prover is dishonest? Here we rely on a line of works studying "proximity gaps" [RVW13; AHIV23; BKS18; BGKS20; BCIKS20; DP24a; GKL24; Zei24]. These works show the following: if G is δ_{UD}-far from the k-fold interleaving of $\mathcal{C}$, then with high probability (over the choice of r) it holds that $r^T G$ is δ_{UD}-far from $\mathcal{C}$. In particular, $r^T G$ will be δ-far from $\mathcal{C}(\mathsf{combo})$.

In [AHIV23], this inconsistency is detected using "spot checks": the verifier samples a random index $i \in [\ell]$ and, letting g_i denote the i-th column of G, checks that $r^T g_i$ is equal to the i-th entry of $\mathcal{C}(\mathsf{combo})$. Repeating $O(\lambda)$ times ensures that an inconsistency is detected with all but $2^{-\lambda}$ probability. Unfortunately, this strategy results in a query complexity of $\Omega(\lambda \cdot \sqrt{n})$, since the verifier needs to read combo and the selected columns of G in their entirety.

Linear queries. To achieve our desired query complexity, we will need to take a different route. Moving forward, we will work in the *linear* query model [IKO07; BCIOP13], which serves as a useful layer of abstraction. In this model, the verifier may access *arbitrary linear functions* of the prover's messages. We emphasize that the verifier still only has point query access to the commitment F, but for the moment we endow the verifier with the ability to issue linear queries to any additional oracles sent by the prover. Importantly, these oracles will be significantly smaller than F, and, using recursion, we will later (in Section 2.4) compile this construction into an interactive oracle PCS in the point query model.

Switching to the linear query model immediately addresses one of our concerns regarding query complexity: since combo is now a linear oracle (and $\mathcal{C}$ is a linear code), the verifier may compute an arbitrary entry of $\mathcal{C}(\mathsf{combo})$ with a single linear query to combo. In particular, the linear function is given by the corresponding row of the code's generator matrix. Since there are $O(\lambda)$ spot checks, the verifier only needs to issue $O(\lambda)$ linear queries.

Our other concern is that the verifier cannot afford to read a selected column of G in its entirety. When working with an interleaved code, this cost seems inherent. Here is where tensors change the picture: the verifier actually has access to (alleged) encodings of the columns of G under $\mathcal{C}$, via the columns of F. Our goal is to exploit this structure to read only a few symbols from F. Specifically, for each sampled index i, it suffices for the verifier to test the following claim: the i-th column of F decodes to a vector whose inner product with r equals the i-th entry of $\mathcal{C}(\mathsf{combo})$.

Code switching. Thus, the task at hand is testing proximity to a codeword that satisfies an additional (linear) constraint. We tackle it following the code switching technique of Ron-Zewi and Rothblum [RR24]. Phrased in the linear query model, the idea is for the prover to send each selected column of G as a fresh *linear* oracle.[8]

In more detail, for each index i sampled by the verifier, the prover sends $\mathsf{col}_i \in \mathbb{F}^k$, which is purported to be the i-th column of G. We emphasize that this is useful because the verifier only has point query access to the columns of F, whereas it has linear query access to each col_i. Note that, by issuing a linear query to each oracle, the verifier can check that $r^T \mathsf{col}_i = \mathcal{C}(\mathsf{combo})[i]$. Therefore, for each sampled i, the prover has two options: either truthfully set $\mathsf{col}_i \coloneqq g_i$, or lie and send $\mathsf{col}_i \neq g_i$. In the former case, the spot check functions identically to [AHIV23]. In the latter case, observe that $\mathcal{C}(\mathsf{col}_i)$ must be δ_{UD}-far from the i-th column of F. This means that the verifier

[8] Looking ahead to compilation (Section 2.4), the prover will actually send, as a point oracle, a fresh *encoding* of the selected columns of G under a different (tensor) code.

can detect the prover's lie by spot checking between $\mathcal{C}(\mathsf{col}_i)$ and i-th column of F (as with combo, the verifier may compute an arbitrary entry of $\mathcal{C}(\mathsf{col}_i)$ with a single linear query).

At first glance, the above test seems to require $\Omega(\lambda^2)$ queries: for each of the roughly λ selected columns, we need to perform roughly λ spot checks. However, looking more closely, it turns out that a *single* spot check for each column suffices. Consider the following spot check procedure, repeated t times in parallel:

1. The verifier samples a random column $i \in [\ell]$. With probability at least δ_{UD}, it holds that $r^T g_i \neq \mathcal{C}(\mathsf{combo})[i]$.
2. The prover sends col_i, and the verifier checks that $r^T \mathsf{col}_i = \mathcal{C}(\mathsf{combo})[i]$. In the case where $r^T g_i \neq \mathcal{C}(\mathsf{combo})[i]$, the prover must send $\mathsf{col}_i \neq g_i$, or else the verifier rejects.
3. The verifier checks that $\mathcal{C}(\mathsf{col}_i)$ and the i-th column of F agree at a random location. In the case where $\mathsf{col}_i \neq g_i$, with probability at least δ_{UD} the verifier rejects.

In each repetition, the verifier rejects with probability at least δ_{UD}^2. The verifier's overall probability of rejecting is therefore at least $1 - (1 - \delta_{\mathsf{UD}}^2)^t$. Aiming for $2^{-\lambda}$ soundness error, we set the number of repetitions to be

$$t := \frac{\lambda}{-\log(1 - \delta_{\mathsf{UD}}^2)} .$$

To conclude, the verifier makes t point queries to F and $O(t)$ linear queries to the linear oracles.

After compiling to the point query model, the $t = O(\lambda)$ queries to F will dominate the verifier's query complexity. Still, the multiplicative constant does not quite match that of Theorem 1.1. In Section 2.3, we describe additional techniques to reduce t, and consequently the number of queries made by the verifier.

2.2 Checking evaluation claims

Assuming the commitment is well-formed, there exists a (unique) matrix $M \in \mathbb{F}^{k \times k}$ whose row-wise encoding disagrees with G at less than a δ_{UD}-fraction of columns. We view M, reshaped into a vector $m \in \mathbb{F}^n$, to be the message underlying F.

Multilinear extension claims. We describe how to check evaluation claims against the multilinear extension of m. Without loss of generality, we focus on checking a single evaluation claim.[9]

Suppose the prover claims that $\hat{m}$ evaluated at a point $z \in \mathbb{F}^{\log n}$ is equal to $v \in \mathbb{F}$. In order to violate the interactive oracle PCS's binding property, the prover should make the verifier accept even though $\hat{m}(z) \neq v$. Our approach for detecting this is inspired by a generalization of the sumcheck protocol to tensor codes [Mei13; RR24].

Moving forward, we treat vectors in $\mathbb{F}^k$ as functions $\{0,1\}^{\log k} \to \mathbb{F}$ mapping indices (in bit representation) to field elements, and vice versa.

Decompose the point $z \in \mathbb{F}^{\log n}$ into two parts $z = (x, y) \in \mathbb{F}^{\log k} \times \mathbb{F}^{\log k}$. The prover sends $\mathsf{eval} \colon \{0,1\}^{\log k} \to \mathbb{F}$, which is purported to be the function $q(b) := \hat{m}(x, b)$. Observe that

$$\hat{q}(y) = \sum_{b \in \{0,1\}^{\log k}} \mathsf{eq}(y, b) \cdot q(b) = \sum_{b \in \{0,1\}^{\log k}} \mathsf{eq}(y, b) \cdot \hat{m}(x, b) = \hat{m}(x, y) = \hat{m}(z) ,$$

[9] Multiple claims can be batched into a single claim with standard techniques [RR24; CBBZ23].

where eq is the equality function on the Boolean hypercube (see Section 3.1). Since the verifier can check that $\widehat{\mathsf{eval}}(y) = v$ with a linear query, a dishonest prover must send $\mathsf{eval} \neq q$.

Consider a random index $i \in [\ell]$. Since $\mathsf{eval} \neq q$, we know that $\mathcal{C}(\mathsf{eval})$ and $\mathcal{C}(q)$ disagree on their i-th symbol with probability at least $\delta(\mathcal{C})$. The verifier can obtain the i-th coordinate of $\mathcal{C}(\mathsf{eval})$ using a linear query, but what about the i-th coordinate of $\mathcal{C}(q)$? Observe that

$$q(b) = \hat{m}(x, b) = \sum_{a \in \{0,1\}^{\log k}} \mathsf{eq}(x, a) \cdot m(a, b) = \mathsf{eq}_x^T M(\cdot, b)\,,$$

where $M(\cdot, b)$ denotes the b-th column of M and $\mathsf{eq}_x^T \in \mathbb{F}^k$ is the *equality coefficient vector* whose a-th entry is $\mathsf{eq}(x, a)$ (indices are in bit representation). Letting $U \in \mathbb{F}^{k \times \ell}$ denote the row-wise encoding of M, observe by linearity of $\mathcal{C}$ that

$$\mathsf{eq}_x^T U = \sum_{a \in \{0,1\}^{\log k}} \mathsf{eq}(x, a) \cdot \mathcal{C}\big(M(a, \cdot)\big) = \mathcal{C}(\mathsf{eq}_x^T M) = \mathcal{C}(q)\,,$$

where $M(a, \cdot)$ denotes the a-th row of M. Since G is δ_{UD}-close to U, it must be that $\mathsf{eq}_x^T G$ is δ_{UD}-close to $\mathcal{C}(q)$. By a triangle inequality, $\mathsf{eq}_x^T G$ must be δ_{UD}-far from $\mathcal{C}(\mathsf{eval})$.[10]

Hence, with probability at least δ_{UD}, the i-th entry of $\mathcal{C}(\mathsf{eval})$ disagrees with $\mathsf{eq}_x^T g_i = \hat{g}_i(x)$. The verifier can obtain $\hat{g}_i(x)$ with code switching, as described in Section 2.1; moreover, the spot check indices procedure to detect inconsistencies in combo can simultaneously be used to detect inconsistencies in eval. The full opening phase is given below, and also depicted in Fig. 1:

1. The verifier samples combination coefficients $r \in \mathbb{F}^k$.
2. The prover sends $\mathsf{combo}, \mathsf{eval} \in \mathbb{F}^k$, and the verifier checks that $\widehat{\mathsf{eval}}(y) = v$. In the honest case, $\mathsf{combo} \coloneqq r^T M$ and $\mathsf{eval} \coloneqq q$ as defined above.
3. The prover and verifier perform the following spot check procedure, repeated t times in parallel:
   (a) The verifier samples a random column $i \in [\ell]$.
   (b) The prover sends $\mathsf{col}_i \in \mathbb{F}^k$, and the verifier checks that $r^T \mathsf{col}_i = \mathcal{C}(\mathsf{combo})[i]$ and $\widehat{\mathsf{col}}_i(x) = \mathcal{C}(\mathsf{eval})[i]$. In the honest case, col_i is the i-th column of G.
   (c) The verifier checks that $\mathcal{C}(\mathsf{col}_i)$ agrees with the i-th column of F at a random location.

Remark 2.1. It suffices for r to be a random equality coefficient vector, which reduces the verifier's randomness complexity. If z is a random point chosen after the commitment phase (as is often the case), then $r = \mathsf{eq}_x$ can serve as the randomness. Doing this means that combo and eval are the same vector, which allows the prover to send one less oracle.

Linear evaluation claims. To facilitate recursion (Section 2.4), we will extend the above protocol to support linear evaluations, i.e., checking claims of the form $\langle m, w \rangle = v$ for some weight vector $w \in \mathbb{F}^n$ and field element $v \in \mathbb{F}$.[11] In order for the verifier to be efficient, we will restrict our attention to weight vectors w that have a succinct description.

[10] When working beyond the unique decoding radius (Section 2.3), we will exclusively use the optimization discussed in Remark 2.1. In this case, security follows from proximity gaps.

[11] A multilinear polynomial evaluation $\hat{m}(z)$ can be expressed as a linear evaluation with the weight vector eq_z.

The main idea is to view the weight vector w as a matrix $W \in \mathbb{F}^{k \times k}$ and run the (standard) sumcheck protocol over the rows of M and W. This results in a linear evaluation claim of the form $\langle m', w' \rangle = v'$, where $m' := \mathsf{eq}_x^T M$ and $w' := \mathsf{eq}_x^T W$ are (multilinear) extension rows of the input matrices; $x \in \mathbb{F}^{\log k}$ is randomness derived from the sumcheck protocol.

The rest of the opening phase is the same as before, except the prover sends $\mathsf{eval} := \mathsf{eq}_x^T M$, and the verifier checks that $\langle \mathsf{eval}, \mathsf{eq}_x^T W \rangle = v$ with a single linear query. As discussed in Remark 2.1, $r = \mathsf{eq}_x$ may serve as the randomness so that combo and eval are the same vector.

2.3 Beyond the unique decoding radius

Recall that our protocol's query complexity is $t = \lambda / -\log(1 - \delta_{\mathsf{UD}}^2)$, which does not quite match Theorem 1.1's guarantee. This is because G and M are defined by correcting less than a δ_{UD}-fraction of errors, where δ_{UD} is the unique decoding radius of $\mathcal{C}$. We obtain Theorem 1.1 by extending our protocol to work in the *list decoding* regime. In more detail, our protocol is instantiated by distance parameters $\delta_{\mathsf{col}}, \delta_{\mathsf{row}} \in (0, \delta)$. Before we define G and M, let us first explain why the following straw man definitions are insufficient.

- Suppose we define G to be the column-wise decoding of F under $\mathcal{C}$, correcting at most a δ_{col}-fraction of errors in each column. If δ_{col} is beyond the unique decoding radius of $\mathcal{C}$, there may be (exponentially) many choices of G.

- Even if G were fixed, suppose we define M to be the decoding of G under the interleaved k-fold interleaving of $\mathcal{C}$, correcting at most a δ_{row}-fraction of columns. If δ_{row} is beyond the unique decoding radius of $\mathcal{C}$, there may be many choices of M.

We address both issues with *out-of-domain sampling* [BGKS20], which enables setting δ_{col} and δ_{row} to be within the list decoding radius of $\mathcal{C}$ and the k-fold interleaving of $\mathcal{C}$, respectively. Unlike prior work, we leverage "two rounds" of out-of-domain sampling, as we now explain.

After the prover sends F in the commitment phase, the verifier samples a random point $\zeta \in \mathbb{F}^{\log k}$. The prover responds with a point oracle $\mathsf{ood} \in \mathbb{F}^{\ell}$, which purportedly contains the multilinear evaluations of each column of G at ζ. The verifier then samples another random point $\zeta' \in \mathbb{F}^{\log n}$, and the prover responds with $\mu \in \mathbb{F}$, which is purported to be $\hat{m}(\zeta')$. Assuming the field $\mathbb{F}$ is sufficiently large, it is not hard to show the following binding conditions hold with high probability:

- The codewords within δ_{col}-distance of any column of F *disagree* on the multilinear evaluations of their underlying messages at ζ (see Section 3.4). In this case, we define G to be the unique column-wise decoding of F under $\mathcal{C}$ (correcting less than a δ_{col}-fraction of errors in each column) whose multilinear evaluations at ζ are consistent with ood.

- The interleaved codewords within δ_{row}-distance of G (viewing each column of G as a symbol) disagree on the multilinear evaluations of their underlying messages at ζ'. In this case, we define M to be the unique decoding of G under the k-fold interleaving of $\mathcal{C}$ (correcting less than a δ_{row}-fraction of columns) whose multilinear evaluation at ζ' is equal to μ.

The commitment phase induces a new evaluation claim $\hat{m}(\zeta') = \mu$, which the verifier must additionally check in the opening phase. Moreover, for each column i sampled in the spot check procedure, the verifier must check that the multilinear extension of col_i evaluated at ζ is equal to $\mathsf{ood}[i]$; this can be done with a linear query to col_i and a point query to ood. Finally, our analysis

requires $\mathcal{C}$ to exhibit proximity gaps (with mutual correlated agreement [ACFY25]) up to distance δ_{row}.[12] Aiming for $2^{-\lambda}$ soundness error, the number of repetitions is

$$t := \frac{\lambda}{-\log(1 - \delta_{\mathsf{row}} \cdot \delta_{\mathsf{col}})} .$$

Remark 2.2 (query optimizations)**.** Our out-of-domain sampling strategy requires the verifier to issue an additional t point queries to ood. We introduce the following optimization, which is motivated by the fact that, looking ahead, it will be desirable to minimize the verifier's point query complexity (even if this increases its linear query complexity). By sending ood as a linear oracle, we only incur t linear queries to ood. In fact, since $\mathsf{ood} \in \mathcal{C}$ is a codeword when the prover is honest, it suffices to send as a linear oracle the decoding of ood (i.e., the multilinear evaluations of each column of M at ζ).

2.4 Compiling linear queries to point queries

So far we have only described an interactive oracle PCS in the linear query model. Since the col_i oracles can be collected into a single linear oracle, the prover only sends $O(1)$ oracles, each of size at most $O(\lambda \cdot \sqrt{n})$.[13]

In order to obtain a commitment scheme in the point query model, we recursively invoke our construction. That is, the prover commits to each linear oracle by *encoding* it with a (smaller) tensor code. For each linear query issued by the verifier, the prover responds with a claimed evaluation. Finally, the verifier checks these linear evaluation claims by running the opening phase.[14] We emphasize that, since the committed oracles are of $O(\lambda \cdot \sqrt{n})$ size, the recursively defined prover sends $O(1)$ linear oracles of $O_\lambda(\sqrt[4]{n})$ size. Invoking our commitment scheme roughly $\log \log n$ times (without Remark 2.2's optimization), on progressively smaller linear oracles, we obtain a commitment scheme for linear evaluations where the prover sends $O(1)$ linear oracles of size at most $O_\lambda(1)$. At this point, the prover simply sends these $O_\lambda(1)$-sized vectors as non-oracle messages, and the verifier may compute the answers to its own linear queries.

However, there is a catch: to enable this approach, the verifier must be able to compute an *explicit* description of this linear query. More specifically, this involves evaluating the multilinear extensions of linear queries issued throughout the protocol. The types of linear queries that we have are:

- Equality coefficient vectors of the form eq_x. Their multilinear extensions can be efficiently evaluated: its evaluation at a point y is simply $\mathsf{eq}(x, y)$.
- Combination coefficients r. Using Remark 2.1, these are also equality coefficient vectors.
- The original weight vector w. When testing evaluation claims against the multilinear extension $\hat{m}$, this is an equality coefficient vector.

[12] Loosely speaking, it is known that general codes exhibit proximity gaps up to the so-called "one-and-a-half Johnson bound" [BGKS20; GKL24; Zei24]. For Reed–Solomon codes, δ_{row} may be arbitrarily close to the Johnson bound (albeit with a larger error) [BCIKS20]. For many codes, it seems reasonable to conjecture that δ_{row} can be close to the minimal distance, although this is not necessarily true [BGKS20; DG25]. See also `https://proximityprize.org`.

[13] If we use an appropriately skewed tensor code, this can be as low as $O(\sqrt{\lambda \cdot n})$.

[14] More precisely, we require a *batch* opening phase for checking linear evaluation claims against multiple commitments. See Section 5.4 for the construction.

- Rows of the generator matrix of $\mathcal{C}$. We refer to their multilinear extensions as *right-extensions* of $\mathcal{C}$; see Section 7 for details.

In summary, the verifier's only non-trivial task is evaluating the right-extensions of $\mathcal{C}$. Ideally, we would instantiate our protocol with Reed–Solomon codes, which admit efficient right-extensions (this fact is implicit in [ACFY25; NA25]). Unfortunately, this would result in a PCS with quasilinear prover time, owing to the quasilinear encoding time of Reed–Solomon codes.

Nonetheless, observe that the commitment schemes used within recursion *can* be instantiated with Reed–Solomon codes, since they only encode messages of size at most $O(\lambda \cdot \sqrt{n})$. This leaves the outer commitment scheme, which must be instantiated with a linear-time encodable code. Since we are not aware of such codes with efficient right-extensions, we instead *delegate* this computation to the prover. This can be done with a generic proof system, but the following recipe leads to our desired efficiency measures.

Given a holographic PIOP for NP (e.g., [Set19; GWC19; CHMMVW20; CBBZ23]) with mild efficiency requirements, we compile it into a holographic IOP for NP, using an instantiation of our PCS that exclusively uses Reed–Solomon codes. The resulting IOP has quasilinear prover time, which nonetheless suffices because the right-extension complexity of a code is at most linear in its encoding time; hence, the size of the delegated computation is at most $O(\lambda \cdot \sqrt{n})$.

Query reduction via high-distance codes. As described so far, there are roughly $\log \log n$ steps of recursion, and the verifier makes $O(\lambda)$ queries in each step. Hence, the verifier makes $\Omega(\lambda \cdot \log \log n)$ queries in total, which falls short of our desired query complexity $O(\lambda)$ (let alone our aim for an explicit small constant).

To avoid this cost, following [RR25; ACFY24; ACFY25] we observe that, as the recursion progresses, the linear oracles become dramatically smaller, and we can afford to encode them using a code with lower rate and much higher minimal distance. Specifically, we instantiate the commitment schemes used within recursion with (Reed–Solomon) codes of rate $1/\log^{\omega(1)}(n)$. This enables us to set δ_{row} and δ_{col} to be $1 - 1/\log^{\omega(1)}(n)$. Consequently, only $o(\lambda/\log \log n)$ queries are required in all but the first round.

2.5 Round-by-round security

In Lemma A.1 we prove that combining a polynomial IOP with an interactive oracle PCS yields an IOP. Such an IOP can then be used to obtain a hash-based SNARG [Kil92; Mic00; BCS16]. This transformation requires the IOP to satisfy *state-restoration security* [BCS16; CY24]; in this paper, we focus on the stronger, more convenient notion of *round-by-round soundness* [CCHLRR18].

Informally, an IOP is round-by-round sound if there exists a well-defined bad event in each round that a (dishonest) prover may hope to cause and should be avoided. The verifier should avoid these bad events separately, with all but negligible error probability, in each round. For an interactive oracle PCS, we establish the round-by-round security of the commitment and opening phases separately.

- The commitment phase is not comparable to an IOP, so the notion of round-by-round soundness is not applicable. We introduce the notion of *round-by-round binding*, which is a natural analogue of round-by-round soundness; see Section 4 for a formal definition.

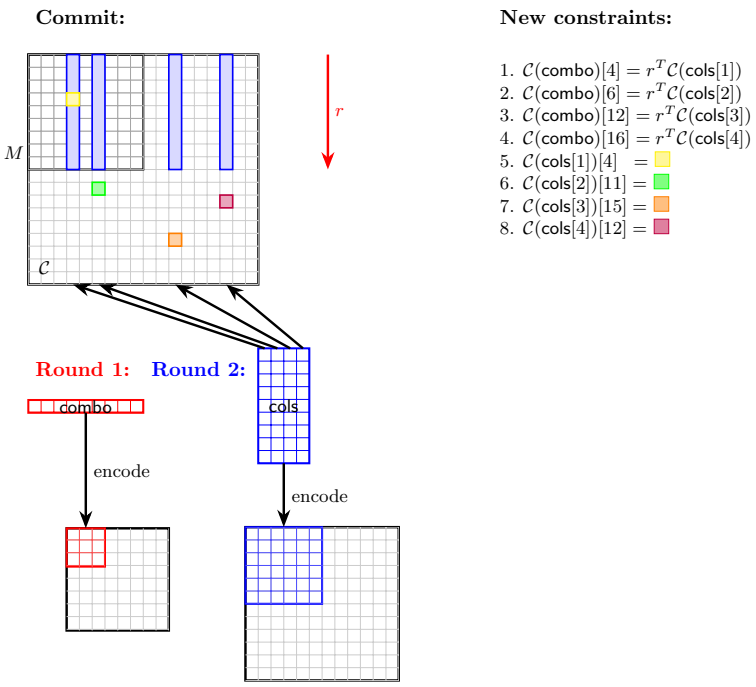


Figure 1: Graphical depiction of one iteration of TensorSwitch. In the commitment phase, the prover sends a tensor encoding of the message. In Round 1 (depicted in red), the prover computes combo, a random combination (with coefficients r) of the message rows, and encodes combo with a new tensor code. In Round 2 (depicted in blue), the verifier samples t (above, 4) columns, denoted cols, and the prover encodes cols with a new tensor code. Finally, the verifier samples one row index per column, and the prover opens the original committed tensor at each index. This induces constraints on the encodings of cols and combo, which are checked in the next iteration.

- The opening phase is an IOP (where the verifier also has query access to its input). Establishing its round-by-round soundness is mostly straightforward except for the following issue. Recall from our analysis that the verifier performs the following spot check procedure, repeated t times in parallel: sample a random column $i \in [\ell]$, receive col_i from the prover, perform some checks on col_i, and check the consistency between $\mathcal{C}(\mathsf{col}_i)$ and the i-th column of F at a random point. The issue is that parallel repetition reduces the round-by-round soundness error in a sub-optimal way (see [CCHLRR18, Proposition 5.5] and [AFK23, Theorem 5]). In particular, since the spot check has two rounds of interaction, the number of repetitions t needs to be doubled (from what is required to achieve standard soundness) to achieve round-by-round soundness error $2^{-\lambda}$.

  We can avoid this constant overhead by having the verifier *over-sample* $s = \tilde{O}(\lambda)$ columns (rather than just t). Recall that a random column is bad with probability δ_{row}. By a Chernoff bound, a random subset of s columns will contain a $(\delta_{\mathsf{row}} - o(1))$-fraction of columns. After the prover sends the col_i oracles, the verifier subsamples $(1 + o(1)) \cdot t$ selected indices to perform the spot check.

In the technical sections, we prove that our protocols satisfy round-by-round security.

Knowledge soundness. When the code $\mathcal{C}$ has an efficient error-tolerant (list) decoder, as is the case for Reed-Solomon codes [GS99], we further prove that our protocols have *round-by-round knowledge soundness* [BCFW25]; assuming the PIOP also satisfies this notion, the resulting hash-based SNARG has *straightline knowledge soundness* in the ROM.[15]

In the case where $\mathcal{C}$ is not known to have an efficient error-tolerant decoder, we do not know how to obtain a straightline extractor. On the other hand, we believe that our protocols satisfy certain notions of rewinding knowledge soundness [BMMS25a]; we leave proving this to future work.

[15] Only the column code needs an efficient decoder, if we use a tensor of different codes.

3 Preliminaries

We define objects and state results that we use in this paper. We use the following notation.

- The set of powers of two is $2^{\mathbb{N}} \coloneqq \{2^n : n \in \mathbb{N}\}$.
- The preimage set of $t \in T$ under a function $f\colon S \to T$ is $f^{-1}(t) \coloneqq \{s \in S : f(s) = t\}$.
- For a finite set S and two functions $f, g\colon S \to \mathbb{F}$, let $\Delta(f, g) \in [0, 1]$ denote the fractional Hamming distance between f and g (the fraction of points in which they disagree).
- For $n \in 2^{\mathbb{N}}$, let $\mathsf{bin}\colon [n] \to \mathbb{F}^{\log n}$ be the function which maps $i \in [n]$ to its bit representation (embedded in $\mathbb{F}$).

Lemma 3.1 (Ore–DeMillo–Lipton–Schwartz–Zippel lemma)**.** *Let $\hat{p} \in \mathbb{F}^{<d}[\mathtt{X}_1, \ldots, \mathtt{X}_m]$ be a non-zero polynomial. Then,*

$$\Pr_{\boldsymbol{x} \leftarrow \mathbb{F}^m}\left[\hat{p}(\boldsymbol{x}) = 0\right] \leq \frac{m \cdot (d-1)}{|\mathbb{F}|}\,.$$

Lemma 3.2 (Chernoff–Hoeffding theorem)**.** *Let $X_1, \ldots, X_n$ be i.i.d. Bernoulli random variables with probability α and $\varepsilon > 0$. Then*

$$\Pr\left[\frac{1}{n}\sum_{i \in [n]} X_i \geq \alpha + \varepsilon\right] \leq e^{-D(\alpha+\varepsilon\|\alpha)\cdot n} \leq e^{-2\cdot\varepsilon^2\cdot n}\,,$$

where $D(x\|y) \coloneqq x \cdot \ln\left(\frac{x}{y}\right) + (1-x)\cdot\ln\left(\frac{1-x}{1-y}\right)$.

Indexed relations. An *indexed relation* is a set of triples $\mathcal{R} = \{(\mathbb{i}, \mathbb{x}, \mathbb{w})\}$ where $\mathbb{i}$ is the index, $\mathbb{x}$ is the instance, and $\mathbb{w}$ is the witness. Let $\mathcal{L}(\mathcal{R}) \coloneqq \{(\mathbb{i}, \mathbb{x}) : \exists \mathbb{w}, (\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}\}$ be the language induced by $\mathcal{R}$. Any (standard) relation may be viewed as an indexed relation with empty indices $\mathbb{i} = \bot$.

Promise relations. A *promise relation* is a pair of relations $\mathcal{R} = (\mathcal{R}^{\text{YES}}, \mathcal{R}^{\text{NO}})$ where $\mathcal{L}(\mathcal{R}^{\text{YES}})$ is disjoint from $\mathcal{L}(\mathcal{R}^{\text{NO}})$. Any relation $\mathcal{R}$ may be viewed as a promise relation where $\mathcal{R}^{\text{YES}} = \mathcal{R}$ and $\mathcal{R}^{\text{NO}} = \overline{\mathcal{R}}$.

Field operations. We measure algorithmic runtimes in terms of finite field operations. As they are the most expensive, we distinguish between field multiplications and *basic field operations*, which include addition, subtraction, and sampling of random elements. Following [HJRRR25, Footnote 4] we also assume access to a constant-sized set of elements with which multiplication is a basic field operation. We also consider *linear field operations*, which consist of addition, subtraction, and scalar multiplication.

3.1 Multilinear extensions

Definition 3.3. *For a function $f\colon \{0,1\}^m \to \mathbb{F}$, the* **multilinear extension of** *f, denoted $\hat{f}\colon \mathbb{F}^m \to \mathbb{F}$, is the unique multilinear polynomial that agrees with f on $\{0,1\}^m$. The polynomial $\hat{f}$ can be explicitly defined as*

$$\hat{f}(x) \coloneqq \sum_{b \in \{0,1\}^m} \mathsf{eq}(x, b) \cdot f(b),$$

where $\mathsf{eq}(x, b) \coloneqq \prod_{i=1}^{m} \big(x_i \cdot b_i + (1 - x_i) \cdot (1 - b_i)\big)$.

Fact 3.4 ([ARZR25, Remark 1.7])**.** There exists an algorithm which, given the evaluations of a function $f\colon \{0,1\}^m \to \mathbb{F}$ and a point $x \in \mathbb{F}^m$, computes $\hat{f}(x)$ in 2^m field multiplications and $O(2^m)$ basic field operations.

Fact 3.5. There exists an algorithm which, given a point $x \in \mathbb{F}^m$, computes the evaluations of $\mathsf{eq}(x, \cdot)$ on $\{0,1\}^m$ in 2^m multiplications and $O(2^m)$ basic field operations.

Definition 3.6. *For a function $g\colon S \to \mathbb{F}^{\{0,1\}^m}$ and point $\gamma \in \mathbb{F}^k$, the* **folding of g by γ** *is the function $\mathsf{Fold}_\gamma(g)\colon S \to \mathbb{F}^{\{0,1\}^{m-k}}$ defined by*

$$\mathsf{Fold}_\gamma(g)(a) \coloneqq \hat{f}(\gamma, \cdot) \text{ where } f \coloneqq g(a)\,.$$

3.2 Interactive oracle proofs

An *oracle class* $\mathcal{O}$ is a set of functions $\mathcal{Q} \to \Sigma$, where $\mathcal{Q}$ is the *query set* and Σ is the *alphabet*. We say that an oracle $\Pi \in \mathcal{O}$ *agrees* with a list of evaluation claims $\mathsf{ans} = (\sigma_i, \tau_i)_{i \in [q]}$ if, for each $i \in [q]$, it holds that $\Pi(\sigma_i) = \tau_i$.

In this work we consider the following classes of answer functions.

- *Point oracles.* For an alphabet Σ and length $n \in \mathbb{N}$, an oracle in Point_n is identified with a string $\Pi \in \Sigma^n$. It maps an index $i \in [n]$ to the symbol $\Pi(i)$.
- *Linear oracles.* For a field $\mathbb{F}$ and length $n \in \mathbb{N}$, an oracle in Lin_n is identified with a vector $\Pi \in \mathbb{F}^n$. It maps a description $[\![w]\!]$ of a weight vector $w \in \mathbb{F}^n$ to the field element $\langle \Pi, w \rangle$.
- *Multilinear polynomial oracles.* For a field $\mathbb{F}$ and number of variables $m \in \mathbb{N}$, an oracle in MLP_m is identified with a function $\Pi\colon \{0,1\}^m \to \mathbb{F}$. It maps a point $\boldsymbol{x} \in \mathbb{F}^m$ to the field element $\hat{\Pi}(\boldsymbol{x})$.

Functional IOPs. A (public-coin, holographic) functional interactive oracle proof (IOP) for an indexed promise relation $\mathcal{R}$ is a pair $\mathsf{FIOP} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$ that works as follows.

- We consider the following parameters.
  - Number of rounds $\mathsf{k} \in \mathbb{N}$, number of index oracles $\mathsf{ni} \in \mathbb{N}$, and number of instance oracles $\mathsf{n} \in \mathbb{N}$.
  - Oracle class $\mathcal{O}$ with query set $\mathcal{Q}$ and alphabet Σ. (In full generality, each index, instance, and proof oracle may use a different oracle class.)
  - For each $i \in [\mathsf{k}]$, a non-oracle message length $\mathsf{m}_i \in \mathbb{N}$ and randomness length $\mathsf{r}_i \in \mathbb{N}$.
- *Inputs.* For all $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, we assume the instance $\mathbb{x}$ can be parsed into a *non-oracle instance* x and *instance oracles* $\mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}} \in \mathcal{O}$.
- *Offline phase.* The indexer $\mathbf{I}$ is a deterministic algorithm that receives as input an index $\mathbb{i}$. It outputs *index oracles* $\mathrm{I}_1, \ldots, \mathrm{I}_{\mathsf{ni}} \in \mathcal{O}$ (when the index is clear from context, we omit the indexer $\mathbf{I}$ and write I_i directly).

- *Interaction phase.* The prover $\mathbf{P}$ is an interactive algorithm that receives $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$, and the verifier $\mathbf{V}$ is an interactive algorithm that receives as (explicit) input x along with query access to index oracles $\mathrm{I}_1, \ldots, \mathrm{I}_{\mathsf{ni}}$ and instance oracles $\mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}}$.[16] They interact over k rounds as follows. In each round $i \in [\mathsf{k}]$, the prover sends an oracle $\Pi_i \in \mathcal{O}$ and message $\pi_i \in \{0,1\}^{\mathsf{m}_i}$, and the verifier responds with uniformly sampled randomness $\alpha_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.

- *Decision phase.* After k rounds of interaction, the verifier issues queries as follows. For any query $\sigma \in \mathcal{Q}$ issued to an index, instance, or proof oracle Π, the verifier receives the answer $\Pi(\sigma)$. Afterwards, the verifier accepts or rejects; without loss of generality, this is a deterministic function of its input and the transcript.

- *Completeness.* We require that, for every $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}^{\mathrm{YES}}$,

$$\Pr_{\alpha_1, \ldots, \alpha_{\mathsf{k}}} \left[\mathbf{V}^{\mathrm{I}_1, \ldots, \mathrm{I}_{\mathsf{ni}}, \mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}}, \Pi_1, \ldots, \Pi_{\mathsf{k}}}(x, \pi_1, \alpha_1, \ldots, \pi_{\mathsf{k}}, \alpha_{\mathsf{k}}) \text{ accepts} \right] = 1 \,.$$

  Above, the oracles $\Pi_1, \ldots, \Pi_{\mathsf{k}}$ and messages $\pi_1, \ldots, \pi_{\mathsf{k}}$ are computed according to $\mathbf{P}(\mathbb{i}, \mathbb{x}, \mathbb{w})$.

Round-by-round soundness. We prove that our protocols satisfy *round-by-round soundness* [CCHLRR18], a strengthening of (standard) soundness that ensures Fiat–Shamir security. When possible, we further prove that our protocols satisfy *round-by-round knowledge soundness* [BCFW25], an analogous strengthening of (standard) knowledge soundness.

Definition 3.7. *Let* $\mathsf{FIOP} = (\mathbf{P}, \mathbf{V})$ *be a functional-IOP for a promise relation* $\mathcal{R}$*. A* **state function** *for* FIOP *is a (possibly inefficient) function* State*, that, on input an index* $\mathbb{i}$*, instance* $\mathbb{x}$*, interaction transcript* tr*, and state function witness* w*, outputs a bit and has the following properties.*

- Empty transcript: $\mathsf{State}(\mathbb{i}, \mathbb{x}, \mathsf{tr}, \mathsf{w}) = 1$ *if and only if* $(\mathbb{i}, \mathbb{x}, \mathsf{w}) \in \overline{\mathcal{R}}^{\mathrm{NO}}$.

- Full transcript: *if* $\mathsf{tr} = ((\Pi_1, \pi_1), \alpha_1, \ldots, (\Pi_{\mathsf{k}}, \pi_{\mathsf{k}}), \alpha_{\mathsf{k}})$ *is a full transcript, then* $\mathsf{State}(\mathbb{i}, \mathbb{x}, \mathsf{tr}, \mathsf{w}) = 1$ *if and only if* $\mathbf{V}$ *accepts.*

Definition 3.8. FIOP *has* **round-by-round knowledge soundness errors** $(\varepsilon_1, \ldots, \varepsilon_{\mathsf{k}})$ **with extraction times** $(\mathsf{et}_1, \ldots, \mathsf{et}_{\mathsf{k}})$ *if there exists a state function* State *for* FIOP *and a deterministic extractor* $\mathbf{E}$ *with the following property: for every index* $\mathbb{i}$*, instance* $\mathbb{x}$*, round index* $i \in [\mathsf{k}]$*, interaction transcript* $\mathsf{tr} = ((\Pi_1, \pi_1), \alpha_1, \ldots, (\Pi_{i-1}, \pi_{i-1}), \alpha_{i-1})$*, and prover message* (Π_i, π_i)*,*

$$\Pr_{\alpha_i} \left[\exists \mathsf{w} : \begin{array}{c} \mathsf{State}\big(\mathbb{i}, \mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| (\Pi_i, \pi_i) \| \alpha_i, \mathsf{w})\big) = 0 \\ \wedge\, \mathsf{State}\big(\mathbb{i}, \mathbb{x}, \mathsf{tr} \| (\Pi_i, \pi_i) \| \alpha_i\big) = 1 \end{array} \right] \leq \varepsilon_i(\mathbb{x}) \,.$$

Above, the extractor $\mathbf{E}$ *runs in time at most* et_i*. The* **(maximum) round-by-round knowledge soundness error** *is* $\max_i \varepsilon_i$ *and the* **(total) extraction time** *is defined to be* $\sum_{i \in [\mathsf{k}]} \mathsf{et}_i$*.*

Remark 3.9. When a functional-IOP has round-by-round knowledge soundness ε but the extractor is inefficient, we can omit the extraction time parameter and say that the protocol has *round-by-round soundness error* ε; this is equivalent to the standard definition of round-by-round soundness.

[16] Typically, the verifier needs to know parameters that are derived from the index. This can be realized with a "verification key" computed by the indexer, which is then passed to the verifier as (explicit) input. For simplicity, we omit this.

3.3 Codes

We state several definitions and facts about (error-correcting) codes.

Definition 3.10. *A* **code** *over an alphabet Σ with message length k and codeword length ℓ is an injective map $\mathcal{C} \colon \Sigma^k \to \Sigma^\ell$. A code $\mathcal{C}$ is a* **linear code** *if $\Sigma = \mathbb{F}$ is a field and $\mathcal{C}$ is a linear transformation over $\mathbb{F}$; the* **generator matrix** *$G_\mathcal{C} \in \mathbb{F}^{\ell \times k}$ is such that $\mathcal{C}(m) = G_\mathcal{C} m$. The* **minimum distance** *of a code $\mathcal{C}$ is $\delta(\mathcal{C}) \coloneqq \min_{u \neq v} \Delta(u, v)$ and its rate is k/ℓ.*

We often view $\mathcal{C} \subseteq \Sigma^\ell$ as the map's image with codewords $u \in \mathcal{C}$.

Definition 3.11. *For a code $\mathcal{C} \subseteq \Sigma^\ell$, function $f \colon [\ell] \to \Sigma$, and proximity bound $\delta \in [0, 1]$, we define the* **list decoding** *of f within δ as*

$$\Lambda(\mathcal{C}, f, \delta) \coloneqq \{u \in \mathcal{C} : \Delta(f, u) \leq \delta\}.$$

We also define $|\Lambda(\mathcal{C}, \delta)| \coloneqq \max_f |\Lambda(\mathcal{C}, f, \delta)|$.

3.3.1 Algorithmic measures

Let $\mathcal{C} \colon \Sigma^k \to \Sigma^\ell$ be a code and $\delta \in [0, 1]$ be a proximity bound. We consider the following algorithmic measures.

- $\mathsf{enc}_\mathcal{C}$ is the number of field operations required to encode a message $m \in \Sigma^k$ into its corresponding codeword $\mathcal{C}(m)$. When $\mathcal{C}$ is a linear code, these are required to be linear field operations.[17]
- $\mathsf{errdec}_\mathcal{C}(\delta)$ is the number of field operations required to *list error decode* $\mathcal{C}$ with up to a δ fraction of errors (Definition 3.12).
- $\mathsf{eradec}_\mathcal{C}(\delta)$ is the number of field operations required to *erasure decode* $\mathcal{C}$ with up to a δ fraction of erasures (Definition 3.13); every linear code supports efficient erasure correction (Fact 3.14).

Definition 3.12. *A code $\mathcal{C} \colon \Sigma^k \to \Sigma^\ell$ supports* **list error decoding in time** $\mathsf{errdec}_\mathcal{C}$ *if there exists a deterministic algorithm $\mathbf{E}_\mathsf{err}$ (the error corrector) parameterized by proximity bound $\delta \in [0, 1]$ that, given function $f \colon [\ell] \to \Sigma$, outputs $(u, \mathcal{C}^{-1}(u))_{u \in \Lambda(\mathcal{C}, f, \delta)}$. Moreover, $\mathbf{E}_\mathsf{err}$ performs at most $\mathsf{errdec}_\mathcal{C}(\delta)$ field operations.*

Definition 3.13. *A code $\mathcal{C} \colon \Sigma^k \to \Sigma^\ell$ supports* **erasure decoding in time** $\mathsf{eradec}_\mathcal{C}$ *if there exists a deterministic algorithm $\mathbf{E}_\mathsf{era}$ (the erasure corrector) parameterized by proximity bound $\delta \in [0, 1]$ that, given function $f \colon [\ell] \to \Sigma \cup \{\perp\}$ where $\left|f^{-1}(\perp)\right| < \delta \cdot \ell$:*

- *if possible, $\mathbf{E}_\mathsf{era}$ outputs a message $m \in \mathcal{C}$ such that $f(i) = \mathcal{C}(m)(i)$ for all $i \in [\ell] \setminus f^{-1}(\perp)$.*
- *if no such codeword exists, $\mathbf{E}_\mathsf{era}$ outputs $\perp$.*

Moreover, $\mathbf{E}_\mathsf{era}$ performs at most $\mathsf{eradec}_\mathcal{C}(\delta)$ field operations.

Fact 3.14. Every linear code $\mathcal{C} \colon \mathbb{F}^k \to \mathbb{F}^\ell$ supports erasure correction in time $\mathsf{eradec}_\mathcal{C}(\delta) = O(\ell^3)$.

Lemma 3.15. *Let $\mathcal{C} \colon \mathbb{F}^k \to \mathbb{F}^\ell$ be a linear code and $t \in \mathbb{N}$. Given indices $x_1, \ldots, x_t \in [\ell]$ and scalars $\gamma_1, \ldots, \gamma_t$, the vector $\sum_{i \in I} \gamma_i \cdot G_\mathcal{C}(i, \cdot)$ is computable with $\mathsf{enc}_\mathcal{C}$ linear field operations and $\ell - k + t$ field additions.*

Proof. Observe that $\sum_{i=1}^t \gamma_i \cdot G_\mathcal{C}(i, \cdot) = G_\mathcal{C}^T \sum_{i \in I} \gamma_i \cdot e_{x_i}$, where $e_x \in \mathbb{F}^k$ denotes the x-th unit vector. The sum $\sum_{i \in I} \gamma_i \cdot e_{x_i}$ is computable with t field additions. Left-multiplication by $G_\mathcal{C}^T$ is computable with $\mathsf{enc}_\mathcal{C}$ linear field operations, plus $\ell - k$ field additions, by the transposition principle [Bor57; Ber]. □

[17] This is the case for linear codes of practical interest, and may be assumed in general without loss of asymptotic efficiency [BCS97, Theorem 13.1].

3.3.2 Product codes

Definition 3.16. *For a finite set S, the* **S-interleaving** *of a code $\mathcal{C}\colon \Sigma^k \to \Sigma^\ell$ is the code $\mathcal{C}^S\colon (\Sigma^S)^k \to (\Sigma^S)^\ell$, where the encoding of a message $\boldsymbol{m}$ is obtained by representing $\boldsymbol{m}$ as a matrix in $\Sigma^{|S|\times k}$, and then encoding each row.*

For an interleaved codeword $\boldsymbol{u} \in \mathcal{C}^S$ and element $a \in S$, we write $u_a \in \mathcal{C}$ to denote the codeword defined by $u_a(i) \coloneqq \boldsymbol{u}(i)(a)$.

Lemma 3.17 ([GGR11])**.** *Let $\mathcal{C}$ be a code and $\delta \in \left[0, \delta(\mathcal{C})\right)$ be a proximity bound. Let $b \coloneqq \left\lceil \frac{\delta}{\delta(\mathcal{C})-\delta} \right\rceil$ and $r \coloneqq \left\lceil \log \frac{\delta(\mathcal{C})}{\delta(\mathcal{C})-\delta} \right\rceil$. For every finite set S, $|\Lambda(\mathcal{C}^S, \delta)| \leq \binom{b+r}{r} \cdot |\Lambda(\mathcal{C}, \delta)|^r$.*

Definition 3.18. *The* **tensor (product) code** *of linear codes $\mathcal{C}_{\mathsf{row}}\colon \mathbb{F}^{k_{\mathsf{row}}} \to \mathbb{F}^{\ell_{\mathsf{row}}}$ and $\mathcal{C}_{\mathsf{col}}\colon \mathbb{F}^{k_{\mathsf{col}}} \to \mathbb{F}^{\ell_{\mathsf{col}}}$ is the linear code $\mathcal{C}_{\mathsf{col}} \otimes \mathcal{C}_{\mathsf{row}}\colon \mathbb{F}^{k_{\mathsf{row}} \times k_{\mathsf{col}}} \to \mathbb{F}^{\ell_{\mathsf{row}} \times \ell_{\mathsf{col}}}$, where the encoding of a message M is obtained by encoding each row of M with $\mathcal{C}_{\mathsf{row}}$ and then encoding each resulting column with $\mathcal{C}_{\mathsf{col}}$.*

3.4 Out-of-domain sampling

We recall (a specialized instantiation of) out-of-domain sampling [BGKS20].

Lemma 3.19. *Let $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ be a code where k is a power of two, $f\colon [\ell] \to \mathbb{F}$ be a function, and $\delta \in [0,1]$ be a proximity bound. Then,*

$$\Pr_{\zeta \leftarrow \mathbb{F}^{\log k}}\left[\exists \text{ distinct } u_1, u_2 \in \Lambda(\mathcal{C}, f, \delta) : \hat{m}_1(\zeta) = \hat{m}_2(\zeta)\right] \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2}{2} \cdot \frac{\log k}{|\mathbb{F}|},$$

where $m_1 \coloneqq \mathcal{C}^{-1}(u_1)$ and $m_2 \coloneqq \mathcal{C}^{-1}(u_2)$ denote the decodings of codewords.

Proof. Fix distinct codewords $u_1, u_2 \in \Lambda(\mathcal{C}, f, \delta)$. Since $u_1 \neq u_2$ and $\mathcal{C}$ is injective, it must be that $m_1 \neq m_2$. By Lemma 3.1, the probability (over ζ) that $\hat{m}_1(\zeta) = \hat{m}_2(\zeta)$ is at most $\frac{\log k}{|\mathbb{F}|}$. The statement follows from a union bound over all $\binom{|\Lambda(\mathcal{C},f,\delta)|}{2} \leq \frac{|\Lambda(\mathcal{C},\delta)|^2}{2}$ pairs of distinct codewords in $\Lambda(\mathcal{C}, f, \delta)$. □

3.5 Mutual correlated agreement

We recall the definition of mutual correlated agreement [ACFY25].

Definition 3.20. *Let $d \in \mathbb{N}$ be an arity, $\mathcal{C} \subseteq \mathbb{F}^\ell$ be a linear code, and $\mathsf{PG}\colon D_{\mathsf{PG}} \to \mathbb{F}^d$ be a function. The* **mutual correlated agreement error of $\mathcal{C}$ with respect to PG** *is defined as*

$$\varepsilon_{\mathsf{PG}}(\mathcal{C}, \delta) \coloneqq \max_{f_1,\ldots,f_d\colon [\ell]\to\mathbb{F}} \Pr_{\substack{\rho \leftarrow D_{\mathsf{PG}} \\ \boldsymbol{\gamma} := \mathsf{PG}(\rho)}} \left[\exists S \subseteq [\ell] : \begin{array}{c} |S| \geq (1-\delta)\cdot \ell \\ \wedge\, \exists u \in \mathcal{C} : \sum_{i\in[d]} \gamma_i \cdot f_i(S) = u(S) \\ \wedge\, \exists i \in [d] : \forall u \in \mathcal{C}, f_i(S) \neq u(S) \end{array}\right].$$

We say that a code $\mathcal{C}$ has correlated agreement up to distance δ, if there exists an efficiently computable PG, such that $\varepsilon_{\mathsf{PG}}(\mathcal{C}, \delta)$ is negligible in a security parameter λ.

Definition 3.21. *Let $\ell \in \mathbb{N}$ and $\mathbb{F}$ be a field. The function $\mathsf{PG}_\ell\colon \mathbb{F}^{\ell-1} \to \mathbb{F}^\ell$ is defined to be $\mathsf{PG}_\ell(\gamma_2, \ldots, \gamma_\ell) \coloneqq (1, \gamma_2, \ldots, \gamma_\ell)$.*

Fact 3.22. Let $\mathcal{C}$ be a linear code and $\mathsf{PG}\colon \mathbb{F} \to \mathbb{F}^2$ be the function defined by $\mathsf{PG}(\gamma) \coloneqq (1-\gamma, \gamma)$. It holds that $\varepsilon_{\mathsf{PG}}(\mathcal{C}, \delta) = \varepsilon_{\mathsf{PG}_2}(\mathcal{C}, \delta)$.

4 Functional commitments with oracles

We define information-theoretic commitments in the (functional) IOP model. For an oracle class $\mathcal{O}$ (Section 3.2), an *interactive $\mathcal{O}$-commitment* is an interactive protocol which enables committing to an oracle $\Pi \in \mathcal{O}$ and later proving evaluation claims of the form $\Pi(\sigma) = \tau$. Moreover, within the protocol, the prover may send oracles in a (different) class $\mathcal{P}$. For example, a multilinear polynomial commitment scheme with point oracles corresponds with $\mathcal{O} = \mathsf{MLP}_m$ and $\mathcal{P} = \mathsf{Point}_n$.

Definition 4.1. *For an oracle class $\mathcal{O}$, an* **interactive $\mathcal{O}$-commitment** *is a tuple*

$$\mathsf{FC} = (\mathsf{Commit}, \mathsf{Open}) ,$$

where Commit *is the* commitment phase *(Definition 4.2) and* Open *is the* opening phase *(Definition 4.5).*

Definition 4.2. *The commitment phase* Commit *is a tuple $(\mathbf{C}, \mathcal{R}_\bullet, \mathcal{L}_\times)$ that works as follows.*

- Specification. *We consider the following parameters.*
  - *Number of rounds $\mathsf{k} \in \mathbb{N}$.*
  - *For each $i \in [\mathsf{k}]$, an oracle class $\mathcal{P}_i$, non-oracle message length $\mathsf{m}_i \in \mathbb{N}$, and randomness length $\mathsf{r}_i \in \mathbb{N}$.*
- Interaction phase. *The* committer $\mathbf{C}$ *is an interactive algorithm that receives an oracle $\Pi \in \mathcal{O}$ and interacts over k rounds as follows. In each round $i \in [\mathsf{k}]$, the committer sends an oracle $\Pi_i \in \mathcal{P}_i$ and message $\pi_i \in \{0,1\}^{\mathsf{m}_i}$, and receives uniformly sampled randomness $\alpha_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.*
- Decision phase. *After k rounds of interaction, the commitment is defined to be the interaction transcript*
$$\mathsf{cm} := (\Pi_i, \pi_i, \alpha_i)_{i \in [\mathsf{k}]} .$$
Additionally, $\mathbf{C}$ *outputs auxiliary information* aux*.*
- Completeness. *The* output (promise) relation $\mathcal{R}_\bullet$ *and* failure language $\mathcal{L}_\times$ *satisfy the following property. For each $\Pi \in \mathcal{O}$, it holds that*
$$\Pr_{\alpha_1, \ldots, \alpha_{\mathsf{k}}} \left[\big(\mathsf{cm}, (\Pi, \mathsf{aux})\big) \in \mathcal{R}_\bullet^{\mathrm{YES}} \wedge \mathsf{cm} \notin \mathcal{L}_\times \right] = 1 .$$
Above, the commitment cm *and auxiliary information* aux *are computed according to $\mathbf{C}(\Pi)$.*
- Binding. *For any commitment* cm*, if there exist distinct $\Pi, \Pi' \in \mathcal{O}$ such that $(\mathsf{cm}, \Pi), (\mathsf{cm}, \Pi') \in \overline{\mathcal{R}}_\bullet^{\mathrm{NO}}$, then $\mathsf{cm} \in \mathcal{L}_\times$.*

Definition 4.3. *A* **state function** *for* Commit *is a (possibly inefficient) function* State *that, on input an interaction transcript* tr*, outputs a bit and has the following properties.*

- Empty transcript. $\mathsf{State}(\emptyset) = 0$.
- Full transcript. *If* tr *is a full transcript, then* $\mathsf{State}(\mathsf{tr}) = 1$ *if and only if* $\mathsf{tr} \in \mathcal{L}_\times$.

Definition 4.4. Commit *has* **round-by-round binding errors** $(\varepsilon_1, \ldots, \varepsilon_{\mathsf{k}})$ *if there exists a state function* State *for* Commit *with the following property: for every round index* $i \in [\mathsf{k}]$*, interaction transcript* $\mathsf{tr} = ((\Pi_1, \pi_1), \alpha_1, \ldots, (\Pi_{i-1}, \pi_{i-1}), \alpha_{i-1})$ *where* $\mathsf{State}(\mathsf{tr}) = 0$ *and prover message* (Π_i, π_i)*, the following holds.*

$$\Pr_{\alpha_i}\left[\mathsf{State}\big(\mathsf{tr} \| (\Pi_i, \pi_i) \| \alpha_i\big) = 1\right] \leq \varepsilon_i \,.$$

Definition 4.5. *The opening phase* Open *is a functional IOP for the promise relation* $\mathcal{R}_{\circ}(\mathcal{R}_{\bullet}, \mathcal{L}_{\times})$*, defined as follows. Instances have the form* $\mathbb{x} = (\mathsf{cm}_i, \mathsf{ans}_i)_{i \in [d]}$*, where each* ans_i *is a list of evaluation claims.*

- $\mathcal{R}_{\circ}^{\mathrm{YES}}(\mathcal{R}_{\bullet}, \mathcal{L}_{\times})$ *consists of instances* $\mathbb{x}$ *and witnesses* $\mathbb{w} = (\Pi_i, \mathsf{aux}_i)_{i \in [d]}$ *such that, for each* $i \in [d]$*, it holds that* $(\mathsf{cm}_i, (\Pi_i, \mathsf{aux}_i)) \in \mathcal{R}_{\bullet}^{\mathrm{YES}}$ *and* Π_i *agrees with* ans_i*.*
- $\overline{\mathcal{R}}_{\circ}^{\mathrm{NO}}(\mathcal{R}_{\bullet}, \mathcal{L}_{\times})$ *consists of instances* $\mathbb{x}$ *and witnesses* $(\Pi_i)_{i \in [d]}$ *such that at least one of the following holds:*
  - *For each* $i \in [d]$*, it holds that* $(\mathsf{cm}_i, \Pi_i) \in \overline{\mathcal{R}}_{\bullet}^{\mathrm{NO}}$ *and* Π_i *agrees with* ans_i*.*
  - *For some* $i \in [d]$*, it holds that* $\mathsf{cm}_i \in \mathcal{L}_{\times}$*.*

We say that d *is the number of commitments and* $q := \sum_{i \in [d]} |\mathsf{ans}_i|$ *is the number of evaluation claims.*

5 Linear commitments with linear oracles

We construct an interactive linear commitment in the point/linear query model that serves as the information-theoretic core of TensorSwitch.

Lemma 5.1. *Fix linear codes $\mathcal{C}_{\mathsf{row}} \colon \mathbb{F}^{k_{\mathsf{row}}} \to \mathbb{F}^{\ell_{\mathsf{row}}}$ and $\mathcal{C}_{\mathsf{col}} \colon \mathbb{F}^{k_{\mathsf{col}}} \to \mathbb{F}^{\ell_{\mathsf{col}}}$ where $k_{\mathsf{row}}, k_{\mathsf{col}} \in 2^{\mathbb{N}}$. Let $n \coloneqq k_{\mathsf{row}} \cdot k_{\mathsf{col}}$. Fix repetition parameters $t \in \mathbb{N}$ and $s \in 2^{\mathbb{N}}$. There exists an interactive Lin_n-commitment such that:*

- *The commitment phase has the following efficiency measures.*
  - Rounds. *The protocol has 2 rounds.*
  - Non-oracle commitment length. *The committer sends 1 field element.*
  - Commitment oracles. *The committer sends 2 point oracles of $\ell_{\mathsf{row}} \cdot \ell_{\mathsf{col}}$ and ℓ_{row} field elements.*
  - Committer time. *The committer performs $(k_{\mathsf{col}} + 1) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{col}}}$ field operations, $3 \cdot n + k_{\mathsf{col}}$ field multiplications, and $O(n)$ basic field operations.*
- *For d commitments and q evaluation claims, the opening phase has the following efficiency measures.*
  - Rounds. *The protocol has $\log k_{\mathsf{col}} + 4$ rounds.*
  - Non-oracle proof length. *The prover sends $2 \cdot \log k_{\mathsf{col}} + 2 \cdot \binom{d}{2}$ field elements.*
  - Proof oracles. *The prover sends 2 linear oracles of k_{row} and $s \cdot k_{\mathsf{col}}$ field elements, and 1 point oracle of $2 \cdot \binom{d}{2} \cdot \ell_{\mathsf{row}}$ field elements.*
  - Commitment query complexity. *For each $i \in [d]$, the verifier issues $2 \cdot t$ point queries to the i-th commitment's oracles.*
  - Proof query complexity. *The verifier issues $4 \cdot t + 1$ linear queries and $2 \cdot \binom{d}{2} \cdot t$ point queries to the proof oracles.*
  - Prover time. *The prover performs $(2 \cdot \binom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ field operations,*

    $$4 \cdot \binom{d}{2} \cdot n + (d-1) \cdot (2 \cdot n + k_{\mathsf{col}}) + (q+4) \cdot n$$

    field multiplications, and $O((d^2 + q) \cdot n)$ basic field operations.
  - Verifier time. *The verifier performs $O(\log k_{\mathsf{col}} + d^2 + q + t)$ field operations.*

Fix security parameter $\lambda \in \mathbb{N}$, proximity bounds $\delta_{\mathsf{row}} \in (0, \delta(\mathcal{C}_{\mathsf{row}}))$ and $\delta_{\mathsf{col}} \in (0, \delta(\mathcal{C}_{\mathsf{col}}))$, and gap parameter $\varepsilon \in (0, \delta_{\mathsf{row}})$. Assuming the field and repetition parameters are such that

$$|\mathbb{F}| \geq 2^{\lambda} \cdot \max \left\{ \begin{array}{l} \ell_{\mathsf{row}} \cdot \dfrac{|\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})|^2}{2} \cdot \log k_{\mathsf{col}} \\ \dfrac{|\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{col}})|^2}{2} \cdot \log n \\ d \cdot |\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})| \\ (|\Lambda(\mathcal{C}_{\mathsf{row}}^{d \cdot k_{\mathsf{col}}}, \delta_{\mathsf{row}})| + \ell_{\mathsf{row}} \cdot |\Lambda(\mathcal{C}_{\mathsf{col}}^{d}, \delta_{\mathsf{col}})|) \cdot 2 \\ \quad + k_{\mathsf{col}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}}) \cdot |\mathbb{F}| + \ell_{\mathsf{row}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}}) \cdot |\mathbb{F}| \\ |\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})| \cdot 2 + k_{\mathsf{row}} \cdot \varepsilon_{\mathsf{PG}_2}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}}) \cdot |\mathbb{F}| \end{array} \right\},$$

$$t \geq \frac{\lambda}{-\log(1 - (\delta_{\mathsf{row}} - \varepsilon) \cdot \delta_{\mathsf{col}})}, \quad s \geq \frac{\lambda}{2 \cdot \log(e) \cdot \varepsilon^2},$$

the commitment phase has round-by-round binding error at most $2^{-\lambda}$ and the opening phase has round-by-round knowledge soundness error at most $2^{-\lambda}$; the extractor performs

$$\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \left(\mathsf{errdec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}}) + |\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})| \cdot O(\ell_{\mathsf{col}})\right) + d \cdot (\ell_{\mathsf{row}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}}) + k_{\mathsf{col}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{row}}}(\delta_{\mathsf{row}}))$$

field operations.

Proof. In Construction 5.6 we describe the commitment phase, and in Lemma 5.7 we analyze its round-by-round binding errors. In **??** 5.8**??** 5.10 we describe the opening phase, and in Lemmas 5.9 and 5.12 we analyze its round-by-round knowledge soundness. □

Remark 5.2. Alternatively, it suffices for the repetition parameters to be such that

$$t \geq \frac{2 \cdot \lambda}{-\log(1 - \delta_{\mathsf{row}} \cdot \delta_{\mathsf{col}})}, \quad s \geq t.$$

(In this case, Construction 5.8's verifier fixes the subsampled spot check indices $z_i := i$ for $i \in [t]$). Security follows from the round-by-round soundness of parallel repetition; see Section 2.5. In certain parameter regimes, this choice of parameters may be concretely more efficient.

5.1 Definitions

Throughout, we fix parameters as in Lemma 5.1. For notational convenience, we write $\kappa_{\mathsf{row}} := \log k_{\mathsf{row}}$ and $\kappa_{\mathsf{col}} := \log k_{\mathsf{col}}$.

Definition 5.3. *The language* Bind *consists of triples (F, w, f), where $F \in \mathbb{F}^{\ell_{\mathsf{col}} \times \ell_{\mathsf{row}}}$ is a matrix, $w \in \mathbb{F}^{k_{\mathsf{col}}}$ is a vector, $f \in \mathbb{F}^{\ell_{\mathsf{row}}}$ is a vector, and the following holds. For each $j \in [\ell_{\mathsf{row}}]$, there exists at most one codeword $v \in \Lambda(\mathcal{C}_{\mathsf{col}}, F(\cdot, j), \delta_{\mathsf{col}})$ such that $\langle \mathcal{C}_{\mathsf{col}}^{-1}(v), w \rangle = f(j)$.*

Definition 5.4. *Let $F \in \mathbb{F}^{\ell_{\mathsf{col}} \times \ell_{\mathsf{row}}}$ be a matrix, $w_{\mathsf{ood}} \in \mathbb{F}^{k_{\mathsf{col}}}$ be a vector, and $\boldsymbol{\gamma} \in \mathbb{F}^i$ be a point, where $i \in \{0, \ldots, \kappa_{\mathsf{col}}\}$. The relation $\mathsf{Rows}_{F,w,f,\boldsymbol{\gamma}}$ consists of instances $\boldsymbol{u} \in \mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}} - i}}$ and witnesses $(S, (T_j)_{j \in S})$, where $S \subseteq [\ell_{\mathsf{row}}]$ is a subset with $|S| \geq (1 - \delta_{\mathsf{row}}) \cdot \ell_{\mathsf{row}}$, each $T_j \subseteq [\ell_{\mathsf{col}}]$ is a subset with $|T_j| \geq (1 - \delta_{\mathsf{col}}) \cdot \ell_{\mathsf{col}}$, and the following holds:*

$$\forall j \in S, \exists h \in \mathbb{F}^{k_{\mathsf{col}}} : \hat{h}(\boldsymbol{\gamma}, \cdot) = \boldsymbol{u}(j), \ \langle h, w \rangle = f(j), \ F(j, T_j) = \mathcal{C}_{\mathsf{col}}(h)(T_j).$$

When $i = 0$, we drop $\boldsymbol{\gamma}$ and write $\mathsf{Rows}_{F,w,f}$ to denote the relation.

Lemma 5.5. *Let $F \in \mathbb{F}^{\ell_{\mathsf{col}} \times \ell_{\mathsf{row}}}$ be a matrix, $w \in \mathbb{F}^{k_{\mathsf{col}}}$ be a vector, and $f \in \mathbb{F}^{\ell_{\mathsf{row}}}$ be a vector. If $(F, w, f) \in \mathsf{Bind}$, then for any $i \in \{0, \ldots, \kappa_{\mathsf{col}}\}$ and $\boldsymbol{\gamma} \in \mathbb{F}^i$, it holds that*

$$\left|\mathcal{L}(\mathsf{Rows}_{F,w,f,\boldsymbol{\gamma}})\right| \leq \left|\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})\right|.$$

Proof. Suppose $(F, w, f) \in \mathsf{Bind}$. Define the function $\boldsymbol{f} \colon [\ell_{\mathsf{row}}] \to \mathbb{F}^{\{0,1\}^{\kappa_{\mathsf{col}} - i}}$ as follows. For each $j \in [\ell_{\mathsf{row}}]$, if there exists a codeword $v \in \Lambda(\mathcal{C}_{\mathsf{col}}, F(\cdot, j), \delta_{\mathsf{col}})$ such that $\langle \mathcal{C}_{\mathsf{col}}^{-1}(v), w \rangle = f(j)$, then it is unique since $(F, w, f) \in \mathsf{Bind}$; set $\boldsymbol{f}(j) := \hat{h}(\boldsymbol{\gamma}, \cdot)$ with $h := \mathcal{C}_{\mathsf{col}}^{-1}(v)$. Otherwise, set $\boldsymbol{f}(j) := 0$.

Observe that any codeword $\boldsymbol{u} \in \mathcal{L}(\mathsf{Rows}_{F,w,f,\boldsymbol{\gamma}})$ must be in $\Lambda(\mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}}}}, \boldsymbol{f}, \delta_{\mathsf{row}})$; the statement follows from the fact that $\mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}}}} \cong \mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$. □

5.2 Committing with tensor codes

Construction 5.6. We describe a commitment protocol for vectors of length n, viewed as matrices in $\mathbb{F}^{k_{\mathsf{col}} \times k_{\mathsf{row}}}$, with output relation $\mathcal{R}_\bullet$ and failure language $\mathcal{L}_\times$ (defined shortly).

- **Inputs.** In the honest case, the prover receives matrix $M \in \mathbb{F}^{k_{\mathsf{col}} \times k_{\mathsf{row}}}$.
- **Interaction phase.**
  1. **Matrix encoding.** The committer sends point oracle $F \in \mathbb{F}^{\ell_{\mathsf{col}} \times \ell_{\mathsf{row}}}$. In the honest case, $F \coloneqq \mathcal{C}_{\mathsf{col}}^{\ell_{\mathsf{row}}}(\boldsymbol{u})$ with $\boldsymbol{u} \coloneqq \mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}(M)$, viewing M as a function $[k_{\mathsf{row}}] \to \mathbb{F}^{k_{\mathsf{col}}}$.
  2. **Column OOD point.** The verifier samples and sends $\zeta \leftarrow \mathbb{F}^{\kappa_{\mathsf{col}}}$.
  3. **Column OOD evaluations.** The committer sends point oracle $f_{\mathsf{ood}} \in \mathbb{F}^{\ell_{\mathsf{row}}}$. In the honest case, $f_{\mathsf{ood}} \coloneqq \mathcal{C}_{\mathsf{row}}(m_{\mathsf{ood}})$ with $m_{\mathsf{ood}} \in \mathbb{F}^{k_{\mathsf{row}}}$ defined by $m_{\mathsf{ood}}(j) \coloneqq \hat{h}_j(\zeta)$, where $h_j \coloneqq M(\cdot, j)$ denotes the j-th column of M.
  4. **Matrix OOD point.** The verifier samples and sends $\alpha \leftarrow \mathbb{F}^{\kappa_{\mathsf{col}}}$ and $\beta \leftarrow \mathbb{F}^{\kappa_{\mathsf{row}}}$.
  5. **Matrix OOD evaluation.** The committer sends $\mu_{\mathsf{ood}} \in \mathbb{F}$. In the honest case, $\mu_{\mathsf{ood}} \coloneqq \hat{M}(\alpha, \beta)$, viewing M as a function $\{0,1\}^{k_{\mathsf{col}}} \times \{0,1\}^{k_{\mathsf{row}}} \to \mathbb{F}$.
- **Decision phase.** The commitment is $\mathsf{cm} \coloneqq (F, \zeta, f_{\mathsf{ood}}, \alpha, \beta, \mu_{\mathsf{ood}})$. The committer outputs auxiliary information $\mathsf{aux} \coloneqq (\boldsymbol{u}, w_{\mathsf{ood}}, W_{\mathsf{ood}})$, where $w_{\mathsf{ood}} \coloneqq \mathsf{eq}(\zeta, \cdot)$ and $W_{\mathsf{ood}} \coloneqq \mathsf{eq}(\alpha, \cdot) \otimes \mathsf{eq}(\beta, \cdot)$.

Output relation and failure language.

- $\mathcal{R}_\bullet^{\mathrm{YES}}$ consists of instances $\mathbb{x} = \mathsf{cm}$ and witnesses $\mathbb{w} = (M, \mathsf{aux})$ such that the following hold:
  - $\boldsymbol{u} = \mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}(M)$, $F = \mathcal{C}_{\mathsf{col}}^{\ell_{\mathsf{row}}}(\boldsymbol{u})$, $w_{\mathsf{ood}} = \mathsf{eq}(\zeta, \cdot)$, and $W_{\mathsf{ood}} = \mathsf{eq}(\alpha, \cdot) \otimes \mathsf{eq}(\beta, \cdot)$.
  - For each $j \in [\ell_{\mathsf{row}}]$, it holds that $\hat{h}_j(\zeta) = f_{\mathsf{ood}}(j)$, where $h_j \coloneqq \boldsymbol{u}(j)$.
  - $\hat{M}(\alpha, \beta) = \mu_{\mathsf{ood}}$.
- $\overline{\mathcal{R}}_\bullet^{\mathrm{NO}}$ consists of instances $\mathbb{x} = \mathsf{cm}$ and witnesses $\mathbb{w} = M$ such that the following hold:
  - $\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}(M) \in \mathsf{Rows}_{F, \mathsf{eq}(\zeta, \cdot), f_{\mathsf{ood}}}$.
  - $\hat{M}(\alpha, \beta) = \mu_{\mathsf{ood}}$.
- $\mathcal{L}_\times$ consists of instances $\mathbb{x} = \mathsf{cm}$ where $(F, \mathsf{eq}(\zeta, \cdot), f_{\mathsf{ood}}) \notin \mathsf{Bind}$, or there exist distinct matrices $M, M' \in \mathbb{F}^{k_{\mathsf{col}} \times k_{\mathsf{row}}}$ such that $(\mathbb{x}, M), (\mathbb{x}, M') \in \overline{\mathcal{R}}_\bullet^{\mathrm{NO}}$.

Prover time. Implementing the prover requires the following $\mathbb{F}$-operations (we omit costs which are sublinear in n):

- $k_{\mathsf{col}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ operations to compute $\boldsymbol{u}$.
- $\ell_{\mathsf{row}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{col}}}$ operations to compute F.
- n multiplications and $O(n)$ basic operations to compute m_{ood}.
- $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ operations to compute f_{ood}.
- n multiplications and $O(n)$ basic operations to compute μ_{ood}.
- k_{col} multiplications and $O(k_{\mathsf{col}})$ basic operations to compute w_{ood}.
- n multiplications and $O(n)$ basic operations to compute W_{ood}.

Lemma 5.7. *Construction 5.6 has round-by-round binding errors*

$$\left(\ell_{\mathsf{row}} \cdot \frac{|\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})|^2}{2} \cdot \frac{\log k_{\mathsf{col}}}{|\mathbb{F}|}, \frac{|\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})|^2}{2} \cdot \frac{\log n}{|\mathbb{F}|}\right).$$

Proof. We define a state function State for Construction 5.6.

1. **Column OOD sample.** At this stage the transcript is empty. The committer sends F, and the verifier sends ζ.

   State function. We set $\mathsf{State}(F\|\zeta) = 1$ if and only if, for some $j \in [\ell_{\mathsf{row}}]$, there exist distinct codewords $v, v' \in \Lambda(F(\cdot, j), \delta_{\mathsf{col}})$ such that $\hat{h}(\zeta) = \hat{h}'(\zeta)$ with $h \coloneqq \mathcal{C}_{\mathsf{col}}^{-1}(v)$ and $h' \coloneqq \mathcal{C}_{\mathsf{col}}^{-1}(v')$.

2. **Matrix OOD sample.** At this stage the transcript has the form $\mathsf{tr} = (F, \zeta)$. The committer sends f_{ood}, and the verifier sends α, β.

   State function. We set $\mathsf{State}(\mathsf{tr}\|f_{\mathsf{ood}}\|(\alpha, \beta)) = 1$ if and only if at least one of the following hold:

   (a) There exist distinct codewords $\boldsymbol{u}, \boldsymbol{u}' \in \mathcal{L}(\mathsf{Rows}_{F,\mathsf{eq}(\zeta,\cdot),f_{\mathsf{ood}}})$ such that $\hat{M}(\alpha, \beta) = \hat{M}'(\alpha, \beta)$ with $M = (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u})$ and $M' = (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u}')$.

   (b) $(F, \mathsf{eq}(\zeta, \cdot), f_{\mathsf{ood}}) \notin \mathsf{Bind}$.

   Observe that if $\mathsf{State}(\mathsf{tr}\|f_{\mathsf{ood}}\|(\alpha, \beta)) = 0$, then $\mathsf{cm} \in \mathcal{L}_{\times}$ regardless of the committer's final message.

Error bounds. We upper bound the round-by-round binding errors based on State.

1. **Column OOD sample.** We show that

   $$\Pr_{\zeta}[\mathsf{State}(F\|\zeta) = 1] \leq \ell_{\mathsf{row}} \cdot \frac{|\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})|^2}{2} \cdot \frac{\log k_{\mathsf{col}}}{|\mathbb{F}|}.$$

   This follows from Lemma 3.19 and a union bound over each $j \in [\ell_{\mathsf{row}}]$.

2. **Matrix OOD sample.** Suppose that $\mathsf{State}(\mathsf{tr}) = 0$. We show that

   $$\Pr_{\alpha,\beta}[\mathsf{State}(\mathsf{tr}\|f_{\mathsf{ood}}\|(\alpha, \beta)) = 1] \leq \frac{\left|\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})\right|^2}{2} \cdot \frac{\log n}{|\mathbb{F}|}.$$

   Observe that Item 2b cannot hold, regardless of the prover's message; it remains to bound the probability that Item 2a holds. The bound follows from an argument similar to the proof of Lemma 3.19, except it uses Lemma 5.5 and a union bound over all pairs of distinct codewords in $\mathcal{L}(\mathsf{Rows}_{F,\mathsf{eq}(\zeta,\cdot),f_{\mathsf{ood}}})$.

□

5.3 Proving linear evaluations

As a step towards constructing the opening phase, we first construct a linear IOP for the promise relation $\mathcal{R}$, defined as follows.

- We consider instances of the form $\mathbb{x} = (F, [\![w_{\mathsf{ood}}]\!], f_{\mathsf{ood}}, [\![W]\!], \mu)$, where:
  - $F \in \mathbb{F}^{\ell_{\mathsf{col}} \times \ell_{\mathsf{row}}}$ is a matrix, $f_{\mathsf{ood}} \in \mathbb{F}^{\ell_{\mathsf{row}}}$ is a vector, and $\mu \in \mathbb{F}$ is a field element.
  - $[\![w_{\mathsf{ood}}]\!]$ and $[\![W]\!]$ are descriptions of weight vector $w_{\mathsf{ood}} \in \mathbb{F}^{k_{\mathsf{col}}}$ and weight matrix $W \in \mathbb{F}^{k_{\mathsf{col}} \times k_{\mathsf{row}}}$.
- $\mathcal{R}^{\mathrm{YES}}$ consists of instances $\mathbb{x}$ and witnesses $\mathbb{w} = (M, \boldsymbol{u})$ where the following holds:
  - $\boldsymbol{u} = \mathcal{C}^{k_{\mathsf{col}}}_{\mathsf{row}}(M)$ and $F = \mathcal{C}^{\ell_{\mathsf{row}}}_{\mathsf{col}}(\boldsymbol{u})$.
  - For each $j \in [\ell_{\mathsf{row}}]$, it holds that $\langle \boldsymbol{u}(j), w_{\mathsf{ood}} \rangle = f_{\mathsf{ood}}(j)$.
  - $\langle M, W \rangle = \mu$.
- $\overline{\mathcal{R}}^{\mathrm{NO}}$ consists of instances $\mathbb{x}$ and witnesses $\mathbb{w} = M$ where $(F, w_{\mathsf{ood}}, f_{\mathsf{ood}}) \notin \mathsf{Bind}$ or the following holds:
  - $\mathcal{C}^{k_{\mathsf{col}}}_{\mathsf{row}}(M) \in \mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}}$.
  - $\langle M, W \rangle = \mu$.

Construction 5.8. We describe a linear IOP for $\mathcal{R}$ (defined above).

- **Inputs.** The prover and verifier receive instance $\mathbb{x} = (F, [\![w_{\mathsf{ood}}]\!], f_{\mathsf{ood}}, [\![W]\!], \mu)$; the verifier has point query access to F and f_{ood}. In the honest case, the prover additionally receives witness $\mathbb{w} = (M, \boldsymbol{u})$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}^{\mathrm{YES}}$; we assume the prover explicitly knows w_{ood} and W.
- **Interaction phase.**
  1. **Sumcheck protocol.** The prover and verifier run a κ_{col}-round sumcheck protocol for the claim:
  $$\sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}}}} \left\langle \hat{M}(\boldsymbol{b}), \hat{W}(\boldsymbol{b}) \right\rangle = \mu \,,$$
  viewing M and W as functions $\{0,1\}^{\kappa_{\mathsf{col}}} \to \mathbb{F}^{k_{\mathsf{row}}}$. For $i \in [\kappa_{\mathsf{col}}]$:
     (a) **Sumcheck polynomial.** The prover sends quadratic polynomial $\hat{p}_i \in \mathbb{F}[\mathtt{X}]$. In the honest case,
     $$\hat{p}_i(\mathtt{X}) \coloneqq \sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}} - i}} \left\langle \hat{M}(\gamma_1, \ldots, \gamma_{i-1}, \mathtt{X}, \boldsymbol{b}), \hat{W}(\gamma_1, \ldots, \gamma_{i-1}, \mathtt{X}, \boldsymbol{b}) \right\rangle .$$
     (b) **Sumcheck randomness.** The verifier samples and sends $\gamma_i \leftarrow \mathbb{F}$.

     Let $\boldsymbol{\gamma} \coloneqq (\gamma_1, \ldots, \gamma_{\kappa_{\mathsf{col}}})$, $m \coloneqq \hat{M}(\boldsymbol{\gamma})$, and $w \coloneqq \hat{W}(\boldsymbol{\gamma})$.
  2. **Combination oracle.** The prover sends linear oracle $\Pi_{\mathsf{row}} \in \mathbb{F}^{k_{\mathsf{row}}}$. In the honest case, $\Pi_{\mathsf{row}} \coloneqq m$.
  3. **Column subset.** The verifier samples and sends $y_1, \ldots, y_s \leftarrow [\ell_{\mathsf{row}}]$.
  4. **Column oracle.** The prover sends linear oracle $\Pi_{\mathsf{cols}} \in \mathbb{F}^{s \times k_{\mathsf{col}}}$. For notational convenience, let $\Pi^{(i)}_{\mathsf{col}} \in \mathbb{F}^{k_{\mathsf{col}}}$ denote $\Pi_{\mathsf{cols}}(i, \cdot)$ for each $i \in [s]$. In the honest case, $\Pi^{(i)}_{\mathsf{col}} \coloneqq \boldsymbol{u}(y_i)$.

5. **Spot checks.** The verifier samples and sends $z_1, \ldots, z_t \leftarrow [s]$ and $x_1, \ldots, x_t \leftarrow [\ell_{\mathsf{col}}]$.

- **Decision phase.** (The verifier rejects if any check fails.)

1. **Sumcheck decision.** The verifier checks the following:
   - $\hat{p}_1(0) + \hat{p}_1(1) = \mu$.
   - For each $i \in [\kappa_{\mathsf{col}})$, it holds that $\hat{p}_{i+1}(0) + \hat{p}_{i+1}(1) = \hat{p}_i(\gamma_i)$.
   - $\langle \Pi_{\mathsf{row}}, w \rangle = \hat{p}_{\kappa_{\mathsf{col}}}(\gamma_{\kappa_{\mathsf{col}}})$, where the left side is obtained via linear query to Π_{row} with weights w. (The verifier implicitly describes w with $\boldsymbol{\gamma}$ and the description $[\![W]\!]$.)
2. **Spot check decision.** For each $i \in [t]$, the verifier checks:
   - $\langle \Pi_{\mathsf{col}}^{(z_i)}, w_{\mathsf{ood}} \rangle = f_{\mathsf{ood}}(y_{z_i})$, where the left side is obtained via linear query to Π_{cols} with weights $\mathsf{eq}(\mathsf{bin}(z_i), \cdot) \otimes w_{\mathsf{ood}}$ and the right side is obtained via point query to f_{ood}.
   - $\hat{\Pi}_{\mathsf{col}}^{(z_i)}(\boldsymbol{\gamma}) = \mathcal{C}_{\mathsf{row}}(\Pi_{\mathsf{row}})(y_{z_i})$, where the left side is obtained via linear query to Π_{cols} with weights $\mathsf{eq}(\mathsf{bin}(z_i), \cdot) \otimes \mathsf{eq}(\boldsymbol{\gamma}, \cdot)$ and the right side is obtained via linear query to Π_{row} with weights $G_{\mathcal{C}_{\mathsf{row}}}(y_i, \cdot)$.
   - $F(x_i, y_{z_i}) = \mathcal{C}_{\mathsf{col}}(\Pi_{\mathsf{col}}^{(z_i)})(x_i)$, where the left side is obtained via point query to F and the right side is obtained via linear query to Π_{cols} with weights $\mathsf{eq}(\mathsf{bin}(z_i), \cdot) \otimes G_{\mathcal{C}_{\mathsf{col}}}(x_i, \cdot)$.

Prover time. Implementing the prover requires the following $\mathbb{F}$-operations (we omit costs which are sublinear in n):

- $4 \cdot n$ multiplications and $O(n)$ basic operations to implement the sumcheck prover, which computes the messages $\hat{p}_1, \ldots, \hat{p}_{\kappa_{\mathsf{col}}}$ as well as the new claim consisting of m, w, and $\langle m, w \rangle$ [HJRRR25, Appendix A].
- $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ operations to answer the verifier's queries (in particular, to compute $\mathcal{C}_{\mathsf{row}}(\Pi_{\mathsf{row}})$).[18]

Lemma 5.9. *Construction 5.8 has round-by-round knowledge soundness errors*

$$(\varepsilon_{\mathsf{sc}}^{(1)}, \ldots, \varepsilon_{\mathsf{sc}}^{(\kappa_{\mathsf{col}})}, \varepsilon_{\mathsf{cols}}, \varepsilon_{\mathsf{spot}}),$$

where:

- $\varepsilon_{\mathsf{sc}}^{(i)} \leq |\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})| \cdot \frac{2}{|\mathbb{F}|} + \frac{k_{\mathsf{col}}}{2^i} \cdot \varepsilon_{\mathsf{PG}_2}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}})$.
- $\varepsilon_{\mathsf{cols}} \leq e^{-2 \cdot \varepsilon^2 \cdot s}$.
- $\varepsilon_{\mathsf{spot}} \leq (1 - (\delta_{\mathsf{row}} - \varepsilon) \cdot \delta_{\mathsf{col}})^t$.

Moreover, the extractor performs

$$\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \left(\mathsf{errdec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}}) + |\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})| \cdot O(k_{\mathsf{col}}) + O(\ell_{\mathsf{col}})\right)$$

field operations.

Proof. We define a state function State for Construction 5.8.

[18] In fact, the prover only needs to compute t symbols of $\mathcal{C}_{\mathsf{row}}(\Pi_{\mathsf{row}})$; this may be easier than computing the entire codeword.

1. **Sumcheck randomness.** At this stage the transcript has the form

$$\mathsf{tr} = (\hat{p}_i, \gamma_i)_{i \in [i^\star)} \text{ for } i^\star \in [\kappa_{\mathsf{col}}] \,.$$

The prover sends $\hat{p}_{i^\star}$, and the verifier sends randomness $\gamma_{i^\star}$. Let $\boldsymbol{\gamma}^{(i^\star)} \coloneqq (\gamma_1, \ldots, \gamma_{i^\star})$.

State function. We set $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| \hat{p}_{i^\star} \| \gamma_{i^\star}, \mathsf{w}) = 1$ if and only if $(F, w_{\mathsf{ood}}, f_{\mathsf{ood}}) \notin \mathsf{Bind}$ or the following hold:

(a) There exists $\boldsymbol{u} \in \mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}} - i^\star}}$ such that $(\boldsymbol{u}, \mathsf{w}) \in \mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}, \boldsymbol{\gamma}^{(i^\star)}}$ and

$$\sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}} - i^\star}} \left\langle \mathcal{C}_{\mathsf{row}}^{-1}(u_{\boldsymbol{b}}), \hat{W}(\boldsymbol{\gamma}^{(i^\star)}, \boldsymbol{b}) \right\rangle = \hat{p}_{i^\star}(\gamma_{i^\star}) \,.$$

(b) $\hat{p}_1(0) + \hat{p}_1(1) = \mu$.

(c) For each $i \in [i^\star)$, it holds that $\hat{p}_{i+1}(0) + \hat{p}_{i+1}(1) = \hat{p}_i(\gamma_i)$.

Extractor. The extractor $\mathbf{E}(\mathbb{x}, \mathsf{tr} \| \hat{p}_{i^\star} \| \gamma_{i^\star}, \mathsf{w}) \coloneqq \mathsf{w}$ is the identity function on the state function witness.

2. **Column subset.** At this stage the transcript has the form

$$\mathsf{tr} = (\hat{p}_i, \gamma_i)_{i \in [\kappa_{\mathsf{col}}]} \,.$$

The prover sends Π_{row}, and the verifier sends $y_1, \ldots, y_s$. Let $\boldsymbol{y} \coloneqq (y_1, \ldots, y_s)$.

State function. We set $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{row}} \| \boldsymbol{y}, \mathsf{w}) = 1$ for $\mathsf{w} = \bot$ if and only if the following hold:

(a) There exists a subset $I \subseteq [s]$ with $|I| \geq (1 - \delta_{\mathsf{row}} + \varepsilon) \cdot s$ such that the following holds:

$$\forall i \in I, \exists h \in \mathbb{F}^{k_{\mathsf{col}}} : \hat{h}(\boldsymbol{\gamma}) = \mathcal{C}_{\mathsf{row}}(\Pi_{\mathsf{row}})(y_i) \,,\ \langle h, w_{\mathsf{ood}} \rangle = f_{\mathsf{ood}}(y_i) \,,\ \Delta\big(F(\cdot, y_i), \mathcal{C}_{\mathsf{col}}(h)\big) \leq \delta_{\mathsf{col}} \,.$$

(b) $\hat{p}_1(0) + \hat{p}_1(1) = \mu$.

(c) For each $i \in [\kappa_{\mathsf{col}})$, it holds that $\hat{p}_{i+1}(0) + \hat{p}_{i+1}(1) = \hat{p}_i(\gamma_i)$.

(d) $\langle \Pi_{\mathsf{row}}, w \rangle = \hat{p}_{\kappa_{\mathsf{col}}}(\gamma_{\kappa_{\mathsf{col}}})$.

Extractor. The extractor $\mathbf{E}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{row}} \| \boldsymbol{y}, \mathsf{w})$ outputs $(S, (T_i)_{i \in S})$, which is computed as follows. The subset $S \subseteq [\ell_{\mathsf{row}}]$ is initialized to be empty, and the extractor computes $u \coloneqq \mathcal{C}_{\mathsf{row}}(\Pi_{\mathsf{row}})$. For each $j \in [\ell_{\mathsf{row}}]$, the extractor list-decodes $F(\cdot, j)$ to obtain $L \coloneqq (v, \mathcal{C}_{\mathsf{col}}^{-1}(v))_{v \in \Lambda(\mathcal{C}_{\mathsf{col}}, F(\cdot, j), \delta_{\mathsf{col}})}$. Then, it attempts to find $(v, h) \in L$ such that $\hat{h}(\boldsymbol{\gamma}) = u(j)$ and $\langle h, w_{\mathsf{ood}} \rangle = f_{\mathsf{ood}}(j)$. If the extractor succeeds, it defines $T_j \subseteq [\ell_{\mathsf{col}}]$ to be the set of indices i where $F(i, j) = v(i)$, and adds j to S.

3. **Spot checks.** At this stage the transcript has the form

$$\mathsf{tr} = \big((\hat{p}_i, \gamma_i)_{i \in [\kappa_{\mathsf{col}}]}, \Pi_{\mathsf{row}}, \boldsymbol{y}\big) \,.$$

The prover sends Π_{cols}, and the verifier sends $z_1, \ldots, z_t, x_1, \ldots, x_t$. Let $\boldsymbol{z} \coloneqq (z_1, \ldots, z_t)$ and $\boldsymbol{x} \coloneqq (x_1, \ldots, x_t)$.

State function. We set $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{cols}} \| (\boldsymbol{z}, \boldsymbol{x}), \mathsf{w}) = 1$ for $\mathsf{w} = \bot$ if and only if the verifier accepts.

Extractor. The extractor is $\mathbf{E}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{cols}} \| (\boldsymbol{z}, \boldsymbol{x}), \mathsf{w}) \coloneqq \bot$.

Extractor time. Implementing the extractor requires the following $\mathbb{F}$-operations (we omit costs which are sublinear in k):

- $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ operations to compute u.
- For each $j \in [\ell_{\mathsf{row}}]$:
  - $\mathsf{errdec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}})$ operations to compute L.
  - $|\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})| \cdot O(k_{\mathsf{col}})$ operations to identify v.
  - $O(\ell_{\mathsf{col}})$ operations to compute T_j.

Error bounds. We upper bound the round-by-round knowledge soundness errors based on State.

1. **Sumcheck randomness.** We show that

$$\Pr_{\gamma_{i^\star}}\left[\exists \mathsf{w} : \begin{array}{l}\mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr}\|\hat{p}_{i^\star}\|\gamma_{i^\star}, \mathsf{w})\big) = 0 \\ \mathsf{State}(\mathbb{x}, \mathsf{tr}\|\hat{p}_{i^\star}\|\gamma_{i^\star}, \mathsf{w}) = 1\end{array}\right] \leq \big|\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})\big| \cdot \frac{2}{|\mathbb{F}|} + \frac{k_{\mathsf{col}}}{2^{i^\star}} \cdot \varepsilon_{\mathsf{PG}_2}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}})\,.$$

   We focus on the case where $i^\star > 1$; the case where $i^\star = 0$ follows the same argument, up to syntactic differences. Assume that $(F, w_{\mathsf{ood}}, f_{\mathsf{ood}}) \in \mathsf{Bind}$, since otherwise the above event never occurs. For $\boldsymbol{u} \in \mathcal{L}(\mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}, \boldsymbol{\gamma}^{(i^\star-1)}})$ and $\gamma_{i^\star} \in \mathbb{F}$, define the event

$$\begin{aligned} E(\boldsymbol{u}, \gamma_{i^\star}) \coloneqq & \sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}} - (i^\star - 1)}} \big\langle \mathcal{C}_{\mathsf{row}}^{-1}(u_{\boldsymbol{b}}), \hat{W}(\boldsymbol{\gamma}^{(i^\star-1)}, \boldsymbol{b}) \big\rangle \neq \hat{p}_{i^\star}(0) + \hat{p}_{i^\star}(1) \\ & \wedge \sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}} - i^\star}} \Big\langle \mathsf{Fold}_{\gamma_{i^\star}, \boldsymbol{b}}\big(\mathcal{C}_{\mathsf{row}}^{-1}(\boldsymbol{u})\big), \hat{W}(\boldsymbol{\gamma}^{(i^\star)}, \boldsymbol{b}) \Big\rangle = \hat{p}_{i^\star}(\gamma_{i^\star})\,. \end{aligned}$$

   By Lemma 3.1, $\Pr_{\gamma_{i^\star}}[E(\boldsymbol{u}, \gamma_{i^\star})] \leq \frac{2}{|\mathbb{F}|}$ for any choice of $\boldsymbol{u}$. Let $\boldsymbol{f} \colon [\ell_{\mathsf{row}}] \to \mathbb{F}^{\{0,1\}^{\kappa_{\mathsf{col}} - (i^\star - 1)}}$ be as described in the proof of Lemma 5.5, using $\boldsymbol{\gamma}^{(i^\star-1)}$. Fix $\gamma_{i^\star}$ such that the following hold:

   (i) For each $\boldsymbol{u} \in \mathcal{L}(\mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}, \boldsymbol{\gamma}^{(i^\star-1)}})$, the event $E(\boldsymbol{u}, \gamma_{i^\star})$ does not occur. By Lemma 5.5 and a union bound, the fraction of $\gamma_{i^\star}$ for which this does not hold is at most $|\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})| \cdot \frac{2}{|\mathbb{F}|}$.

   (ii) For any subset $S \subseteq [\ell_{\mathsf{row}}]$ with $|S| \geq (1 - \delta_{\mathsf{row}}) \cdot \ell_{\mathsf{row}}$ such that $\mathsf{Fold}_{\gamma_{i^\star}}(\boldsymbol{f})$ agrees with $\mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}} - i^\star}}$ on S, it holds that $\boldsymbol{f}$ agrees with $\mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}} - (i^\star - 1)}}$ on S. By Definition 3.20, Fact 3.22, and a union bound, the fraction of $\gamma_{i^\star}$ for which this is not true is at most $\frac{k_{\mathsf{col}}}{2^{i^\star}} \cdot \varepsilon_{\mathsf{PG}_2}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}})$.

   Overall, the fraction of $\gamma_{i^\star}$ for which any condition does not hold is at most

$$\big|\Lambda\big(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}}\big)\big| \cdot \frac{2}{|\mathbb{F}|} + \frac{k_{\mathsf{col}}}{2^{i^\star}} \cdot \varepsilon_{\mathsf{PG}_2}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}})\,.$$

   Consider a state function witness $\mathsf{w} = (S, (T_i)_{i \in S})$ where $\mathsf{State}(\mathbb{x}, \mathsf{tr}\|\hat{p}_{i^\star}\|\gamma_{i^\star}, \mathsf{w}) = 1$; since $(F, w_{\mathsf{ood}}, f_{\mathsf{ood}}) \in \mathsf{Bind}$, it must be that Items 1a to 1c hold for $i^\star$. We show that

$$\mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr}\|\hat{p}_{i^\star}\|\gamma_{i^\star}, \mathsf{w})\big) = \mathsf{State}(\mathbb{x}, \mathsf{tr}, \mathsf{w}) = 1\,,$$

which proves the upper bound. Clearly Items 1b and 1c hold for $i^\star - 1$; the remaining task is to show that Item 1a holds for $i^\star - 1$. Since Item 1a holds for $i^\star$, there exists a codeword $\boldsymbol{u}^{(i^\star)} \in \mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{col}} - i^\star}}$ such that $(\boldsymbol{u}^{(i^\star)}, \mathsf{w}) \in \mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}, \boldsymbol{\gamma}^{(i^\star)}}$ and

$$\sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}} - i^\star}} \left\langle \mathcal{C}_{\mathsf{row}}^{-1}(u_{\boldsymbol{b}}^{(i^\star)}), \hat{W}(\boldsymbol{\gamma}^{(i^\star)}, \boldsymbol{b}) \right\rangle = \hat{p}_{i^\star}(\gamma_{i^\star}) \,.$$

Since $(F, \zeta, f_{\mathsf{ood}}) \in \mathsf{Bind}$, it must be that $\boldsymbol{u}^{(i^\star)}$ agrees with $\mathsf{Fold}_{\gamma_{i^\star}}(\boldsymbol{f})$ on S. By Item (ii), $\boldsymbol{f}$ agrees with a codeword $\boldsymbol{u}^{(i^\star - 1)} \in \mathcal{C}_{\mathsf{row}}^{\{0,1\}^{\kappa_{\mathsf{row}} - (i^\star - 1)}}$ on S, and hence $(\boldsymbol{u}^{(i^\star - 1)}, \mathsf{w}) \in \mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}, \boldsymbol{\gamma}^{(i^\star - 1)}}$. Observe that $\mathsf{Fold}_{\gamma_{i^\star}}(\boldsymbol{u}^{(i^\star - 1)}) = \boldsymbol{u}^{(i^\star)}$, since $\delta_{\mathsf{row}} < \delta(\mathcal{C}_{\mathsf{row}})$. By Item (i), it must be that

$$\sum_{\boldsymbol{b} \in \{0,1\}^{\kappa_{\mathsf{col}} - (i^\star - 1)}} \left\langle \mathcal{C}_{\mathsf{row}}^{-1}(u_{\boldsymbol{b}}), \hat{W}(\boldsymbol{\gamma}^{(i^\star - 1)}, \boldsymbol{b}) \right\rangle = \hat{p}_{i^\star}(0) + \hat{p}_{i^\star}(1) = \hat{p}_{i^\star - 1}(\gamma_{i^\star - 1}) \,,$$

where the final equality follows from the fact that Item 1c holds for $i^\star$. We conclude that Item 1a holds for $i^\star - 1$.

2. **Column subset.** We show that

$$\Pr_{\boldsymbol{y}} \left[\exists \mathsf{w} : \begin{array}{l} \mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{row}} \| \boldsymbol{y}, \mathsf{w})\big) = 0 \\ \mathsf{State}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{row}} \| \boldsymbol{y}, \mathsf{w}) = 1 \end{array} \right] \leq e^{-2 \cdot \varepsilon^2 \cdot s} \,.$$

It suffices to bound the probability (over $\boldsymbol{y}$) that Item 2a holds, assuming that the extractor (which behaves independently of $\boldsymbol{y}$) fails. For $y \in [\ell_{\mathsf{row}}]$, define the event

$$E(y) \coloneqq \exists h \in \mathbb{F}^{k_{\mathsf{col}}} : \hat{h}(\boldsymbol{\gamma}) = \mathcal{C}_{\mathsf{row}}(\Pi_{\mathsf{row}})(y) \,,\ \langle h, w_{\mathsf{ood}} \rangle = f_{\mathsf{ood}}(y) \,,\ \Delta\big(F(\cdot, y), \mathcal{C}_{\mathsf{col}}(h)\big) \leq \delta_{\mathsf{col}}$$

Let $\alpha \coloneqq \Pr_{y \leftarrow [\ell_{\mathsf{col}}]}[E(y)]$; observe that the extractor failing implies $\alpha < 1 - \delta_{\mathsf{row}}$. Hence,

$$\begin{aligned} \Pr_{\boldsymbol{y}}[\text{Item 2a holds}] &\leq \Pr_{y_1, \ldots, y_s \leftarrow [\ell_{\mathsf{col}}]} \left[\frac{1}{s} \sum_{i \in [s]} E(y_i) \geq 1 - \delta_{\mathsf{row}} + \varepsilon \right] \\ &< \Pr_{y_1, \ldots, y_s \leftarrow [\ell_{\mathsf{col}}]} \left[\frac{1}{s} \sum_{i \in [s]} E(y_i) \geq \alpha + \varepsilon \right] \,, \end{aligned}$$

and the bound follows from Lemma 3.2.

3. **Spot checks.** We show that

$$\Pr_{\boldsymbol{z}, \boldsymbol{x}} \left[\exists \mathsf{w} : \begin{array}{l} \mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{cols}} \| (\boldsymbol{z}, \boldsymbol{x}), \mathsf{w})\big) = 0 \\ \mathsf{State}(\mathbb{x}, \mathsf{tr} \| \Pi_{\mathsf{cols}} \| (\boldsymbol{z}, \boldsymbol{x}), \mathsf{w}) = 1 \end{array} \right] \leq \big(1 - (\delta_{\mathsf{row}} - \varepsilon) \cdot \delta_{\mathsf{col}}\big)^t \,.$$

It suffices to bound the probability (over $\boldsymbol{z}$ and $\boldsymbol{x}$) that the verifier accepts, assuming that Item 2a does not hold. For each $i \in [t]$, the probability that the verifier's i-th spot check detects an inconsistency is at least $(\delta_{\mathsf{row}} + \varepsilon) \cdot \delta_{\mathsf{col}}$. The bound follows from the fact that each spot check is independent.

□

5.4 Batch proving linear evaluations

Construction 5.10. We describe a functional IOP for $\mathcal{R}_\circ(\mathsf{Lin}_n, \mathcal{R}_\bullet, \mathcal{L}_\times)$ (Definition 4.5 and Construction 5.6).

- **Inputs.** For each $i \in [d]$, the prover and verifier receive commitment and evaluation claims

$$\mathsf{cm}_i = (F^{(i)}, \zeta^{(i)}, f^{(i)}_{\mathsf{ood}}, \alpha^{(i)}, \beta^{(i)}, \mu^{(i)}_{\mathsf{ood}}), \quad \mathsf{ans}_i \coloneqq ([\![W^{(i)}_j]\!], \mu^{(i)}_j)_{j\in[q_i]}.$$

  The verifier has point query access to $F^{(i)}$ and linear query access to $f^{(i)}_{\mathsf{ood}}$. In the honest case, the prover additionally receives message $\Pi_i = M^{(i)}$ and auxiliary information $\mathsf{aux}_i = (\boldsymbol{u}^{(i)}, w^{(i)}_{\mathsf{ood}}, W^{(i)}_{\mathsf{ood}})$ where $(\mathsf{cm}_i, (\Pi_i, \mathsf{aux}_i)) \in \mathcal{R}^{\mathrm{YES}}_\bullet$ and, for each $j \in [q_i]$, it holds that $\langle M^{(i)}, W^{(i)}_j\rangle = \mu^{(i)}_j$; we assume the prover explicitly has the entries of $W^{(i)}_j$.

- **Interaction phase.**

  1. **Query compression.** The verifier samples and sends $\gamma'_1, \ldots, \gamma'_{q^\star} \leftarrow \mathbb{F}$, where $q^\star \coloneqq \max_i q_i$. Define

$$W^{(i)} \coloneqq W^{(i)}_{\mathsf{ood}} + \sum_{j\in[q_i]} \gamma'_j \cdot W^{(i)}_j, \quad \mu^{(i)} \coloneqq \mu^{(i)}_{\mathsf{ood}} + \sum_{j\in[q_i]} \gamma'_j \cdot \mu^{(i)}_j.$$

     In the honest case, the prover computes $W^{(i)}$ and $\mu^{(i)}$.

  2. **Cross terms.** Let $I \subseteq [d] \times [d]$ denote the set of pairs (i, i') where $i \neq i'$. The prover sends point oracle $\Pi_{\mathsf{ct}} \colon I \times [\ell_{\mathsf{row}}] \to \mathbb{F}$ and $\mu^{(i,i')} \in \mathbb{F}$ for each $(i, i') \in I$. For notational convenience, let $f^{(i,i')}_{\mathsf{ood}} \in \mathbb{F}^{\ell_{\mathsf{row}}}$ denote $\Pi_{\mathsf{ct}}((i, i'), \cdot)$. In the honest case, $f^{(i,i')}_{\mathsf{ood}} \coloneqq \mathcal{C}_{\mathsf{row}}(m^{(i,i')}_{\mathsf{ood}})$ with $m^{(i,i')}_{\mathsf{ood}}$ defined by $m^{(i,i')}_{\mathsf{ood}}(j) \coloneqq \hat{h}^{(i)}_j(\zeta^{(i')})$, where $h^{(i)}_j \coloneqq M^{(i)}(\cdot, j)$ denotes the j-th column of $M^{(i)}$, and $\mu^{(i,i')} \coloneqq \langle M^{(i)}, W^{(i')}\rangle$.

  3. **Batching randomness.** The verifier samples and sends $\gamma_2, \ldots, \gamma_d \leftarrow \mathbb{F}$. Set $\gamma_1 \coloneqq 1$ and define

$$F \coloneqq \sum_{i\in[d]} \gamma_i \cdot F^{(i)}, \quad w_{\mathsf{ood}} \coloneqq \sum_{i\in[d]} \gamma_i \cdot w^{(i)}_{\mathsf{ood}}, \quad W \coloneqq \sum_{i\in[d]} \gamma_i \cdot W^{(i)},$$
$$f_{\mathsf{ood}} \coloneqq \sum_{i\in[d]} \gamma_i^2 \cdot f^{(i)}_{\mathsf{ood}} + \sum_{(i,i')\in I} \gamma_i \cdot \gamma_{i'} \cdot f^{(i,i')}_{\mathsf{ood}}, \quad \mu \coloneqq \sum_{i\in[d]} \gamma_i^2 \cdot \mu^{(i)} + \sum_{(i,i')\in I} \gamma_i \cdot \gamma_{i'} \cdot \mu^{(i,i')}.$$

     In the honest case, define $M \coloneqq \sum_{i\in[d]} \gamma_i \cdot M^{(i)}$; the prover computes M, w_{ood}, W, and μ.

  4. **Linear evaluation proof.** Set the descriptions

$$[\![w_{\mathsf{ood}}]\!] \coloneqq (\gamma_i, \zeta^{(i)})_{i\in[d]}, \quad [\![W]\!] \coloneqq (\alpha^{(i)}, \beta^{(i)}, \gamma_i, (\gamma'_j, [\![W^{(i)}_j]\!])_{j\in[q_i]})_{i\in[d]}$$

     The prover and verifier jointly run Construction 5.8 on instance $(F, [\![w_{\mathsf{ood}}]\!], f_{\mathsf{ood}}, [\![W]\!], \mu)$, where the point oracles F and f_{ood} are simulated given point query access to $F^{(1)}, \ldots, F^{(d)}$ and $f^{(1)}_{\mathsf{ood}}, \ldots, f^{(d)}_{\mathsf{ood}}, \Pi_{\mathsf{ct}}$. In the honest case, the witness is $(M, \boldsymbol{u})$, where $\boldsymbol{u}(j) \coloneqq \sum_{i\in[d]} \gamma_i \cdot \boldsymbol{u}^{(i)}(j)$ is computed when necessary.

Prover time. Implementing the prover requires the following $\mathbb{F}$-operations (we omit costs which are sublinear in n):

- For each $i \in [d]$, $q_i \cdot n$ multiplications and $q_i \cdot O(n)$ basic operations to compute $W^{(i)}$.
- For each $(i, i') \in I$:
  - $2 \cdot n$ multiplications and $2 \cdot O(n)$ basic operations to compute $m_{\mathsf{ood}}^{(i,i')}$ and $\mu^{(i,i')}$.
  - $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ operations to compute $f_{\mathsf{ood}}^{(i,i')}$.
- $2 \cdot (d-1) \cdot n$ multiplications and $2 \cdot (d-1) \cdot O(n)$ basic operations to compute M and W.
- $(d-1) \cdot k_{\mathsf{col}}$ multiplications and $(d-1) \cdot O(k_{\mathsf{col}})$ basic operations to compute w_{ood}.
- $4 \cdot n$ multiplications, $O(n)$ basic operations, and $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ operations required to implement Construction 5.8's prover.

Remark 5.11 (optimizations)**.** To reduce the randomness complexity of Construction 5.10, one can sample the query compression and batching randomness with a small-bias generator. Separately, the cross term oracle may be "stacked", i.e., of the form $\Pi_{\mathsf{ct}} \colon [\ell_{\mathsf{row}}] \to \mathbb{F}^I$, where $\Pi_{\mathsf{ct}}(j)$ consists of the j-th symbols from each $f_{\mathsf{ood}}^{(i,i')}$. This reduces the verifier's queries to Π_{ct} from $(d-1) \cdot d \cdot t$ to t (but the alphabet grows from $\mathbb{F}$ to $\mathbb{F}^I$).

Lemma 5.12. *Construction 5.10 has round-by-round knowledge soundness errors*

$$\left(\varepsilon_{\mathsf{compress}}, \varepsilon_{\mathsf{batch}}, \varepsilon_{\mathsf{sc}}^{(1)}, \ldots, \varepsilon_{\mathsf{sc}}^{(\kappa_{\mathsf{col}})}, \varepsilon_{\mathsf{cols}}, \varepsilon_{\mathsf{spot}}\right),$$

where:

- $\varepsilon_{\mathsf{compress}} \leq d \cdot |\Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}})| \cdot \frac{1}{|\mathbb{F}|}$.
- $\varepsilon_{\mathsf{batch}} \leq (|\Lambda(\mathcal{C}_{\mathsf{row}}^{d \cdot k_{\mathsf{col}}}, \delta_{\mathsf{row}})| + \ell_{\mathsf{row}} \cdot |\Lambda(\mathcal{C}_{\mathsf{col}}^{d}, \delta_{\mathsf{col}})|) \cdot \frac{2}{|\mathbb{F}|} + k_{\mathsf{col}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}}) + \ell_{\mathsf{row}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})$.
- $\varepsilon_{\mathsf{sc}}^{(i)}$, $\varepsilon_{\mathsf{cols}}$, *and* $\varepsilon_{\mathsf{spot}}$ *are as in Lemma 5.9.*

Moreover, the extractor performs

$$\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \left(\mathsf{errdec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}}) + |\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})| \cdot O(\ell_{\mathsf{col}})\right) + d \cdot (\ell_{\mathsf{row}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{col}}} + k_{\mathsf{col}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{row}}})$$

field operations.

Proof. We define a state function State for Construction 5.10.

1. **Query compression.** At this stage the transcript is empty. The verifier sends $\boldsymbol{\gamma}' \coloneqq (\gamma'_1, \ldots \gamma'_{q^\star})$.

   State function. We set $\mathsf{State}(\mathbb{x}, \boldsymbol{\gamma}', \mathsf{w}) = 1$ for $\mathsf{w} = (M^{(i)})_{i \in [d]}$ if and only if at least one of the following hold:

   (a) For each $i \in [d]$, it holds that $\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}(M^{(i)}) \in \mathcal{L}(\mathsf{Rows}_{F^{(i)}, \mathsf{eq}(\zeta^{(i)}, \cdot), f_{\mathsf{ood}}^{(i)}})$ and $\langle M^{(i)}, W^{(i)} \rangle = \mu_i$.

   (b) For some $i \in [d]$, it holds that $(F^{(i)}, \mathsf{eq}(\zeta^{(i)}, \cdot), f_{\mathsf{ood}}) \notin \mathsf{Bind}$.

   Extractor. The extractor $\mathbf{E}(\mathbb{x}, \boldsymbol{\gamma}', \mathsf{w}) \coloneqq \mathsf{w}$ is the identity function on the state function witness.

2. **Batching randomness.** At this stage the transcript has the form

   $$\mathsf{tr} = (\boldsymbol{\gamma}').$$

   The prover sends Π_{ct} and $\boldsymbol{\mu} \coloneqq (\mu_{i,i'})_{(i,i') \in I}$, and the verifier sends $\boldsymbol{\gamma} \coloneqq (\gamma_2, \ldots, \gamma_d)$.

   State function. We set $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| (\Pi_{\mathsf{ct}}, \boldsymbol{\mu}) \| \boldsymbol{\gamma}, \mathsf{w}) = 1$ if and only if at least one of the following hold:

(a) There exists $\boldsymbol{u} \in \mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$ such that $(\boldsymbol{u}, \mathsf{w}) \in \mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}}$ and $\langle (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u}), W \rangle = \mu$.

(b) $(F, w_{\mathsf{ood}}, f_{\mathsf{ood}}) \notin \mathsf{Bind}$.

Extractor. The extractor $\mathbf{E}(\mathbb{x}, \mathsf{tr} \| (\Pi_{\mathsf{ct}}, \boldsymbol{\mu}) \| \boldsymbol{\gamma}, \mathsf{w})$ parses $\mathsf{w} = (S, (T_j)_{j \in S})$ and outputs $(M^{(i)})_{i \in [d]}$ (or $\perp$ if erasure correction fails in any of the following steps), where each $M^{(i)}$ is computed as follows. First, the extractor constructs $\boldsymbol{f}^{(i)} \colon [\ell_{\mathsf{row}}] \to \mathbb{F}^{k_{\mathsf{col}}} \cup \{\perp\}$, where $\boldsymbol{f}_i(j)$ is the erasure decoding (under $\mathcal{C}_{\mathsf{col}}$) of $F^{(i)}(T_j, j)$ if $j \in S$, or $\perp$ otherwise. Then, the extractor erasure decodes $\boldsymbol{f}_i$ (under $\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$) to obtain $M^{(i)}$.

Extractor time. Implementing the extractor requires the following $\mathbb{F}$-operations (we omit costs which are sublinear in k):

- For each $i \in [d]$:
  - $\ell_{\mathsf{row}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}})$ operations to compute $\boldsymbol{f}^{(i)}$.
  - $k_{\mathsf{col}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{row}}}(\delta_{\mathsf{row}})$ operations to compute $M^{(i)}$.

Error bounds. We upper bound the round-by-round knowledge soundness error based on State.

1. **Query compression.** We show that

$$\Pr_{\boldsymbol{\gamma}'} \left[\exists \mathsf{w} : \begin{array}{l} \mathsf{State}\big(\mathbb{x}, \emptyset, \mathbf{E}(\mathbb{x}, \boldsymbol{\gamma}', \mathsf{w})\big) = 0 \\ \mathsf{State}(\mathbb{x}, \boldsymbol{\gamma}', \mathsf{w}) = 1 \end{array} \right] \leq d \cdot \left| \Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}}) \right| \cdot \frac{1}{|\mathbb{F}|} .$$

Assume that Item 1b does not hold, since otherwise the above event never occurs. For $i \in [d]$, $\boldsymbol{u} \in \mathcal{L}(\mathsf{Rows}_{F^{(i)}, \mathsf{eq}(\zeta^{(i)}, \cdot), f_{\mathsf{ood}}})$, and $\boldsymbol{\gamma}' \in \mathbb{F}^q$ define the event

$$\begin{aligned} E(i, \boldsymbol{u}, \boldsymbol{\gamma}') \coloneqq \Big(\hat{M}(\alpha_i, \beta_i) \neq \mu_{\mathsf{ood}}^{(i)} \vee \exists j \in [q_i], \langle M, W_j^{(i)} \rangle \neq \mu_j^{(i)} \Big) \\ \wedge \langle M, W^{(i)} \rangle = \mu^{(i)} \text{ where } M \coloneqq (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u}) . \end{aligned}$$

By Lemma 3.1, $\Pr_{\boldsymbol{\gamma}'}[E(i, \boldsymbol{u}, \boldsymbol{\gamma}')] \leq \frac{1}{|\mathbb{F}|}$ for any choice of i and $\boldsymbol{u}$. Fix $\boldsymbol{\gamma}'$ such that, for each $i \in [d]$ and $\boldsymbol{u} \in \mathcal{L}(\mathsf{Rows}_{F^{(i)}, \mathsf{eq}(\zeta^{(i)}, \cdot), f_{\mathsf{ood}}^{(i)}})$, the event $E(i, \boldsymbol{u}, \boldsymbol{\gamma}')$ does not occur. By Lemma 5.5 and a union bound, the fraction of $\boldsymbol{\gamma}'$ for which this is not true is at most

$$d \cdot \left| \Lambda(\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}, \delta_{\mathsf{row}}) \right| \cdot \frac{1}{|\mathbb{F}|} .$$

Consider a state function witness $\mathsf{w} = (M^{(i)})_{i \in [d]}$ where $\mathsf{State}(\mathbb{x}, \boldsymbol{\gamma}', \mathsf{w}) = 1$; since Item 1b does not hold, it must be that Item 1a holds. We show that

$$\mathsf{State}\big(\mathbb{x}, \emptyset, \mathbf{E}(\mathbb{x}, \boldsymbol{\gamma}', \mathsf{w})\big) = \mathsf{State}(\mathbb{x}, \emptyset, \mathsf{w}) = 1 ,$$

which proves the upper bound. Consider arbitrary $i \in [d]$. By Item 1a and our choice of $\boldsymbol{\gamma}'$, it must be that $\hat{M}^{(i)}(\alpha_i, \beta_i) = \mu_{\mathsf{ood}}^{(i)}$ and $\langle M, W_j^{(i)} \rangle = \mu_j^{(i)}$ for each $j \in [q_i]$.

2. **Batching randomness.** We show that

$$\Pr_{\boldsymbol{\gamma}}\left[\exists \mathsf{w}: \begin{array}{l}\mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr}\|(\Pi_{\mathsf{ct}}, \boldsymbol{\mu})\|\boldsymbol{\gamma}, \mathsf{w})\big) = 0 \\ \mathsf{State}(\mathbb{x}, \mathsf{tr}\|(\Pi_{\mathsf{ct}}, \boldsymbol{\mu})\|\boldsymbol{\gamma}, \mathsf{w}) = 1\end{array}\right]$$
$$\leq \left(\left|\Lambda(\mathcal{C}_{\mathsf{row}}^{d\cdot k_{\mathsf{col}}}, \delta_{\mathsf{row}})\right| + \ell_{\mathsf{row}} \cdot \left|\Lambda(\mathcal{C}_{\mathsf{col}}^{d}, \delta_{\mathsf{col}})\right|\right) \cdot \frac{2}{|\mathbb{F}|} + k_{\mathsf{col}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}}) + \ell_{\mathsf{row}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}}) .$$

Assume that Item 1b does not hold, since otherwise the above event cannot occur. For each $j \in [\ell_{\mathsf{col}}]$, define the function $\boldsymbol{g}^{(j)} \colon [\ell_{\mathsf{col}}] \to \mathbb{F}^d$ by $g_i^{(j)} := F^{(i)}(\cdot, j)$. For each $i \in [d]$, define the functions $\boldsymbol{f}^{(1)}, \ldots, \boldsymbol{f}^{(d)} \colon [\ell_{\mathsf{row}}] \to \mathbb{F}^{k_{\mathsf{col}}}$ as follows. If there exists an interleaved codeword $\boldsymbol{v} \in \Lambda(\mathcal{C}_{\mathsf{col}}^d, \boldsymbol{g}^{(j)}, \delta_{\mathsf{col}})$ such that, for each $i \in [d]$, it holds that $\hat{h}_i(\zeta^{(i)}) = f_{\mathsf{ood}}^{(i)}(j)$ with $h_i := \mathcal{C}_{\mathsf{col}}^{-1}(v_i)$, then this is unique since Item 1b does not hold; for each $i \in [d]$, set $\boldsymbol{f}^{(i)}(j) := h_i$. Otherwise, set $\boldsymbol{f}^{(1)}(j) = \cdots = \boldsymbol{f}^{(d)}(j) := 0$. Define the relation

$$\mathsf{Rows} := \left\{ \left((\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{(d)}), ((S, T_j)_{j\in S})\right) : \forall i \in [d], \left(\boldsymbol{u}^{(i)}, ((S, T_j)_{j\in S})\right) \in \mathsf{Rows}_{F^{(i)}, \mathsf{eq}(\zeta^{(i)}, \cdot), f_{\mathsf{ood}}^{(i)}} \right\} .$$

By an argument analogous to the proof of Lemma 5.5, it holds that $|\mathcal{L}(\mathsf{Rows})| \leq |\Lambda(\mathcal{C}_{\mathsf{row}}^{d\cdot k_{\mathsf{col}}}, \delta_{\mathsf{row}})|$. For $(\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{d}) \in \mathcal{L}(\mathsf{Rows})$ and $\boldsymbol{\gamma} \in \mathbb{F}^d$, define the event

$$E_1(\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{(d)}, \boldsymbol{\gamma}) := \left(\exists i \in [d], \left\langle (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{row}}})^{-1}(\boldsymbol{u}^{(i)}), W^{(i)} \right\rangle \neq \mu^{(i)}\right)$$
$$\wedge \left\langle (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{row}}})^{-1}(\boldsymbol{u}), W \right\rangle = \mu \text{ where } \boldsymbol{u} := \sum_{i\in[d]} \gamma_i \cdot \boldsymbol{u}^{(i)} .$$

By Lemma 3.1, $\Pr_{\boldsymbol{\gamma}}[E_1(\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{(d)}, \boldsymbol{\gamma})] \leq \frac{2}{|\mathbb{F}|}$ for any choice of $\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{(d)}$. For $j \in [\ell_{\mathsf{row}}]$, $\boldsymbol{v} \in \Lambda(\mathcal{C}_{\mathsf{col}}^d, \boldsymbol{g}^{(j)}, \delta_{\mathsf{col}})$, and $\boldsymbol{\gamma} \in \mathbb{F}^{d-1}$, define the event

$$E_2(j, \boldsymbol{v}, \boldsymbol{\gamma}) := \left(\exists i \in [d], \hat{h}_i(\zeta^{(i)}) \neq f_{\mathsf{ood}}^{(i)}(j)\right)$$
$$\wedge \langle h, W \rangle = f_{\mathsf{ood}}(j) \text{ where } h_i := \mathcal{C}_{\mathsf{col}}^{-1}(v_i) , h := \sum_{i\in[d]} \gamma_i \cdot h_i .$$

By Lemma 3.1, $\Pr_{\boldsymbol{\gamma}}[E_2(j, \boldsymbol{v}, \boldsymbol{\gamma})] \leq \frac{2}{|\mathbb{F}|}$ for any choice of j and $\boldsymbol{v}$. Fix $\boldsymbol{\gamma}$ such that the following hold:

(i) For each $(\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{(d)}) \in \mathcal{L}(\mathsf{Rows})$, the event $E_1(\boldsymbol{u}^{(1)}, \ldots, \boldsymbol{u}^{(d)}, \boldsymbol{\gamma})$ does not occur. By a union bound and our bound on the size of $\mathcal{L}(\mathsf{Rows})$, the fraction of $\boldsymbol{\gamma}$ for which this does not hold is at most $|\Lambda(\mathcal{C}_{\mathsf{row}}^{d\cdot k_{\mathsf{col}}}, \delta_{\mathsf{row}})| \cdot \frac{2}{|\mathbb{F}|}$.

(ii) For each $j \in [\ell_{\mathsf{row}}]$ and $\boldsymbol{v} \in \Lambda(\mathcal{C}_{\mathsf{col}}^d, \boldsymbol{g}_j, \delta_{\mathsf{col}})$, the event $E_2(j, \boldsymbol{v})$ does not occur. By a union bound, the fraction of $\boldsymbol{\gamma}$ for which this does not hold is at most $\ell_{\mathsf{row}} \cdot |\Lambda(\mathcal{C}_{\mathsf{col}}^d, \delta_{\mathsf{col}})| \cdot \frac{2}{|\mathbb{F}|}$.

(iii) For any subset $S \subseteq [\ell_{\mathsf{row}}]$ with $|S| \geq (1 - \delta_{\mathsf{row}}) \cdot \ell_{\mathsf{row}}$ such that $\boldsymbol{f} := \sum_{i\in[d]} \gamma_i \cdot \boldsymbol{f}^{(i)}$ agrees with $\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$ on S, it holds that $\boldsymbol{f}^{(i)}$ agrees with $\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$ on S. By Definition 3.20, the fraction of $\boldsymbol{\gamma}$ for which this is not true is at most $k_{\mathsf{col}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}})$.

(iv) For each $j \in [\ell_{\mathsf{row}}]$ and any subset $T \subseteq [\ell_{\mathsf{col}}]$ with $|T| \geq (1 - \delta_{\mathsf{col}}) \cdot \ell_{\mathsf{col}}$ such that $F(\cdot, j)$ agrees with $\mathcal{C}_{\mathsf{col}}$ on T, it holds that $\boldsymbol{g}^{(j)}$ agrees with $\mathcal{C}_{\mathsf{col}}^d$ on T. By Definition 3.20 and a union bound, the fraction of $\boldsymbol{\gamma}$ for which this is not true is at most $\ell_{\mathsf{row}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})$.

Overall, the fraction of $\boldsymbol{\gamma}$ for which any condition does not hold is at most

$$\left(\left|\Lambda(\mathcal{C}_{\mathsf{row}}^{d\cdot k_{\mathsf{col}}}, \delta_{\mathsf{row}})\right| + \ell_{\mathsf{row}} \cdot \left|\Lambda(\mathcal{C}_{\mathsf{col}}^{d}, \delta_{\mathsf{col}})\right|\right) \cdot \frac{2}{|\mathbb{F}|} + k_{\mathsf{col}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{row}}, \delta_{\mathsf{row}}) + \ell_{\mathsf{row}} \cdot \varepsilon_{\mathsf{PG_d}}(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}}) \,.$$

Consider a state function witness $\mathsf{w} = (S, T_j)_{j\in S}$ where $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| (\Pi_{\mathsf{ct}}, \boldsymbol{\mu}) \| \boldsymbol{\gamma}, \mathsf{w}) = 1$. We show that

$$\mathsf{State}\big(\mathbb{x}, \mathsf{tr}, (M^{(i)})_{i\in[d]}\big) = 1 \text{ for } (M^{(i)})_{i\in[d]} \coloneqq \mathbf{E}(\mathbb{x}, \mathsf{tr} \| (\Pi_{\mathsf{ct}}, \boldsymbol{\mu}) \| \boldsymbol{\gamma}, \mathsf{w}) \,,$$

which proves the upper bound. We first argue that Item 2b cannot hold. For arbitrary $j \in [\ell_{\mathsf{row}}]$, consider the set

$$\Lambda\big(\mathcal{C}_{\mathsf{col}}, F(\cdot, j), \delta_{\mathsf{col}}\big) = \left\{ \sum_{i\in[d]} \gamma_i \cdot v_i : \boldsymbol{v} \in \Lambda(\mathcal{C}_{\mathsf{col}}, \boldsymbol{g}^{(j)}, \delta_{\mathsf{col}}) \right\} ,$$

where equality is due to Item (iv) and the fact that $\delta_{\mathsf{col}} < \delta(\mathcal{C}_{\mathsf{col}})$. Since Item 1b does not hold, there exists at most one choice of $\boldsymbol{v} \in \Lambda(\mathcal{C}_{\mathsf{col}}, \boldsymbol{g}^{(j)}, \delta_{\mathsf{col}})$ where, for each $i \in [d]$, it holds that $\hat{h}_i(\zeta^{(i)}) = f_{\mathsf{ood}}^{(i)}(j)$ with $h_i \coloneqq \mathcal{C}_{\mathsf{col}}^{-1}(v_i)$. By Item (ii), any other choice of $\boldsymbol{v}$ yields $v \coloneqq \sum_{i\in[d]} \gamma_i \cdot v_i$ such that $\langle \mathcal{C}_{\mathsf{col}}^{-1}(v), w_{\mathsf{ood}} \rangle \neq f_{\mathsf{ood}}(j)$.

Since $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| (\Pi_{\mathsf{ct}}, \boldsymbol{\mu}) \| \boldsymbol{\gamma}, \mathsf{w}) = 1$ and Item 2b does not hold, it must be that Item 2a holds. In other words, there exists an interleaved codeword $\boldsymbol{u} \in \mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$ such that $(\boldsymbol{u}, \mathsf{w}) \in \mathsf{Rows}_{F, w_{\mathsf{ood}}, f_{\mathsf{ood}}}$ and $\langle (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u}), W \rangle = \mu$. Consider arbitrary $j \in S$ and let $h \coloneqq \boldsymbol{v}(j)$. Observe that the function $F(\cdot, j)$ agrees with $\mathcal{C}_{\mathsf{col}}(\boldsymbol{u}(j))$ on T_j. By Item (iv), $\boldsymbol{g}^{(j)}$ agrees with a codeword $\boldsymbol{v} \in \mathcal{C}_{\mathsf{col}}^{d}$ on T_j; for each $i \in [d]$, let $h_i \coloneqq \mathcal{C}_{\mathsf{col}}^{-1}(v_i)$. Observe that $h = \sum_{i\in[d]} \gamma_i \cdot h_i$, since $\delta_{\mathsf{col}} < \delta(\mathcal{C}_{\mathsf{col}})$. Since $\langle h, w_{\mathsf{ood}} \rangle = f_{\mathsf{ood}}(j)$ and Item (ii) holds, it must be that $\hat{h}_i(\zeta^{(i)}) = f_{\mathsf{ood}}(j)$ for each $i \in [d]$. By construction, it must be that $\boldsymbol{f}^{(i)}(j) = h_i$ for each $i \in [d]$, and hence $\sum_{i\in[d]} \gamma_i \cdot \boldsymbol{f}^{(i)}(j) = h$.

We have shown that that $\sum_{i\in[d]} \gamma_i \cdot \boldsymbol{f}^{(i)}$ agrees with $\boldsymbol{u}$ on S. By Item (iii), each $\boldsymbol{f}^{(i)}$ agrees with an interleaved codeword $\boldsymbol{u}^{(i)} \in \mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}}$ on S. Consider arbitrary $i \in [d]$. Observe that $\boldsymbol{u} = \sum_{i\in[d]} \gamma_i \cdot \boldsymbol{u}^{(i)}$, since $\delta_{\mathsf{col}} < \delta(\mathcal{C}_{\mathsf{row}})$. Since $\langle (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{row}}})^{-1}(\boldsymbol{u}), W \rangle = \mu$ and Item (i) holds, it must be that $\langle (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u}^{(i)}), W^{(i)} \rangle = \mu^{(i)}$ for each $i \in [d]$. Finally, observe that the extractor outputs $M^{(i)} \coloneqq (\mathcal{C}_{\mathsf{row}}^{k_{\mathsf{col}}})^{-1}(\boldsymbol{u}^{(i)})$. We conclude that Item 1a holds.

□

6 Multilinear polynomial commitments with linear oracles

We construct (query-optimized) multilinear polynomial commitments in the point/linear query model.

Lemma 6.1. *Fix linear codes* $\mathcal{C}_{\mathsf{row}} \colon \mathbb{F}^{k_{\mathsf{row}}} \to \mathbb{F}^{\ell_{\mathsf{row}}}$ *and* $\mathcal{C}_{\mathsf{col}} \colon \mathbb{F}^{k_{\mathsf{col}}} \to \mathbb{F}^{\ell_{\mathsf{col}}}$ *where* $k_{\mathsf{row}}, k_{\mathsf{col}} \in 2^{\mathbb{N}}$. *Let* $n \coloneqq k_{\mathsf{row}} \cdot k_{\mathsf{col}}$. *Fix repetition parameters* $t \in \mathbb{N}$ *and* $s \in 2^{\mathbb{N}}$. *There exists an interactive* $\mathsf{MLP}_{\log n}$*-commitment such that (differences between Lemma 5.1 are red):*

- *The commitment phase has the following efficiency measures.*
  - Rounds. *The protocol has* 2 *rounds.*
  - Non-oracle commitment length. *The committer sends* 1 *field element.*
  - Commitment oracles. *The committer sends* 1 *linear oracle of* k_{row} *field elements and* 1 *point oracle of* $\ell_{\mathsf{row}} \cdot \ell_{\mathsf{col}}$ *field elements.*
  - Committer time. *The committer performs* $(k_{\mathsf{col}} + 1) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{col}}}$ *field operations,* $3 \cdot n + k_{\mathsf{col}}$ *field multiplications, and* $O(n)$ *basic field operations.*
- *For* d *commitments and* q *evaluation claims, the opening phase has the following efficiency measures.*
  - Rounds. *The protocol has* $\log k_{\mathsf{col}} + 4$ *rounds.*
  - Non-oracle proof length. *The prover sends* $2 \cdot \log k_{\mathsf{col}} + 2 \cdot \binom{d}{2}$ *field elements.*
  - Proof oracles. *The prover sends* 3 *linear oracles of* k_{row}, $s \cdot k_{\mathsf{col}}$, *and* $2 \cdot \binom{d}{2} \cdot k_{\mathsf{row}}$ *field elements.*
  - Commitment query complexity. *For each* $i \in [d]$, *the verifier issues* t *linear queries and* t *point queries to the* i*-th commitment's oracles.*
  - Proof query complexity. *The verifier issues* $(4 + 2 \cdot \binom{d}{2}) \cdot t + 1$ *linear queries to the proof oracles.*
  - Prover time. *The prover performs* $(2 \cdot \binom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ *field operations,*

$$4 \cdot \binom{d}{2} \cdot n + (d-1) \cdot (2 \cdot n + k_{\mathsf{col}}) + (2 \cdot q + 4) \cdot n$$

    field multiplications, and $O((d^2 + q) \cdot n)$ *basic field operations.*
  - Verifier time. *The verifier performs* $O(\log k_{\mathsf{col}} + d^2 + q + t)$ *field operations.*

The round-by-round errors are as in Lemma 5.1; the extractor performs an additional $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ *field operations.*

Proof. For a function $f \colon \{0,1\}^{\log n} \to \mathbb{F}$, the multilinear polynomial evaluation $f(\boldsymbol{x})$ is equivalent to the inner product $\langle f, \mathsf{eq}(\boldsymbol{x}, \cdot) \rangle$. We use the interactive Lin_n-commitment established in Lemma 5.1 to construct a $\mathsf{MLP}_{\log n}$-commitment; the additional prover time is due to the prover needing to explicitly compute the weight vector $\mathsf{eq}(\boldsymbol{x}, \cdot)$ for each evaluation query $\boldsymbol{x} \in \mathbb{F}^{\log n}$.

In order to minimize the number of point queries issued in the opening phase, we also modify the Section 5's constructions as follows. Instead of sending the column OOD evaluations f_{ood} (Construction 5.6) as a point oracle, the prover sends it as a linear oracle. In fact, since $f_{\mathsf{ood}} \in \mathcal{C}_{\mathsf{row}}$ is a codeword in the honest case, the prover sends its *decoding* m_{ood} as a linear oracle. A point query to f_{ood} with index $y \in [\ell_{\mathsf{row}}]$ can be simulated with a linear query to m_{ood} with weight vector $G_{\mathcal{C}_{\mathsf{row}}}(y, \cdot)$. Similarly, the prover can send the cross terms Π_{ct} (Construction 5.10) as a linear oracle. □

7 Right-extensions of linear codes

We show how to compute or delegate *right extensions* of linear codes.

Let $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ be a linear code. Overloading notation, we view the generator matrix $G_\mathcal{C}$ as a function $G_\mathcal{C}\colon \{0,1\}^{\log \ell} \times \{0,1\}^{\log k} \to \mathbb{F}$ in the natural way. Let $\hat{G}_\mathcal{C}\colon \mathbb{F}^{\log \ell} \times \mathbb{F}^{\log k} \to \mathbb{F}$ denote the multilinear extension of the above function. Further, for any $\boldsymbol{\beta} \in \mathbb{F}^{\log k}$, we let $\mathsf{eq}_{\boldsymbol{\beta}} \coloneqq (\mathsf{eq}(\boldsymbol{b}, \boldsymbol{\beta}))_{\boldsymbol{b} \in \{0,1\}^{\log k}}$.

Proposition 7.1. *For any $\boldsymbol{\alpha} \in \mathbb{F}^{\log \ell}$ and $\boldsymbol{\beta} \in \mathbb{F}^{\log k}$ it holds that $\hat{G}_\mathcal{C}(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \widehat{\mathcal{C}(\mathsf{eq}_{\boldsymbol{\beta}})}(\boldsymbol{\alpha})$.*

Proof.

$$\begin{aligned}
\hat{G}_\mathcal{C}(\boldsymbol{\alpha}, \boldsymbol{\beta}) &= \sum_{(\boldsymbol{a},\boldsymbol{b}) \in \{0,1\}^{\log \ell + \log k}} G_\mathcal{C}[\boldsymbol{a}, \boldsymbol{b}] \cdot \mathsf{eq}(\boldsymbol{a}, \boldsymbol{\alpha}) \cdot \mathsf{eq}(\boldsymbol{b}, \boldsymbol{\beta}) \\
&= \sum_{\boldsymbol{a} \in \{0,1\}^{\log \ell}} \mathsf{eq}(\boldsymbol{a}, \boldsymbol{\alpha}) \cdot \left(\sum_{\boldsymbol{b} \in \{0,1\}^{\log k}} G_\mathcal{C}[\boldsymbol{a}, \boldsymbol{b}] \cdot \mathsf{eq}(\boldsymbol{b}, \boldsymbol{\beta}) \right) \\
&= \sum_{\boldsymbol{a} \in \{0,1\}^{\log \ell}} \mathsf{eq}(\boldsymbol{a}, \boldsymbol{\alpha}) \cdot \mathcal{C}(\mathsf{eq}_{\boldsymbol{\beta}})[\boldsymbol{a}] \\
&= \widehat{\mathcal{C}(\mathsf{eq}_{\boldsymbol{\beta}})}(\boldsymbol{\alpha}) \,.
\end{aligned}$$

□

The quantity above implicitly appears in [ACFY25] for the special case where $\boldsymbol{\alpha} \in \{0,1\}^{\log \ell}$, which we refer to as a right-extension of $\mathcal{C}$.

Definition 7.2. *A linear code $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ has* **right-extension complexity** *t_{rex} if for any $\boldsymbol{b} \in \{0,1\}^{\log \ell}$ and $\boldsymbol{\beta} \in \mathbb{F}^{\log k}$ the quantity $\mathcal{C}(\mathsf{eq}_{\boldsymbol{\beta}})[\boldsymbol{b}]$ can be computed in at most t_{rex} field operations. For $t \in \mathbb{N}$, the* **t-right-extension complexity** *of $\mathcal{C}$ is the complexity of computing $\mathcal{C}(\mathsf{eq}_{\boldsymbol{\beta}})[\boldsymbol{b}]$ for t distinct values of $\boldsymbol{b}$ (and a fixed $\boldsymbol{\beta}$).*

For any linear code $\mathcal{C}$, the right-extension can be computed in time proportional to its encoding time $O(\mathsf{enc}_\mathcal{C})$ field operations. In fact, computing the right-extension of t many points (at a fixed $\boldsymbol{\beta}$) can also be computed in the same time (and independent of t).

Fact 7.3. Let $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ be a linear code. Then $\mathcal{C}$ has t-right-extension complexity $O(\mathsf{enc}_\mathcal{C})$.

7.1 Succinct right-extensions

Some families of code admit *succinct right-extensions*, i.e., the extensions can be computed in $O(\log \ell)$ field operations. One such example is Reed–Solomon codes, as was implicitly observed by [ACFY25] for fields of non-binary characteristic and by [NA25] in the binary characteristic case.

Definition 7.4. *Let $k \in \mathbb{N}$, $\mathbb{F}$ be a finite field, and $\mathcal{L} \subseteq \mathbb{F}$ be an evaluation domain. We define the* **Reed–Solomon code**

$$\mathsf{RS}[\mathbb{F}, \mathcal{L}, k] = \left\{ f\colon \mathcal{L} \to \mathbb{F} : \exists \hat{f} \in \mathbb{F}^{<k}[X] \text{ s.t. } \forall x \in \mathcal{L}, \hat{f}(x) = f(x) \right\} .$$

Throughout, we let $\ell := |\mathcal{L}|$ and further assume that there exists a bijection $\varphi\colon \{0,1\}^{\log \ell} \to \mathcal{L}$ computable in $O(\log \ell)$ field operations.[19]

Lemma 7.5. *Let $\mathcal{C} := \mathsf{RS}[\mathbb{F}, \mathcal{L}, k]$ be a Reed–Solomon code. Then $\mathcal{C}$ has right-extension complexity $O(\log \ell)$.*

Proof. Let $\varphi\colon \{0,1\}^{\log \ell} \to \mathcal{L}$ be a bijection computable in $O(\log \ell)$ field operations. For any $\boldsymbol{x} \in \{0,1\}^{\log \ell}$ and $\boldsymbol{b} \in \{0,1\}^{\log k}$, it holds that

$$G_{\mathcal{C}}[\boldsymbol{x}, \boldsymbol{b}] = \varphi(\boldsymbol{x})^{\sum_{t \in [\log k]} b_t \cdot 2^{t-1}} = \prod_{t \in [\log k]} \varphi(\boldsymbol{x})^{b_t \cdot 2^{t-1}} = \prod_{t \in [\log k]} \left((1 - b_t) \cdot 1 + b_t \cdot \varphi(\boldsymbol{x})^{2^{t-1}}\right).$$

The claim follows by observing that this quantity is multilinear in $\boldsymbol{b}$ and therefore computable in $O(\log \ell)$ field operations. □

Corollary 7.6. *Let $\mathcal{C} := \mathsf{RS}[\mathbb{F}, \mathcal{L}, k]$ be a Reed–Solomon code. Then $\mathcal{C}$ has t-right-extension complexity at most $O(t \cdot \log \ell)$.*

7.2 Delegating right-extension computations

To achieve linear prover time, our main construction must employ a linear-time encodable code, which precludes Reed–Solomon codes. To our knowledge, no linear-time encodable code admits a succinct right-extension. Instead, we achieve succinct verification by delegating this computation to the prover.

We show that, in the (holographic) polynomial IOP model, every code admits a delegation protocol.

Definition 7.7. *For a linear code $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ and $t \in \mathbb{N}$, the* **t-right-extension delegation language of $\mathcal{C}$** *is*

$$\mathcal{L}_{\mathcal{C},t} := \left\{(\boldsymbol{b}_1, \mu_1, \ldots, \boldsymbol{b}_t, \mu_t, \boldsymbol{\beta}) : \forall i \in [t], \mathcal{C}(\mathsf{eq}_{\boldsymbol{\beta}})[\boldsymbol{b}_i] = \mu_i\right\}.$$

Lemma 7.8. *Fix a linear code $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ and $t \in \mathbb{N}$. There exists a (holographic) polynomial IOP for $\mathcal{L}_{\mathcal{C},t}$ with the following characteristics:*

- Index oracles. *The indexer outputs $O(1)$ polynomial oracles, each containing $\tilde{O}(\mathsf{enc}_{\mathcal{C}})$ field elements.*
- Indexer time. *The indexer performs $\tilde{O}(\mathsf{enc}_{\mathcal{C}})$ field operations.*
- Round complexity. *The protocol has $O(\log \mathsf{enc}_{\mathcal{C}})$ rounds.*
- Non-oracle proof length. *The prover sends $O(\log \mathsf{enc}_{\mathcal{C}})$ field elements.*
- Query complexity. *The verifier issues a single (polynomial) query to each of the indexer's oracles.*
- Prover time. *The prover performs $\tilde{O}(\mathsf{enc}_{\mathcal{C}})$ field operations.*
- Verifier time. *The verifier performs $O(\log \mathsf{enc}_{\mathcal{C}})$ field operations.*

The round-by-round soundness error is $O\left(\frac{\log \mathsf{enc}_{\mathcal{C}}}{|\mathbb{F}|}\right)$.

[19] This in particular holds for the common case of Reed–Solomon codes over a multiplicative subgroup $\langle \omega \rangle$ of $\mathbb{F}$, where the map is $\varphi(\boldsymbol{b}) = \omega^{\mathsf{int}(\boldsymbol{b})}$ ($\mathsf{int}(\boldsymbol{b})$ is the integer whose bit representation is $\boldsymbol{b}$).

Proof. By Fact 7.3, $\mathcal{L}_{\mathcal{C},t}$ can be decided in $O(\mathsf{enc}_{\mathcal{C}})$ field operations, and as thus it can be represented as a R1CS instance of size $O(\mathsf{enc}_{\mathcal{C}})$. The statement follows by using a holographic PIOP for R1CS, e.g. [Set19], instantiated with the permutation argument in [CBBZ23, Sec 3.6] (and noticing that, since the computation is deterministic, the prover does not need to send any witness). □

Remark 7.9. We remark that variations of Lemma 7.8 can be obtained by using different polynomial IOPs for NP with succinct verification, e.g., [GWC19; CHMMVW20; CBBZ23].

The following is an application of Lemma A.1 to Lemma 7.8. The specific polynomial class (e.g., univariate or multilinear) is not important, so long as it can be modeled as a multilinear extension.

Corollary 7.10. *Fix a linear code $\mathcal{C}\colon \mathbb{F}^k \to \mathbb{F}^\ell$ and $t \in \mathbb{N}$ and suppose that $\mathsf{FC} = (\mathsf{Commit}, \mathsf{Open})$ is an interactive polynomial commitment (which only sends point-oracles). There exists a (holographic) IOP for $\mathcal{L}_{\mathcal{C},t}$ with the following characteristics:*

- Index oracles. *The indexer outputs the same oracles as produced by $O(1)$ executions of the* Commit *committer.*
- Indexer time. *The indexer performs $\tilde{O}(\mathsf{enc}_{\mathcal{C}})$ field operations and $O(1)$ executions of the* Commit *committer.*
- Round complexity. *The protocol has $O(\log \mathsf{enc}_{\mathcal{C}})$ rounds plus the round complexity of an execution of* Open *on $O(1)$ commitments and $O(1)$ evaluations claims.*
- Non-oracle proof length. *The prover sends $O(\log \mathsf{enc}_{\mathcal{C}})$ field elements plus the non-oracle proof length of an execution of* Open *on $O(1)$ commitments and $O(1)$ evaluations claims.*
- Proof oracles. *The prover sends the oracles corresponding to an execution of* Open *on $O(1)$ commitments and $O(1)$ evaluations claims.*
- Query complexity. *The verifier has the query complexity of an execution of* Open *on $O(1)$ commitments and $O(1)$ evaluations claims.*
- Prover time. *The prover performs $\tilde{O}(\mathsf{enc}_{\mathcal{C}})$ field operations plus the prover time of an execution of* Open *on $O(1)$ commitments and $O(1)$ evaluations claims.*
- Verifier time. *The prover performs $O(\log \mathsf{enc}_{\mathcal{C}})$ field operations plus the verifier time of an execution of* Open *on $O(1)$ commitments and $O(1)$ evaluations claims.*

The round-by-round soundness error is the maximum of the round-by-round knowledge soundness error of Open *and $O\left(\frac{\log(\mathsf{enc}_{\mathcal{C}})}{|\mathbb{F}|}\right)$.*

8 Multilinear polynomial commitments with point oracles

We construct multilinear polynomial commitments in the point query model. To bootstrap our construction, we use the following (trivial) commitment scheme which does not send oracles.

Lemma 8.1. *Fix $n \in \mathbb{N}$. There exists an interactive Lin_n-commitment such that:*

- *The commitment phase has the following efficiency measures.*
  - Rounds. *The protocol has 1 round.*
  - Commitment length. *The prover sends n field elements (as a non-oracle message).*
  - Committer time. *The prover performs no field operations.*
- *For d commitments and q evaluations claims, the opening phase has the following efficiency measures.*
  - Rounds. *The protocol has $\log n + 2$ rounds.*
  - Proof length. *The prover sends $2 \cdot \binom{d}{2} + 2 \cdot \log n$ field elements (as a non-oracle message).*
  - Prover time. *The prover performs*

$$2 \cdot \binom{d}{2} \cdot n + (d-1) \cdot n + (q+4) \cdot n$$

    multiplications and $O((d^2+q) \cdot n)$ basic field operations.
  - Verifier time. *The verifier performs $O(d \cdot n + d^2 + q)$ field operations and, for each of the q evaluation claims, 1 query to its multilinear extension.*

The commitment phase has round-by-round binding error 0 and the opening phase has round-by-round knowledge soundness error $\frac{2}{|\mathbb{F}|}$; the extractor performs no field operations.

Proof. In the commitment phase, the prover simply sends the message. In the opening phase, the prover and verifier engage in a (batched) sumcheck protocol to reduce the evaluations to a multilinear extension of the message, that the verifier computes directly. □

8.1 Linear commitments with point oracles

We recursively apply Corollary A.5 with Lemma 5.1 (or Lemma 8.1 in the base case) to obtain a linear commitment in the point query model.

Lemma 8.2. *Let $M \in \mathbb{N}$ be a number of iterations. For $i \in [M]$, fix the following parameters:*

- *Linear codes $\mathcal{C}^{(i)}_{\mathsf{row}} \colon \mathbb{F}^{k^{(i)}_{\mathsf{row}}} \to \mathbb{F}^{\ell^{(i)}_{\mathsf{row}}}$ and $\mathcal{C}^{(i)}_{\mathsf{col}} \colon \mathbb{F}^{k^{(i)}_{\mathsf{col}}} \to \mathbb{F}^{\ell^{(i)}_{\mathsf{col}}}$, where $k^{(i)}_{\mathsf{row}}, k^{(i)}_{\mathsf{col}} \in 2^{\mathbb{N}}$. Let $n^{(i)} \coloneqq k^{(i)}_{\mathsf{row}} \cdot k^{(i)}_{\mathsf{col}}$. Let $\mathsf{enc}_{\mathcal{C}^{(i)}} \coloneqq k_{\mathsf{col}} \cdot \mathsf{enc}_{\mathcal{C}^{(i)}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \mathsf{enc}_{\mathcal{C}^{(i)}_{\mathsf{col}}}$ denote the number of field operations used to encode a message under the tensor code $\mathcal{C}^{(i)} \coloneqq \mathcal{C}^{(i)}_{\mathsf{col}} \otimes \mathcal{C}^{(i)}_{\mathsf{row}}$.*
- *Repetition parameters $t^{(i)} \in \mathbb{N}$ and $s^{(i)} \in 2^{\mathbb{N}}$.*

For $i \in [2, M]$, assume that $n^{(i)} \geq \max\{k^{(i-1)}_{\mathsf{row}}, s^{(i-1)} \cdot k^{(i-1)}_{\mathsf{col}}\}$. Let $n^\star \coloneqq \max\{k^{(M)}_{\mathsf{row}}, t^{(M)} \cdot k^{(M)}_{\mathsf{col}}\}$. There exists an interactive $\mathsf{Lin}_{n^{(1)}}$-commitment such that:

- *The commitment phase has the following efficiency measures.*

- Rounds. *The protocol has* 2 *rounds.*
- Non-oracle commitment length. *The committer sends* 1 *field element.*
- Commitment oracles. *The committer sends* 2 *point oracles of* $\ell^{(1)}_{\mathsf{row}} \cdot \ell^{(1)}_{\mathsf{col}} + \ell^{(1)}_{\mathsf{row}}$ *field elements in total.*
- Committer time. *The committer performs* $\mathsf{enc}_{\mathcal{C}^{(1)}} + \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}}$ *field operations,* $3 \cdot n^{(1)}$ *field multiplications, and* $O(n^{(1)})$ *basic field operations.*

- *For* d *commitments and* q *evaluation claims, the opening phase has the following efficiency measures.*
  - Round complexity. *The protocol has* $(\sum_{i\in[M]} \log k^{(i)}_{\mathsf{col}}) + \log n^\star + 6 \cdot M$ *rounds.*
  - Non-oracle proof length. *The prover sends* $2 \cdot \binom{d}{2} + (\sum_{i\in[M]} 2 \cdot \log k^{(i)}_{\mathsf{col}}) + 2 \cdot \log n^\star + 2 \cdot n^\star + (\sum_{i\in[M]} 4 \cdot t^{(i)}) + 5 \cdot M - 2$ *field elements.*
  - Proof oracles. *The prover sends* $5 \cdot M - 4$ *point oracles of* $2 \cdot \binom{d}{2} \cdot \ell^{(1)}_{\mathsf{row}} + \sum_{i\in[2,M]} 2 \cdot \ell^{(i)}_{\mathsf{row}} \cdot \ell^{(i)}_{\mathsf{col}} + 4 \cdot \ell^{(i)}_{\mathsf{row}}$ *field elements in total.*
  - Query complexity. *The verifier issues* $2 \cdot d \cdot t^{(1)} + 2 \cdot \binom{d}{2} \cdot t^{(1)} + \sum_{i\in[2,M]} 6 \cdot t^{(i)}$ *point queries in total.*
  - Prover time. *The prover performs*

$$(2 \cdot \tbinom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}} + \sum_{i\in[2,M]} 2 \cdot \mathsf{enc}_{\mathcal{C}^{(i)}} + 2 \cdot \mathsf{enc}_{\mathcal{C}^{(i)}_{\mathsf{row}}} + 6 \cdot \mathsf{enc}_{\mathcal{C}^{(i)}_{\mathsf{row}}}$$

    field operations,

$$\begin{aligned} &2 \cdot \tbinom{d}{2} \cdot n^{(1)} + (d-1) \cdot (2 \cdot n^{(1)} + k^{(1)}_{\mathsf{col}}) + (q+4) \cdot n^{(1)} \\ &\quad + \left(\sum_{i\in[2,M]} (4 \cdot t^{(i-1)} + 9) \cdot n^{(i)} + k^{(i)}_{\mathsf{col}} \right) + (4 \cdot t^{(M)} + 8) \cdot n^\star \end{aligned}$$

    field multiplications, and $O((d^2+q) \cdot n^{(1)} + (\sum_{i\in[2,M]} t^{(i-1)} \cdot n^{(i)}) + t^{(M)} \cdot n^\star)$ *basic field operations.*
  - Verifier time. *The verifier performs* $O(d^2 + q + (\sum_{i\in[M]} \log k^{(i)}_{\mathsf{col}} + t^{(i)} \cdot \log s^{(i)}) + n^\star)$ *field operations and:*
    - *For each of the* q *evaluation claims,* 1 *query to its multilinear extension.*
    - *For each* $i \in [M]$, $t^{(i)}$ *queries to the right-extension of* $\mathcal{C}^{(i)}_{\mathsf{row}}$.
    - *For each* $i \in [M]$, $t^{(i)}$ *queries to the right-extension of* $\mathcal{C}^{(i)}_{\mathsf{col}}$.

Fix security parameter $\lambda \in \mathbb{N}$. *For each* $i \in [M]$, *fix proximity bounds* $\delta^{(i)}_{\mathsf{row}} \in (0, \delta(\mathcal{C}^{(i)}_{\mathsf{row}}))$ *and* $\delta^{(i)}_{\mathsf{col}} \in (0, \delta(\mathcal{C}^{(i)}_{\mathsf{col}}))$. *Assuming each set of parameters satisfy the conditions in Lemma 5.1, the commitment phase has round-by-round binding error at most* $2^{-\lambda}$ *and the opening phase has round-by-round knowledge soundness error at most* $2^{-\lambda}$*; the extractor performs*

$$\begin{aligned} &\sum_{i\in[M]} \mathsf{enc}_{\mathcal{C}^{(i)}_{\mathsf{row}}} + \ell^{(i)}_{\mathsf{row}} \cdot \left(\mathsf{errdec}_{\mathcal{C}^{(i)}_{\mathsf{col}}}(\delta^{(i)}_{\mathsf{col}}) + |\Lambda(\mathcal{C}^{(i)}_{\mathsf{col}}, \delta^{(i)}_{\mathsf{col}})| \cdot O(\ell^{(i)}_{\mathsf{col}}) \right) \\ &\quad + d \cdot (\ell^{(1)}_{\mathsf{row}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{col}}}(\delta^{(1)}_{\mathsf{col}}) + k^{(1)}_{\mathsf{col}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{row}}}(\delta^{(1)}_{\mathsf{row}})) \\ &\quad + \sum_{i\in[2,M]} 2 \cdot (\ell^{(i)}_{\mathsf{row}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{col}}}(\delta^{(i)}_{\mathsf{col}}) + k^{(i)}_{\mathsf{col}} \cdot \mathsf{eradec}_{\mathcal{C}_{\mathsf{row}}}(\delta^{(i)}_{\mathsf{row}})) \end{aligned}$$

field operations.

Proof. Let T be the transformation described in Corollary A.5. By Lemma 5.1, there exists an interactive $\mathsf{Lin}_{n^{(1)}}$-commitment $\mathsf{FC}^{(1)}$ using codes $\mathcal{C}^{(1)}_{\mathsf{row}}$ and $\mathcal{C}^{(1)}_{\mathsf{col}}$. Recall that the prover sends 2 linear oracles of $k^{(1)}_{\mathsf{row}}$ and $s^{(1)} \cdot k^{(1)}_{\mathsf{col}}$ field elements. We prove the theorem by induction on M.

Base case. Suppose $M = 1$, i.e., we perform one step of recursion. Recall that $n^\star$ is defined to be $\max\{k^{(1)}_{\mathsf{row}}, s^{(1)} \cdot k^{(1)}_{\mathsf{col}}\}$. Without loss of generality, we may assume that the prover of $\mathsf{FC}^{(1)}$, by padding with zeros, sends linear oracles of $n^\star$ field elements (the other efficiency measures are unaffected). Let $\mathsf{FC}^\star$ denote the interactive $\mathsf{Lin}_{n^\star}$-commitment of Lemma 8.1. The base case is satisfied by $\mathrm{T}[\mathsf{FC}^{(1)}, \mathsf{FC}^\star]$, which we analyze below.

- The commitment phase has the following efficiency measures.
  - *Rounds.* The protocol has 2 rounds.
  - *Non-oracle commitment length.* The committer sends 1 field elements.
  - *Commitment oracles.* The committer sends 2 point oracles of $\ell^{(1)}_{\mathsf{row}} \cdot \ell^{(1)}_{\mathsf{col}} + \ell^{(1)}_{\mathsf{row}}$ field elements in total.
  - *Committer time.* The committer performs $\mathsf{enc}_{\mathcal{C}^{(1)}} + \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}}$ field operations, $3 \cdot n^{(1)} + k^{(1)}_{\mathsf{col}}$ field multiplications, and $O(n^{(1)})$ basic field operations.
- For d commitments and q evaluations claims, the opening phase has the following efficiency measures.
  - *Rounds.* The protocol has $\log k^{(1)}_{\mathsf{col}} + \log n^\star + 6$ rounds.
  - *Non-oracle proof length.* The prover sends $2 \cdot \log k^{(1)}_{\mathsf{col}} + 2 \cdot \binom{d}{2} + 2 \cdot n^\star + 2 \cdot \log n^\star + 4 \cdot t^{(1)} + 3$ field elements.
  - *Proof oracles.* The prover sends one point oracle of $2 \cdot \binom{d}{2} \cdot \ell^{(1)}_{\mathsf{row}}$ field elements.
  - *Query complexity.* The verifier issues $2 \cdot d \cdot t^{(1)} + 2 \cdot \binom{d}{2} \cdot t^{(1)}$ point queries in total.
  - *Prover time.* The prover performs $(2 \cdot \binom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}}$ field operations,

    $$2 \cdot \tbinom{d}{2} \cdot n^{(1)} + (d-1) \cdot (2 \cdot n^{(1)} + k^{(1)}_{\mathsf{col}}) + (q+4) \cdot n^{(1)} + (4 \cdot t^{(1)} + 8) \cdot n^\star$$

    field multiplications, and $O((d^2 + q) \cdot n^{(1)} + t^{(1)} \cdot n^\star)$ basic field operations.
  - *Verifier time.* $O(\log k^{(1)}_{\mathsf{col}} + d^2 + q + t^{(1)} + n^\star)$ field operations and:
    - For each of the q evaluation claims, 1 query to its multilinear extension.
    - $O(t^{(1)} \cdot \log s^{(1)} + \log k^{(1)}_{\mathsf{col}})$ field operations.
    - $t^{(1)}$ queries to the right-extension of $\mathcal{C}^{(1)}_{\mathsf{row}}$.
    - $t^{(1)}$ queries to the right-extension of $\mathcal{C}^{(1)}_{\mathsf{col}}$.

Inductive step. Suppose $M > 1$. By assumption, we have $n^{(2)} \geq \max\{k^{(1)}_{\mathsf{row}}, s^{(1)} \cdot k^{(1)}_{\mathsf{col}}\}$. Without loss of generality, we may assume that the prover of $\mathsf{FC}^{(1)}$, by padding with zeros, sends linear oracles of $n^{(2)}$ field elements (the other efficiency measures are unaffected). By the inductive hypothesis, there exists an interactive $\mathsf{Lin}_{n^{(2)}}$-commitment $\mathsf{FC}^{(2)}$ using codes $\mathcal{C}^{(2)}_{\mathsf{row}}$ and $\mathcal{C}^{(2)}_{\mathsf{col}}$. We use the following notation for the complexity measures of $\mathsf{FC}^{(2)}$.

- *Commitment phase.* Let k_c, o_c, m_c, l_c, ct denote the round complexity, number of commitment (point) oracles, non-oracle commitment length, total oracle length, and committer time.

- *Opening phase.* For d commitments and q evaluation claims, let k_o, o_o, $\mathsf{m}_o(d)$, $\mathsf{l}_o(d)$, $\mathsf{q}(d)$, $\mathsf{pt}(d,q)$, $\mathsf{vt}(d,q)$ denote the round complexity, number of proof (point) oracles, non-oracle proof length, total oracle length, total number of (point) queries, prover time, and verifier time.

The inductive step is satisfied by $\mathrm{T}[\mathsf{FC}^{(1)}, \mathsf{FC}^{(2)}]$, which we analyze below.

- The commitment phase has the following efficiency measures.
  - *Rounds.* The protocol has 2 rounds.
  - *Non-oracle commitment length.* The committer sends 1 field element.
  - *Commitment oracles.* The committer sends 2 point oracles of $\ell^{(1)}_{\mathsf{row}} \cdot \ell^{(1)}_{\mathsf{col}} + \ell^{(1)}_{\mathsf{row}}$ field elements in total.
  - *Committer time.* The committer performs $\mathsf{enc}_{\mathcal{C}^{(1)}} + \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}}$ field operations, $3 \cdot n^{(1)}$ field multiplications, and $O(n^{(1)})$ basic field operations.
- For d commitments and q evaluation claims, the opening phase has the following efficiency measures.
  - *Rounds.* The protocol has $\log k^{(1)}_{\mathsf{col}} + 2 + 2 \cdot \mathsf{k}_c + \mathsf{k}_o$ rounds.
  - *Non-oracle proof length.* The prover sends $2 \cdot \log k^{(1)}_{\mathsf{col}} + 2 \cdot \binom{d}{2} + 4 \cdot t^{(1)} + 1 + 2 \cdot \mathsf{m}_c + \mathsf{m}_o(2)$ field elements.
  - *Proof oracles.* The prover sends $1 + 2 \cdot \mathsf{o}_c + \mathsf{o}_o$ point oracles of $2 \cdot \binom{d}{2} \cdot \ell^{(1)}_{\mathsf{row}} + 2 \cdot \mathsf{l}_c + \mathsf{l}_o(2)$ field elements in total.
  - *Query complexity.* The verifier issues $2 \cdot d \cdot t^{(1)} + 2 \cdot \binom{d}{2} \cdot t^{(1)} + \mathsf{q}(2)$ point queries in total.
  - *Prover time.* The prover performs

$$(2 \cdot \tbinom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}} + 2 \cdot \mathsf{ct} + \mathsf{pt}(2, 4 \cdot t^{(1)} + 1)$$

    field operations,

$$2 \cdot \tbinom{d}{2} \cdot n^{(1)} + (d-1) \cdot (2 \cdot n^{(1)} + k^{(1)}_{\mathsf{col}}) + (q+4) \cdot n^{(1)}$$

    field multiplications, and $O((d^2 + q) \cdot n^{(1)})$ basic field operations.
  - *Verifier time.* $O(\log k^{(1)}_{\mathsf{col}} + d^2 + q + t^{(1)}) + \mathsf{vt}(2, 4 \cdot t^{(1)} + 1)$ field operations.

Solving the recurrences yields the claimed complexities. □

8.2 Multilinear polynomial commitments with point oracles

We apply Corollary A.5 to Lemma 6.1 and Lemma 8.2 to obtain a (query-optimized) multilinear polynomial commitment in the point query model.

Lemma 8.3. *Let $M \in \mathbb{N}$ be a number of iterations; for simplicity, we assume $M \geq 2$. For $i \in [M]$, fix the following parameters:*

- *Linear codes $\mathcal{C}^{(i)}_{\mathsf{row}} \colon \mathbb{F}^{k^{(i)}_{\mathsf{row}}} \to \mathbb{F}^{\ell^{(i)}_{\mathsf{row}}}$ and $\mathcal{C}^{(i)}_{\mathsf{col}} \colon \mathbb{F}^{k^{(i)}_{\mathsf{col}}} \to \mathbb{F}^{\ell^{(i)}_{\mathsf{col}}}$, where $k^{(i)}_{\mathsf{row}}, k^{(i)}_{\mathsf{col}} \in 2^{\mathbb{N}}$. Let $n^{(i)} \coloneqq k^{(i)}_{\mathsf{row}} \cdot k^{(i)}_{\mathsf{col}}$. We write $\mathsf{enc}_{\mathcal{C}^{(i)}} \coloneqq k_{\mathsf{col}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{col}}}$ to denote the number of field operations used to encode a message under the tensor code $\mathcal{C}^{(i)} \coloneqq \mathcal{C}_{\mathsf{col}} \otimes \mathcal{C}_{\mathsf{row}}$.*
- *Repetition parameters $t^{(i)} \in \mathbb{N}$ and $s^{(i)} \in 2^{\mathbb{N}}$.*

For $i \in [2, M]$, assume that $n^{(i)} \geq \max\{k_{\mathsf{row}}^{(i-1)}, s^{(i-1)} \cdot k_{\mathsf{col}}^{(i-1)}\}$. Let $n^\star := \max\{k_{\mathsf{row}}^{(M)}, t^{(M)} \cdot k_{\mathsf{col}}^{(M)}\}$. There exists an interactive $\mathsf{MLP}_{\log n^{(1)}}$-commitment such that:

- *The commitment phase has the following efficiency measures.*
  - Rounds. *The protocol has 3 rounds.*
  - Non-oracle commitment length. *The committer sends 2 field elements.*
  - Commitment oracles. *The committer sends 3 point oracles of $\ell_{\mathsf{row}}^{(1)} \cdot \ell_{\mathsf{col}}^{(1)} + \ell_{\mathsf{row}}^{(2)} \cdot \ell_{\mathsf{col}}^{(2)} + \ell_{\mathsf{row}}^{(2)}$ field elements in total.*
  - Committer time. *The committer performs $\mathsf{enc}_{\mathcal{C}^{(1)}} + \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(1)}} + \mathsf{enc}_{\mathcal{C}^{(2)}} + \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(2)}}$ field operations, $3 \cdot n^{(1)} + 3 \cdot n^{(2)}$ field multiplications, and $O(n^{(1)} + n^{(2)})$ basic field operations.*
- *For d commitments and q evaluation claims, assume that $n^{(2)} \geq 2 \cdot \binom{d}{2} \cdot k_{\mathsf{row}}^{(1)}$. The opening phase has the following efficiency measures.*
  - Round complexity. *The protocol has $(\sum_{i\in[M]} \log k_{\mathsf{col}}) + \log n^\star + 6 \cdot M + 1$ rounds.*
  - Non-oracle proof length. *The prover sends $2 \cdot \binom{d}{2} + 2 \cdot \binom{d+3}{2} + (4 + d + 2 \cdot \binom{d}{2}) \cdot t^{(1)} + (\sum_{i\in[M]} 2 \cdot \log k_{\mathsf{col}}^{(i)}) + 2 \cdot \log n^\star + 2 \cdot n^\star + (\sum_{i\in[2,M]} 4 \cdot t^{(i)}) + 5 \cdot M - 3$ field elements.*
  - Proof oracles. *The prover sends $5 \cdot M - 3$ point oracles of $3 \cdot \ell_{\mathsf{row}}^{(2)} \cdot \ell_{\mathsf{col}}^{(2)} + (2 \cdot \binom{d+3}{2} + 3) \cdot \ell_{\mathsf{row}}^{(2)} + \sum_{i\in[3,M]} 2 \cdot \ell_{\mathsf{row}}^{(i)} \cdot \ell_{\mathsf{col}}^{(i)} + 4 \cdot \ell_{\mathsf{row}}^{(i)}$ field elements in total.*
  - Query complexity. *The verifier issues $d \cdot t^{(1)} + (d+3) \cdot (d+4) \cdot t^{(2)} + \sum_{i\in[3,M]} 6 \cdot t^{(i)}$ point queries in total.*
  - Prover time. *The prover performs*

$$\begin{aligned}&(2 \cdot \tbinom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(1)}} + 3 \cdot \mathsf{enc}_{\mathcal{C}^{(2)}} + (2 \cdot \tbinom{d+3}{2} + 4) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(2)}} \\ &\quad + \sum_{i\in[3,M]} 2 \cdot \mathsf{enc}_{\mathcal{C}^{(i)}} + 2 \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(i)}} + 6 \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(i)}}\end{aligned}$$

    field operations,

$$\begin{aligned}&2 \cdot \tbinom{d}{2} \cdot n^{(1)} + (d-1) \cdot (2 \cdot n^{(1)} + k_{\mathsf{col}}^{(1)}) + (2 \cdot q + 4) \cdot n^{(1)} \\ &\quad + 2 \cdot \tbinom{d+3}{2} \cdot n^{(2)} + (d+2) \cdot (2 \cdot n^{(2)} + k_{\mathsf{col}}^{(2)}) + \big((4 + d + \tbinom{d+3}{2}) \cdot t^{(1)} + 5\big) \cdot n^{(2)} \\ &\quad + \left(\sum_{i\in[3,M]} (4 \cdot t^{(i-1)} + 9) \cdot n^{(i)} + k_{\mathsf{col}}^{(i)} \right) + (4 \cdot t^{(M)} + 8) \cdot n^\star\end{aligned}$$

    field multiplications, and $O((d^2 + q) \cdot n^{(1)} + d^2 \cdot t^{(1)} \cdot n^{(2)} + (\sum_{i\in[3,M]} t^{(i-1)} \cdot n^{(i)}) + t^{(M)} \cdot n^\star)$ basic field operations.
  - Verifier time. *The verifier performs $O(d^2 \cdot t^{(1)} + q \cdot \log n + (\sum_{i\in[M]} \log k_{\mathsf{col}}^{(i)} + t^{(i)} \cdot \log s^{(i)}) + n^\star)$ field operations and:*
    - *For each $i \in [M]$, $t^{(i)}$ queries to the right-extensions of $\mathcal{C}_{\mathsf{row}}^{(i)}$.*
    - *For each $i \in [M]$, $t^{(i)}$ queries to the right-extensions of $\mathcal{C}_{\mathsf{col}}^{(i)}$.*

The round-by-round errors are as in Lemma 8.2; the extractor performs an additional $\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}^{(1)}}$ field operations.

Proof. Let T be the transformation described in Corollary A.5. By Lemma 6.1, there exists an interactive $\mathsf{MLP}_{n^{(1)}}$-commitment $\mathsf{FC}^{(1)}$ using codes $\mathcal{C}^{(1)}_{\mathsf{row}}$ and $\mathcal{C}^{(1)}_{\mathsf{col}}$. Recall that the committer sends 1 linear oracle of length $k^{(1)}_{\mathsf{row}}$, and the prover sends 3 linear oracles of $2 \cdot \binom{d}{2} \cdot k^{(1)}_{\mathsf{row}}$, $k^{(1)}_{\mathsf{row}}$, $s^{(1)} \cdot k^{(1)}_{\mathsf{col}}$ field elements.

By assumption, we have $n^{(2)} \geq \max\{2 \cdot \binom{d}{2} \cdot k^{(1)}_{\mathsf{row}}, k^{(1)}_{\mathsf{row}}, s^{(1)} \cdot k^{(1)}_{\mathsf{col}}\}$. Without loss of generality, we may assume that the committer and prover of $\mathsf{FC}^{(2)}$, by padding with zeros, send linear oracles of $n^{(2)}$ field elements (the other efficiency measures are unaffected). By Lemma 8.2, there exists an interactive $\mathsf{Lin}_{n^{(2)}}$-commitment $\mathsf{FC}^{(2)}$ using codes $\mathcal{C}^{(2)}_{\mathsf{row}}$ and $\mathcal{C}^{(2)}_{\mathsf{col}}$. We use the following notation for the complexity measures of $\mathsf{FC}^{(2)}$.

- *Commitment phase.* Let k_c, o_c, m_c, l_c, ct denote the round complexity, number of commitment (point) oracles, non-oracle commitment length, total oracle length, and committer time.
- *Opening phase.* For d commitments and q evaluation claims, let k_o, o_o, $\mathsf{m}_o(d)$, $\mathsf{l}_o(d)$, $\mathsf{q}(d)$, $\mathsf{pt}(d,q)$, $\mathsf{vt}(d,q)$ denote the round complexity, number of proof (point) oracles, non-oracle proof length, total oracle length, total number of (point) queries, prover time, and verifier time.

The lemma is satisfied by $\mathrm{T}[\mathsf{FC}^{(1)}, \mathsf{FC}^{(2)}]$, which we analyze below.

- The commitment phase has the following efficiency measures.
  - *Rounds.* The protocol has $1 + \mathsf{k}_c$ rounds.
  - *Non-oracle commitment length.* The committer sends $1 + \mathsf{m}_c$ field element.
  - *Commitment oracles.* The committer sends $1 + \mathsf{o}_c$ point oracles of $\ell^{(1)}_{\mathsf{row}} \cdot \ell^{(1)}_{\mathsf{col}} + \mathsf{l}_c$ field elements in total.
  - *Committer time.* The committer performs $\mathsf{enc}_{\mathcal{C}^{(1)}} + \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}} + \mathsf{ct}$ field operations, $3 \cdot n^{(1)}$ field multiplications, and $O(n^{(1)})$ basic field operations.
- For d commitments and q evaluation claims, the opening phase has the following efficiency measures.
  - *Rounds.* The protocol has $\log k^{(1)}_{\mathsf{col}} + 1 + 3 \cdot \mathsf{k}_c + \mathsf{k}_o$ rounds.
  - *Non-oracle proof length.* The prover sends $2 \cdot \log k^{(1)}_{\mathsf{col}} + 2 \cdot \binom{d}{2} + (4 + d + 2 \cdot \binom{d}{2}) \cdot t^{(1)} + 1 + 3 \cdot \mathsf{m}_c + \mathsf{m}_o(d+3)$ field elements.
  - *Proof oracles.* The prover sends $3 \cdot \mathsf{o}_c + \mathsf{o}_o$ point oracles of $3 \cdot \mathsf{l}_c + \mathsf{l}_o(d+3)$ field elements in total.
  - *Query complexity.* The verifier issues $d \cdot t^{(1)} + \mathsf{q}(4)$ point queries in total.
  - *Prover time.* The prover performs

    $$(2 \cdot \tbinom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}^{(1)}_{\mathsf{row}}} + 3 \cdot \mathsf{ct} + \mathsf{pt}\big(d+3, (4 + d + 2 \cdot \tbinom{d}{2}) \cdot t^{(1)} + 1\big)$$

    field operations,

    $$2 \cdot \tbinom{d}{2} \cdot n^{(1)} + (d-1) \cdot (2 \cdot n^{(1)} + k^{(1)}_{\mathsf{col}}) + (2 \cdot q + 4) \cdot n^{(1)}$$

    field multiplications, and $O((d^2 + q) \cdot n^{(1)})$ basic field operations.
  - *Verifier time.* The verifier performs $O(\log k^{(1)}_{\mathsf{col}} + d^2 + q + t^{(1)}) + \mathsf{vt}(d+3, (4+d+2 \cdot \binom{d}{2}) \cdot t^{(1)} + 1)$ field operations.

□

8.3 Main theorem

We parameterize Lemma 8.3 and invoke the delegation protocol of Corollary 7.10 to achieve our main theorem.

Corollary 8.4. *Fix security parameter $\lambda \in \mathbb{N}$. Fix linear codes $\mathcal{C}_{\mathsf{row}} \colon \mathbb{F}^{k_{\mathsf{row}}} \to \mathbb{F}^{\ell_{\mathsf{row}}}$ and $\mathcal{C}_{\mathsf{col}} \colon \mathbb{F}^{k_{\mathsf{col}}} \to \mathbb{F}^{\ell_{\mathsf{col}}}$, where $k_{\mathsf{row}}, k_{\mathsf{col}} \in 2^{\mathbb{N}}$. Let $n \coloneqq k_{\mathsf{row}} \cdot k_{\mathsf{col}}$. Moreover:*

- *Assume that $k_{\mathsf{row}} = O(\sqrt{\lambda \cdot n})$ and $k_{\mathsf{col}} = O(\sqrt{n/\lambda})$.*
- *Let ρ_{row} and ρ_{col} denote the rates of $\mathcal{C}_{\mathsf{row}}$ and $\mathcal{C}_{\mathsf{col}}$, respectively. Let $\rho \coloneqq \sqrt{\rho_{\mathsf{row}} \cdot \rho_{\mathsf{col}}}$ denote their (geometric) average.*
- *Fix proximity bounds $\delta_{\mathsf{row}} \in (0, \delta(\mathcal{C}_{\mathsf{row}}))$ and $\delta_{\mathsf{col}} \in (0, \delta(\mathcal{C}_{\mathsf{col}}))$. Let $\delta \coloneqq \sqrt{\delta_{\mathsf{row}} \cdot \delta_{\mathsf{col}}}$ denote their (geometric) average.*
- *Let $\mathsf{enc}_{\mathcal{C}} \coloneqq k_{\mathsf{col}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{col}}}$ denote the number of field operations used to encode a message under the tensor code $\mathcal{C} \coloneqq \mathcal{C}_{\mathsf{col}} \otimes \mathcal{C}_{\mathsf{row}}$.*

There exists an interactive $\mathsf{MLP}_{\log n}$-commitment such that:

- *The commitment phase has the following efficiency measures.*
  - Rounds. *The protocol has 3 rounds.*
  - Non-oracle commitment length. *The committer sends 2 field elements.*
  - Commitment oracles. *The committer sends 3 point oracles of $n/\rho^2 + (\lambda \cdot n)^{0.5+o(1)}$ field elements in total.*
  - Committer time. *The committer performs $\mathsf{enc}_{\mathcal{C}} + \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ field operations, $3 \cdot n + (\lambda \cdot n)^{0.5+o(1)}$ field multiplications, and $O(n)$ basic field operations.*
- *For $d = O(1)$ commitments and $q = O(1)$ evaluation claims, the opening phase has the following efficiency measures.*
  - Round complexity. *The protocol has $O(\log n)$ rounds.*
  - Non-oracle proof length. *the prover sends $O(\lambda + \log n)$ field elements.*
  - Proof oracles. *The prover sends $O(\log \log n)$ point oracles of $(\lambda \cdot n)^{0.5+o(1)}$ field elements in total.*
  - Query complexity. *The verifier issues $\left(\frac{1}{-\log(1-\delta^2)} + o(1)\right) \cdot \lambda$ point queries in total.*
  - Prover time. *The prover performs $(2 \cdot \binom{d}{2} + 1) \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}}$ field operations,*

    $$((d-1) \cdot (d+2) + 2 \cdot q + 4) \cdot n + (\lambda \cdot n)^{0.5+o(1)}$$

    field multiplications, and $O(n)$ basic field operations.
  - Verifier time. *The verifier performs $O(\lambda \cdot \log n)$ field operations and:*
    - *$O(\lambda)$ queries to the right-extensions of $\mathcal{C}_{\mathsf{row}}$.*
    - *$O(\lambda)$ queries to the right-extensions of $\mathcal{C}_{\mathsf{col}}$.*

Assuming the field is sufficiently large, the commitment phase has round-by-round binding error at most $2^{-\lambda}$ and the opening phase has round-by-round knowledge soundness error at most $2^{-\lambda}$; the extractor performs

$$2 \cdot \mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}} \cdot \mathsf{errdec}_{\mathcal{C}_{\mathsf{col}}}(\delta_{\mathsf{col}}) + |\Lambda(\mathcal{C}_{\mathsf{col}}, \delta_{\mathsf{col}})| \cdot O(n/\rho^2) + O(n^3/\rho^6)$$

field operations.

Proof. We instantiate Lemma 8.2 with the following parameters. Set the number of iterations $M = O(\log \log n)$ and the gap parameter $\varepsilon = \Theta(1/\sqrt{\log n})$. Choose $\mathcal{C}_{\mathsf{row}}^{(1)} := \mathcal{C}_{\mathsf{row}}$ and $\mathcal{C}_{\mathsf{col}}^{(1)} := \mathcal{C}_{\mathsf{col}}$. Set the repetition parameters

$$t^{(1)} := \frac{\lambda}{-\log(1-(\delta_{\mathsf{row}}-\varepsilon)\cdot\delta_{\mathsf{col}})} = \frac{\lambda}{-\log(1-\delta_{\mathsf{row}}\cdot\delta_{\mathsf{col}})} + o(\lambda)\,, \quad s^{(1)} := O(\lambda\cdot\log n)\,.$$

For each $i \in [2, M]$, choose $\mathcal{C}_{\mathsf{row}}^{(i)}$ and $\mathcal{C}_{\mathsf{col}}^{(i)}$ to be Reed–Solomon codes of rate $\frac{1}{\log^{\omega(1)}(n)}$ such that

$$n^{(i)} = O(s^{(i-1)}\cdot k_{\mathsf{col}}^{(i-1)})\,, \quad k_{\mathsf{row}}^{(i)} = O\big(\sqrt{\lambda\cdot n^{(i)}}\big)\,, \quad k_{\mathsf{col}}^{(i)} = O\big(\sqrt{n^{(i)}/\lambda}\big)\,.$$

Fix the proximity bounds $\delta_{\mathsf{row}}^{(i)} = 1 - \frac{1}{\log^{\omega(1)}(n)}$ and $\delta_{\mathsf{col}}^{(i)} = 1 - \frac{1}{\log^{\omega(1)}(n)}$ to be arbitrarily close to the Johnson bound. Set the repetition parameters

$$t^{(i)} := \frac{\lambda}{-\log(1-\delta_{\mathsf{row}}\cdot\delta_{\mathsf{col}})} = o(\lambda/\log\log n)\,, \quad s^{(i)} := t^{(i)}\,,$$

so that soundness follows from Remark 5.2. This enables us to set $n^\star = O(\lambda)$. □

Theorem 8.5. *Fix security parameter $\lambda \in \mathbb{N}$. Fix linear codes $\mathcal{C}_{\mathsf{row}} \colon \mathbb{F}^{k_{\mathsf{row}}} \to \mathbb{F}^{\ell_{\mathsf{row}}}$ and $\mathcal{C}_{\mathsf{col}} \colon \mathbb{F}^{k_{\mathsf{col}}} \to \mathbb{F}^{\ell_{\mathsf{col}}}$, where $k_{\mathsf{row}}, k_{\mathsf{col}} \in 2^{\mathbb{N}}$. Let $n := k_{\mathsf{row}} \cdot k_{\mathsf{col}}$. Moreover:*

- *Assume that $k_{\mathsf{col}} \le k_{\mathsf{row}}$ and the skewness $\alpha := \sqrt{n}/k_{\mathsf{col}}$ is at most $O(\sqrt{\lambda})$.*
- *Let ρ_{row} and ρ_{col} denote the rates of $\mathcal{C}_{\mathsf{row}}$ and $\mathcal{C}_{\mathsf{col}}$, respectively. Let $\rho := \sqrt{\rho_{\mathsf{row}}\cdot\rho_{\mathsf{col}}}$ denote their (geometric) average.*
- *Fix proximity bounds $\delta_{\mathsf{row}} \in (0, \delta(\mathcal{C}_{\mathsf{row}}))$ and $\delta_{\mathsf{col}} \in (0, \delta(\mathcal{C}_{\mathsf{col}}))$. Let $\delta := \sqrt{\delta_{\mathsf{row}}\cdot\delta_{\mathsf{col}}}$ denote their (geometric) average.*
- *Let $\mathsf{enc}_{\mathcal{C}} := k_{\mathsf{col}}\cdot\mathsf{enc}_{\mathcal{C}_{\mathsf{row}}} + \ell_{\mathsf{row}}\cdot\mathsf{enc}_{\mathcal{C}_{\mathsf{col}}}$ denote the number of field operations used to encode a message under the tensor code $\mathcal{C} := \mathcal{C}_{\mathsf{col}} \otimes \mathcal{C}_{\mathsf{row}}$. Assume that $\mathcal{C}_{\mathsf{row}}$ and $\mathcal{C}_{\mathsf{col}}$ are linear-time encodable.*

There exists an interactive $\mathsf{MLP}_{\log n}$-commitment such that:

- *The commitment phase has the following efficiency measures.*
  - Rounds. *The protocol has 3 rounds.*
  - Non-oracle commitment length. *The committer sends 2 field elements.*
  - Commitment oracles. *The committer sends 3 point oracles of $n/\rho^2 + (\lambda\cdot n)^{0.5+o(1)}$ field elements in total.*
  - Committer time. *The committer performs $\mathsf{enc}_{\mathcal{C}}$ field operations, $3\cdot n + (\lambda\cdot n)^{0.5+o(1)}$ field multiplications, and $O(n)$ basic field operations.*
- *For $d = O(1)$ commitments and $q = O(1)$ evaluation claims, the (holographic) opening phase has the following efficiency measures.*
  - Index oracles. *The indexer outputs $O(1)$ oracles of $(\lambda\cdot n)^{0.5+o(1)}$ field elements in total.*
  - Indexer time. *The indexer performs $(\lambda\cdot n)^{0.5+o(1)}$ field operations.*
  - Round complexity. *The protocol has $O(\log n)$ rounds.*
  - Non-oracle proof length. *the prover sends $O(\lambda + \log n)$ field elements.*

– Proof oracles. *The prover sends* $O(\log \log n)$ *point oracles of* $(\lambda \cdot n)^{0.5+o(1)}$ *field elements in total.*
– Query complexity. *The verifier issues* $\left(\frac{1}{-\log(1-\delta^2)} + o(1)\right) \cdot \lambda$ *point queries in total.*
– Prover time. *The prover performs*

$$((d-1) \cdot (d+2) + 2 \cdot q + 4) \cdot n + (\lambda \cdot n)^{0.5+o(1)}$$

field multiplications and $O(n)$ *basic field operations.*
– Verifier time. *The verifier performs* $O(\lambda \cdot \log n)$ *field operations.*

The round-by-round errors and extractor time are as in Corollary 8.4.

Proof. We instantiate Corollary 8.4 with low-rate Reed–Solomon codes to obtain a commitment scheme (for polynomials of size $(\lambda \cdot n)^{0.5+o(1)}$) with a succinct verifier. Applying Corollary 7.10, we obtain a delegation protocol for right-extensions of $\mathcal{C}_{\mathsf{row}}$ and $\mathcal{C}_{\mathsf{col}}$. Finally, we augment Corollary 8.4 with this delegation protocol to make the verifier succinct (the other efficiency measures are unaffected). □

Acknowledgements

Benedikt Bünz and William Wang are partially supported by Alpen Labs, Chaincode Labs, and Google. Giacomo Fenzi is partially supported by the Ethereum Foundation and the Sui Foundation. Part of this work was conducted while Giacomo Fenzi and William Wang were at Succinct and visiting the Simons Institute for the Theory of Computing.

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "STIR: Reed–Solomon Proximity Testing with Fewer Queries". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 380–413.

[ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification". In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '25. 2025.

[ACY23] Gal Arnon, Alessandro Chiesa, and Eylon Yogev. "IOPs with Inverse Polynomial Soundness Error". In: *64th IEEE Annual Symposium on Foundations of Computer Science , FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*. IEEE, 2023, pp. 752–761.

[AFK23] Thomas Attema, Serge Fehr, and Michael Klooß. "Fiat-Shamir Transformation of Multi-Round Interactive Proofs (Extended Version)". In: *J. Cryptol.* 36.4 (2023), p. 36. DOI: 10.1007/S00145-023-09478-Y. URL: https://doi.org/10.1007/s00145-023-09478-y.

[AHIV23] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. "Ligero: lightweight sublinear arguments without a trusted setup". In: *Des. Codes Cryptogr.* 91.11 (2023), pp. 3379–3424. DOI: 10.1007/S10623-023-01222-8. URL: https://doi.org/10.1007/s10623-023-01222-8.

[ARZR25] Noor Athamnah, Noga Ron-Zewi, and Ron D. Rothblum. *Linear Prover IOPs in Log Star Rounds*. Cryptology ePrint Archive, Paper 2025/1269. 2025.

[BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. "Fast Reed–Solomon Interactive Oracle Proofs of Proximity". In: *Proceedings of the 45th International Colloquium on Automata, Languages and Programming*. ICALP '18. 2018, 14:1–14:17.

[BCFRRZ25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. "Blaze: Fast SNARKs from Interleaved RAA Codes". In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '25. 2025.

[BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. *Linear-Time Accumulation Schemes*. Cryptology ePrint Archive, Paper 2025/753. 2025.

[BCGRS17] Eli Ben-Sasson, Alessandro Chiesa, Ariel Gabizon, Michael Riabzev, and Nicholas Spooner. "Interactive Oracle Proofs with Constant Rate and Query Complexity". In: *Proceedings of the 44th International Colloquium on Automata, Languages and Programming*. ICALP '17. 2017, 40:1–40:15.

[BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. "Proximity Gaps for Reed–Solomon Codes". In: *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*. FOCS '20. 2020, pp. 900–909.

[BCIOP13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. "Succinct Non-interactive Arguments via Linear Interactive Proofs". In: *Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3-6, 2013. Proceedings*. Ed. by Amit Sahai. Vol. 7785. Lecture Notes in Computer Science. Springer, 2013, pp. 315–333. DOI: 10.1007/978-3-642-36594-2_18. URL: https://doi.org/10.1007/978-3-642-36594-2_18.

[BCLMS21] Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. "Proof-Carrying Data Without Succinct Arguments". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 681–710.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. "Interactive Oracle Proofs". In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC '16-B. 2016, pp. 31–60.

[BCS97] Peter Bürgisser, Michael Clausen, and Mohammad Amin Shokrollahi. *Algebraic Complexity Theory*. Red. by S. S. Chern et al. Vol. 315. Grundlehren der mathematischen Wissenschaften. Berlin, Heidelberg: Springer, 1997. ISBN: 978-3-642-08228-3 978-3-662-03338-8. DOI: 10.1007/978-3-662-03338-8.

[BFHVXZ20] Rishabh Bhadauria, Zhiyong Fang, Carmit Hazay, Muthuramakrishnan Venkitasubramaniam, Tiancheng Xie, and Yupeng Zhang. "Ligero++: A New Optimized Sublinear IOP". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna. ACM, 2020, pp. 2025–2038. DOI: 10.1145/3372297.3417893. URL: https://doi.org/10.1145/3372297.3417893.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. "Transparent SNARKs from DARK Compilers". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020, pp. 677–706.

[BGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. "DEEP-FRI: Sampling Outside the Box Improves Soundness". In: *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*. ITCS '20. 2020, 5:1–5:32.

[BKS18] Eli Ben-Sasson, Swastik Kopparty, and Shubhangi Saraf. "Worst-Case to Average Case Reductions for the Distance to a Code". In: *Proceedings of the 33rd ACM Conference on Computer and Communications Security*. CCS '18. 2018, 24:1–24:23.

[BMMS25a] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. *FICS and FACS: Fast IOPPs and Accumulation via Code-Switching*. Cryptology ePrint Archive, Report 2025/737. 2025.

[BMMS25b] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. "Query-Optimal IOPPs for Linear-Time Encodable Codes". In: *IACR Cryptol. ePrint Arch.* (2025), p. 1588. URL: https://eprint.iacr.org/2025/1588.

[BMNW25] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. "Arc: Accumulation for Reed–Solomon Codes". In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO '25. 2025.

[Ber] Daniel J. Bernstein. *The transposition principle*. URL: https://cr.yp.to/transposition.html.

[Bor57] J. L. Bordewijk. "Inter-reciprocity applied to electrical networks". In: *Applied Scientific Research, Section B* 6.1 (Dec. 1, 1957), pp. 1–74. ISSN: 0365-7140. DOI: 10.1007/BF02920362. URL: https://doi.org/10.1007/BF02920362.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '23. 2023, pp. 499–530.

[CCHLRR18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. *Fiat–Shamir From Simpler Assumptions*. Cryptology ePrint Archive, Report 2018/1004. 2018.

[CDGS23] Alessandro Chiesa, Marcel Dall'Agnol, Ziyi Guan, and Nicholas Spooner. *On the Security of Succinct Interactive Arguments from Vector Commitments*. Cryptology ePrint Archive, Paper 2023/1737. 2023. URL: https://eprint.iacr.org/2023/1737.

[CHMMVW20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020.

[CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. "Succinct Arguments in the Quantum Random Oracle Model". In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC '19. 2019, pp. 1–29.

[CT12] Alessandro Chiesa and Eran Tromer. "Proof-Carrying Data: Secure Computation on Untrusted Platforms (high-level description)". In: *The Next Wave: The National Security Agency's review of emerging technologies* 19.2 (2012), pp. 40–46.

[CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: https://github.com/hash-based-snargs-book.

[DG25] Benjamin E. Diamond and Angus Gruen. *On the Distribution of the Distances of Random Words*. Cryptology ePrint Archive, Paper 2025/2010. 2025. URL: https://eprint.iacr.org/2025/2010.

[DP24a] Benjamin E. Diamond and Jim Posen. "Proximity Testing with Logarithmic Randomness". In: *IACR Communications in Cryptology* 1.1 (2024).

[DP24b] Benjamin Diamond and Jim Posen. "Polylogarithmic Proofs for Multilinears over Binary Towers". In: *IACR Cryptol. ePrint Arch.* (2024), p. 504.

[DP25] Benjamin Diamond and Jim Posen. "Succinct Arguments over Towers of Binary Fields". In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '25. 2025.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. "Quadratic Span Programs and Succinct NIZKs without PCPs". In: *Proceedings of the 32nd Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '13. 2013, pp. 626–645.

[GGR11] Parikshit Gopalan, Venkatesan Guruswami, and Prasad Raghavendra. "List Decoding Tensor Products and Interleaved Codes". In: *SIAM Journal on Computing* 40.5 (2011). Preliminary version appeared in STOC '09., pp. 1432–1462.

[GKL24] Yiwen Gao, Haibin Kan, and Yuan Li. *Linear Proximity Gap for Linear Codes within the 1.5 Johnson Bound*. Cryptology ePrint Archive, Report 2024/1810. 2024.

[GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. "Brakedown: Linear-time and field-agnostic SNARKs for R1CS". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023.

[GS99] Venkatesan Guruswami and Madhu Sudan. "Improved decoding of Reed-Solomon and algebraic-geometry codes". In: *IEEE Trans. Inf. Theory* 45.6 (1999), pp. 1757–1767. DOI: 10.1109/18.782097. URL: https://doi.org/10.1109/18.782097.

[GW11] Craig Gentry and Daniel Wichs. "Separating Succinct Non-Interactive Arguments From All Falsifiable Assumptions". In: *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing*. STOC '11. 2011, pp. 99–108.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Report 2019/953. 2019.

[Gol25] Sophia Gold. *Shipping an L1 zkEVM #1: Realtime Proving*. https://blog.ethereum.org/2025/07/10/realtime-proving. 2025.

[Gro16] Jens Groth. "On the Size of Pairing-Based Non-interactive Arguments". In: *Proceedings of the 35th Annual International Conference on Theory and Applications of Cryptographic Techniques*. EUROCRYPT '16. 2016, pp. 305–326.

[HJRRR25] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. *Jagged Polynomial Commitments (or: How to Stack Multilinears)*. Cryptology ePrint Archive, Paper 2025/917. 2025.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. "Circle STARKs". In: *IACR Cryptol. ePrint Arch.* (2024), p. 278. URL: https://eprint.iacr.org/2024/278.

[HS24] Thomas den Hollander and Daniel Slamanig. "A Crack in the Firmament: Restoring Soundness of the Orion Proof System and More". In: *IACR Cryptol. ePrint Arch.* (2024), p. 1164. URL: https://eprint.iacr.org/2024/1164.

[IKO07] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. "Efficient Arguments without Short PCPs". In: *22nd Annual IEEE Conference on Computational Complexity (CCC 2007), 13-16 June 2007, San Diego, California, USA*. IEEE Computer Society, 2007, pp. 278–291. DOI: 10.1109/CCC.2007.10. URL: https://doi.org/10.1109/CCC.2007.10.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. "Constant-Size Commitments to Polynomials and Their Applications". In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '10. 2010, pp. 177–194.

[Kil92] Joe Kilian. "A note on efficient zero-knowledge proofs and arguments". In: *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*. STOC '92. 1992, pp. 723–732.

[MZ25] Dor Minzer and Kai Zhe Zheng. "Improved Round-by-round Soundness IOPs via Reed-Muller Codes". In: *CoRR* abs/2504.00346 (2025). DOI: 10.48550/ARXIV.2504.00346. arXiv: 2504.00346. URL: https://doi.org/10.48550/arXiv.2504.00346.

[Mei13] Or Meir. "IP = PSPACE Using Error-Correcting Codes". In: *SIAM Journal on Computing* 42.1 (2013), pp. 380–403.

[Mic00] Silvio Micali. "Computationally Sound Proofs". In: *SIAM Journal on Computing* 30.4 (2000). Preliminary version appeared in FOCS '94., pp. 1253–1298.

[NA25] Andrija Novakovic and Guillermo Angeris. "Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme". In: *IACR Cryptol. ePrint Arch.* (2025), p. 1187. URL: https://eprint.iacr.org/2025/1187.

[RR24] Noga Ron-Zewi and Ron Rothblum. "Local Proofs Approaching the Witness Length". In: *J. ACM* 71.3 (2024), p. 18. DOI: 10.1145/3661483. URL: https://doi.org/10.1145/3661483.

[RR25] Noga Ron-Zewi and Ron Rothblum. "Proving as Fast as Computing: Succinct Arguments with Constant Prover Overhead". In: *J. ACM* 72.2 (2025), 15:1–15:54. DOI: 10.1145/3721477. URL: https://doi.org/10.1145/3721477.

[RRR21] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. "Constant-Round Interactive Proofs for Delegating Computation". In: *SIAM J. Comput.* 50.3 (2021). DOI: 10.1137/16M1096773. URL: https://doi.org/10.1137/16M1096773.

[RVW13] Guy N. Rothblum, Salil P. Vadhan, and Avi Wigderson. "Interactive proofs of proximity: delegating computation in sublinear time". In: *Proceedings of the 45th ACM Symposium on the Theory of Computing*. STOC '13. 2013, pp. 793–802.

[SW14] Amit Sahai and Brent Waters. "How to use indistinguishability obfuscation: deniable encryption, and more". In: *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*. Ed. by David B. Shmoys. ACM, 2014, pp. 475–484. DOI: 10.1145/2591796.2591825. URL: https://doi.org/10.1145/2591796.2591825.

[Set19] Srinath Setty. *Spartan: Efficient and general-purpose zkSNARKs without trusted setup*. Cryptology ePrint Archive, Report 2019/550. 2019.

[Sta21] StarkWare. *ethSTARK Documentation*. Cryptology ePrint Archive, Paper 2021/582. https://eprint.iacr.org/2021/582. 2021. URL: https://eprint.iacr.org/2021/582.

[Val08] Paul Valiant. "Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency". In: *Proceedings of the 5th Theory of Cryptography Conference*. TCC '08. 2008, pp. 1–18.

[XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Orion: Zero Knowledge Proof with Linear Prover Time". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 299–328.

[ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes". In: *Proceedings of the 44th Annual International Cryptology Conference*. Vol. 14929. CRYPTO '24. 2024, pp. 138–169.

[Zei24] Hadas Zeilberger. *Khatam: Reducing the Communication Complexity of Code-Based SNARKs*. Cryptology ePrint Archive, Report 2024/1843. 2024.

A Composition lemmas

We show how to compose protocols with interactive oracle functional commitments.

- *Proof composition.* In Section A.1 we show that, given a functional IOP that uses $\mathcal{O}$-oracles and an interactive $\mathcal{O}$-commitment that uses $\mathcal{P}$-oracles, one can construct a functional IOP that uses $\mathcal{P}$-oracles.
- *Commitment composition.* In Section A.2 we show that, given an interactive $\mathcal{O}'$-commitment that uses $\mathcal{O}$-oracles and an interactive $\mathcal{O}$-commitment that uses $\mathcal{P}$-oracles, one can construct an interactive $\mathcal{O}'$-commitment that uses $\mathcal{P}$-oracles.

A.1 Proof composition

Lemma A.1. *There exists a transformation T_p such that the following holds. Let $\mathcal{O}$ be an oracle class, and suppose we have:*

- FIOP*, a functional IOP for a relation $\mathcal{R}$. Without loss of generality, instances of $\mathcal{R}$ have the form $\mathbb{x} = (x, \mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}})$, where n is the number of instance oracles. Let $I \subseteq [\mathsf{n}]$ denote the subset of indices i where X_i is an $\mathcal{O}$-oracle. Let di and d denote the number of index and proof $\mathcal{O}$-oracles. Let q denote the number of $\mathcal{O}$-queries made by the verifier.*
- $\mathsf{FC} = (\mathsf{Commit}, \mathsf{Open})$*, an interactive $\mathcal{O}$-commitment where* Commit *has output relation $\mathcal{R}_\bullet$ and failure language $\mathcal{L}_\times$.*

Then, $\mathrm{T}_p[\mathsf{FIOP}, \mathsf{FC}]$ is a functional IOP for the relation $\mathcal{R}^\dagger$, defined as follows. (If the instance does not contain oracles, then $\mathcal{R}^\dagger$ is equivalent to $\mathcal{R}$.) Instances have the form

$$\mathbb{x}^\dagger = \left(x, (\mathsf{cm}_i)_{i \in I}, (\mathrm{X}_i)_{i \notin I}\right).$$

- $(\mathcal{R}^\dagger)^{\mathrm{YES}}$ *consists of indices $\mathbb{i}^\dagger = \mathbb{i}$, instances $\mathbb{x}^\dagger$, and witnesses $\mathbb{w}^\dagger = (\mathbb{w}, (\mathrm{X}_i, \mathsf{aux}_i)_{i \in I})$ where the following holds:*
  - *With $\mathbb{x} = (x, \mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}})$, it holds that $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}^{\mathrm{YES}}$.*
  - *For each $i \in I$, it holds that $(\mathsf{cm}_i, (\mathrm{X}_i, \mathsf{aux}_i)) \in \mathcal{R}_\bullet^{\mathrm{YES}}$.*
- $(\overline{\mathcal{R}}^\dagger)^{\mathrm{NO}}$ *consists of indices $\mathbb{i}^\dagger = \mathbb{i}$, instances $\mathbb{x}^\dagger$, and witnesses $\mathbb{w}^\dagger = (\mathbb{w}, (\mathrm{X}_i)_{i \in I})$ where at least one of the following holds:*
  - *For some $i \in I$, it holds that $\mathsf{cm}_i \in \mathcal{L}_\times$.*
  - *The following hold:*
    - *With $\mathbb{x} = (x, \mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}})$, it holds that $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \overline{\mathcal{R}}^{\mathrm{NO}}$.*
    - *For each $i \in I$, it holds that $(\mathsf{cm}_i, \mathrm{X}_i) \in \overline{\mathcal{R}}_\bullet^{\mathrm{NO}}$.*

Moreover, $\mathrm{T}_p[\mathsf{FIOP}, \mathsf{FC}]$ has the following efficiency measures.

- *The indexer time corresponds to:*
  - *An execution of the* FIOP *indexer.*
  - di *executions of the* Commit *committer.*
- *The index oracles consist of:*

  - *All but the* di $\mathcal{O}$*-oracles produced by the* FIOP *indexer.*
  - *Oracles from the* di *commitments produced by the* Commit *committer.*
- *The number of rounds, non-oracle message lengths, randomness lengths, prover time, and verifier time correspond to:*
  - *An execution of the* FIOP *interactive protocol.*
  - d *executions of the* Commit *committer.*
  - *An execution of* Open *on* $\mathsf{di} + \mathsf{d} + |I|$ *commitments and* q *evaluation claims.*
- *The proof oracles consist of:*
  - *All but the* d $\mathcal{O}$*-oracles produced by the* FIOP *prover.*
  - *Oracles from* d *commitments produced by the* Commit *committer.*
  - *Oracles produced by the* Open *prover.*

The round-by-round knowledge soundness error is the maximum of the round-by-round knowledge soundness error of FIOP *and* Open*, as well as the round-by-round binding error* Commit*. The extraction time is the sum of the extraction times of* FIOP *and* Open*.*

Proof. In Construction A.2 we describe $\mathrm{T}_p[\mathsf{FIOP}, \mathsf{FC}]$, and in Lemma A.3 we analyze its round-by-round knowledge soundness errors. □

Construction A.2. We describe $\mathrm{T}_p[\mathsf{FIOP}, \mathsf{FC}]$. Let $\mathsf{FIOP} = (\mathbf{I}, \mathbf{P}, \mathbf{V})$, $\mathsf{Commit} = (\mathbf{C}, \mathcal{R}_\bullet, \mathcal{L}_\times)$. Let k and $\mathsf{k}^\bullet$ denote the number of rounds in FIOP and Commit, respectively. Let $\mathsf{r}_1, \ldots, \mathsf{r}_\mathsf{k}$ and $\mathsf{r}^\bullet_1, \ldots, \mathsf{r}^\bullet_{\mathsf{k}^\bullet}$ denote the randomness lengths in each round of FIOP and Commit, respectively. For simplicity, we assume the following:
- The indexer $\mathbf{I}$ outputs one oracle.
- The prover $\mathbf{P}$ and committer $\mathbf{C}$ do not send non-oracle messages.
- The index, instance, and proof oracles in FIOP are all $\mathcal{O}$-oracles.

The construction and security analysis naturally extend to the general setting.

- **Indexer.** The indexer receives the index $\mathbb{i}^\dagger = \mathbb{i}$ and does the following. For each $j \in [\mathsf{k}^\bullet]$,

$$\Pi^\bullet_j \coloneqq \mathbf{C}(\mathrm{I}, \alpha^\bullet_1, \ldots \alpha^\bullet_{j-1}) \,,$$

  where $\mathrm{I} \coloneqq \mathbf{I}(\mathbb{i})$ and each $\alpha^\bullet_j = 0$ is fixed to be zero. Set $\mathsf{icm} \coloneqq (\Pi^\bullet_j, \alpha^\bullet_j)_{j \in [\mathsf{k}^\bullet]}$ and $\mathsf{iaux} \coloneqq \mathbf{C}(\mathrm{I}, \alpha^\bullet_1, \ldots \alpha^\bullet_{\mathsf{k}^\bullet})$. The indexer outputs $\Pi^\bullet_1, \ldots, \Pi^\bullet_{\mathsf{k}^\bullet}$ and the oracles output by the indexer of Open.

- **Inputs.** The prover and verifier receive the instance $\mathbb{x}^\dagger = (x, \mathsf{cm}_1, \ldots, \mathsf{cm}_\mathsf{n})$. In the honest case, the prover additionally receives the witness $\mathbb{w}^\dagger = (\mathbb{w}, (\mathrm{X}_1, \mathsf{aux}_1), \ldots, (\mathrm{X}_\mathsf{n}, \mathsf{aux}_\mathsf{n}))$.

- **FIOP interaction phase.** In the honest case, the prover sets $\mathbb{x} = (x, \mathrm{X}_1, \ldots, \mathrm{X}_\mathsf{n})$. For $i \in [\mathsf{k}]$:

  1. **FIOP oracle.** In the honest case, the prover computes

$$\Pi_i \coloneqq \mathbf{P}(\mathbb{x}, \mathbb{w}, \alpha_1, \ldots, \alpha_{i-1}) \,.$$

  2. **Commitment phase.** For $j \in [\mathsf{k}^\bullet]$:
     (a) **Commitment oracle.** The prover sends $\Pi^\bullet_{i,j}$. In the honest case,

$$\Pi^\bullet_{i,j} \coloneqq \mathbf{C}(\Pi_i, \alpha^\bullet_{i,1}, \ldots, \alpha^\bullet_{i,j-1}) \,.$$

(b) **Commitment randomness.** The verifier samples and sends $\alpha^{\bullet}_{i,j} \leftarrow \{0,1\}^{\mathsf{r}^{\bullet}_j}$.

Set $\mathsf{pcm}_i := (\Pi^{\bullet}_{i,j}, \alpha^{\bullet}_{i,j})_{j\in[\mathsf{k}^{\bullet}]}$. In the honest case, the prover computes $\mathsf{paux}_i := \mathbf{C}(\Pi_i, \alpha^{\bullet}_{i,1}, \ldots, \alpha^{\bullet}_{i,\mathsf{k}^{\bullet}})$.

3. **FIOP randomness.** The verifier samples and sends $\alpha_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.

- **FIOP answers.** The prover sends $\mathsf{ians}, \mathsf{ans}_1, \ldots, \mathsf{ans}_{\mathsf{n}}, \mathsf{pans}_1, \ldots, \mathsf{pans}_{\mathsf{k}} \subseteq \mathcal{Q} \times \Sigma$. In the honest case, these are derived from an execution of $\mathbf{V}(x, \alpha_1, \ldots, \alpha_{\mathsf{k}})$:
  - ians contains query-answers pairs of the form $(\sigma, \mathrm{I}(\sigma))$, where the verifier issues query σ to the index oracle.
  - ans_i contains query-answer pairs of the form $(\sigma, \mathrm{X}_i(\sigma))$, where the verifier issues query σ to the i-th instance oracle.
  - pans_i contains query-answer pairs of the form $(\sigma, \Pi_i(\sigma))$, where the verifier issues query σ to the i-th proof oracle.

  Set $\mathsf{Ans} := (\mathsf{ians}, \mathsf{ans}_1, \ldots, \mathsf{ans}_{\mathsf{n}}, \mathsf{pans}_1, \ldots, \mathsf{pans}_{\mathsf{k}})$.

- **Opening phase.** The prover and verifier jointly execute Open on the instance

$$\mathbb{x}^{\circ} := \big((\mathsf{icm}, \mathsf{ians}), (\mathsf{cm}_1, \mathsf{ans}_1), \ldots, (\mathsf{cm}_{\mathsf{n}}, \mathsf{ans}_{\mathsf{n}}), (\mathsf{pcm}_1, \mathsf{pans}_1), \ldots, (\mathsf{pcm}_{\mathsf{k}}, \mathsf{pans}_{\mathsf{k}})\big)\,.$$

  In the honest case, the witness is

$$\mathbb{w}^{\circ} := \big((\mathrm{I}, \mathsf{iaux}), (\mathrm{X}_1, \mathsf{aux}_1), \ldots, (\mathrm{X}_{\mathsf{n}}, \mathsf{aux}_{\mathsf{n}}), (\Pi_1, \mathsf{paux}_1), \ldots, (\Pi_{\mathsf{k}}, \mathsf{paux}_{\mathsf{k}})\big)\,.$$

- **Decision phase.** (The verifier rejects if any check fails.)
  1. **Outer opening decision.** The verifier checks that $\mathbf{V}^{\mathsf{Ans}}(x, \alpha_1, \ldots, \alpha_{\mathsf{k}})$ accepts, i.e., it simulates an execution of $\mathbf{V}$ and answers queries according to Ans.
  2. **Inner opening decision.** The verifier checks that the verifier of Open accepts.

Lemma A.3. *Let* k° *be the number of rounds in* Open*. Suppose that* FIOP *has round-by-round knowledge soundness errors* $(\varepsilon_1, \ldots, \varepsilon_{\mathsf{k}})$*,* Commit *has round-by-round binding errors* $(\varepsilon^{\bullet}_1, \ldots, \varepsilon^{\bullet}_{\mathsf{k}^{\bullet}})$*, and* Open *has round-by-round knowledge soundness errors* $(\varepsilon^{\circ}_1, \ldots, \varepsilon^{\circ}_{\mathsf{k}^{\circ}})$*. Then, Construction A.2 has round-by-round knowledge soundness errors*

$$\big((\varepsilon^{\bullet}_j)_{j\in[\mathsf{k}^{\bullet}]}, \varepsilon_1, \ldots, (\varepsilon^{\bullet}_j)_{j\in[\mathsf{k}^{\bullet}]}, \varepsilon_{\mathsf{k}}, (\varepsilon^{\circ}_i)_{i\in[\mathsf{k}^{\circ}]}\big)\,.$$

The extraction time is the sum of the extraction times of FIOP *and* Open*.*

Proof. Let $\mathsf{State}_{\mathsf{FIOP}}$, $\mathsf{State}_{\mathsf{Commit}}$, and $\mathsf{State}_{\mathsf{Open}}$ be state functions for FIOP, Commit, and Open, respectively, with round-by-round errors as in Lemma A.3. Let $\mathbf{E}_{\mathsf{FIOP}}$ and $\mathbf{E}_{\mathsf{Open}}$ be corresponding extractors for $\mathsf{State}_{\mathsf{FIOP}}$ and $\mathsf{State}_{\mathsf{Open}}$, respectively. We define a state function State and extractor $\mathbf{E}$ for Construction A.2.

1. **Commitment randomness.** At this stage the transcript has the form

$$\mathsf{tr} = \big((\mathsf{pcm}_i, \alpha_i)_{i\in[i^{\star})}, (\Pi^{\bullet}_{i^{\star},j}, \alpha^{\bullet}_{i^{\star},j})_{j\in[j^{\star})}\big) \text{ for } i^{\star} \in [\mathsf{k}], j^{\star} \in [\mathsf{k}^{\bullet}]\,.$$

   The prover sends $\Pi^{\bullet}_{i^{\star},j^{\star}}$, and the verifier sends $\alpha^{\bullet}_{i^{\star},j^{\star}}$.

   State function. We set $\mathsf{State}(\mathbb{i}^{\dagger}, \mathbb{x}^{\dagger}, \mathsf{tr} \| \Pi^{\bullet}_{i^{\star},j^{\star}} \| \alpha^{\bullet}_{i^{\star},j^{\star}}, \mathsf{w}) = 1$ for

$$\mathsf{w} = \big(\mathsf{w}_{\mathsf{FIOP}}, (\mathrm{X}_i)_{i\in[\mathsf{n}]}, (\Pi_i)_{i\in[i^{\star})}\big)$$

   if and only if at least one of the following hold:

(a) The following hold:
   i. $\mathsf{State}_{\mathsf{FIOP}}(\mathbb{i}, \mathbb{x}, (\Pi_i, \alpha_i)_{i\in[i^\star)}, \mathsf{w}_{\mathsf{FIOP}}) = 1$, where $\mathbb{x} \coloneqq (x, \mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}})$.
   ii. For each $i \in [\mathsf{n}]$, it holds that $(\mathsf{cm}_i, \mathrm{X}_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_{\bullet}$.
   iii. For each $i \in [i^\star)$, it holds that $(\mathsf{pcm}_i, \Pi_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_{\bullet}$.

(b) For some $i \in [\mathsf{n}]$, it holds that $\mathsf{cm}_i \in \mathcal{L}_\times$.

(c) For some $i \in [i^\star)$, it holds that $\mathsf{pcm}_i \in \mathcal{L}_\times$.

(d) $\mathsf{State}_{\mathsf{Commit}}((\Pi^{\bullet}_{i^\star,j}, \alpha^{\bullet}_{i^\star,j})_{j\in[j^\star]}) = 1$.

Extractor. The extractor $\mathbf{E}(\mathbb{i}^\dagger, \mathbb{x}^\dagger, \mathsf{tr} \| \Pi^{\bullet}_{i^\star,j^\star} \| \alpha^{\bullet}_{i^\star,j^\star}, \mathsf{w}) \coloneqq \mathsf{w}$ is the identity function on the state function witness.

2. **FIOP randomness.** At this stage the transcript has the form

$$\mathsf{tr} = \left((\mathsf{pcm}_i, \alpha_i)_{i\in[i^\star)}, \mathsf{pcm}_{i^\star}\right) \text{ for } i^\star \in [\mathsf{k}]\,.$$

The verifier sends $\alpha_{i^\star}$.

State function. We set $\mathsf{State}(\mathbb{i}^\dagger, \mathbb{x}^\dagger, \mathsf{tr} \| \alpha_{i^\star}, \mathsf{w}) = 1$ for

$$\mathsf{w} = \left(\mathsf{w}_{\mathsf{FIOP}}, (\mathrm{X}_i)_{i\in[\mathsf{n}]}, (\Pi_i)_{i\in[i^\star]}\right)$$

if and only if at least one of the following hold:

(a) The following hold:
   i. $\mathsf{State}_{\mathsf{FIOP}}(\mathbb{i}, \mathbb{x}, (\Pi_i, \alpha_i)_{i\in[i^\star]}, \mathsf{w}_{\mathsf{FIOP}}) = 1$, where $\mathbb{x} \coloneqq (x, \mathrm{X}_1, \ldots, \mathrm{X}_{\mathsf{n}})$.
   ii. For each $i \in [\mathsf{n}]$, it holds that $(\mathsf{cm}_i, \mathrm{X}_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_{\bullet}$.
   iii. For each $i \in [i^\star]$, it holds that $(\mathsf{pcm}_i, \Pi_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_{\bullet}$.

(b) For some $i \in [\mathsf{n}]$, it holds that $\mathsf{cm}_i \in \mathcal{L}_\times$.

(c) For some $i \in [i^\star]$, it holds that $\mathsf{pcm}_i \in \mathcal{L}_\times$.

Extractor. The extractor $\mathbf{E}(\mathbb{i}^\dagger, \mathbb{x}^\dagger, \mathsf{tr} \| \alpha_{i^\star}, \mathsf{w})$ computes

$$\mathsf{w}'_{\mathsf{FIOP}} \coloneqq \mathbf{E}_{\mathsf{FIOP}}(\mathbb{i}, \mathbb{x}, (\Pi_i, \alpha_i)_{i\in[i^\star]}, \mathsf{w}_{\mathsf{FIOP}})$$

and outputs $(\mathsf{w}'_{\mathsf{FIOP}}, (\mathrm{X}_i)_{i\in[\mathsf{n}]}, (\Pi_i)_{i\in[i^\star)})$.

3. **FIOP answers.** At this stage the transcript has the form

$$\mathsf{tr} = \left((\mathsf{pcm}_i, \alpha_i)_{i\in[\mathsf{k}]}\right).$$

The prover sends Ans.

State function. We set $\mathsf{State}(\mathbb{i}^\dagger, \mathbb{x}^\dagger, \mathsf{tr} \| \mathsf{Ans}, \mathsf{w}) = 1$ for

$$\mathsf{w} = \left((\mathrm{X}_i)_{i\in[\mathsf{n}]}, (\Pi_i)_{i\in[\mathsf{k}]}\right)$$

if and only if at least one of the following hold:

(a) The following hold:
   i. $\mathbf{V}^{\mathsf{Ans}}_{\mathsf{FIOP}}(x, \alpha_1, \ldots, \alpha_{\mathsf{k}})$ accepts.
   ii. For each $i \in [\mathsf{n}]$, it holds that $(\mathsf{cm}_i, \mathrm{X}_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_{\bullet}$ and X_i agrees with ans_i.

iii. For each $i \in [\mathsf{k}]$, it holds that $(\mathsf{pcm}_i, \Pi_i) \in \overline{\mathcal{R}}_{\bullet}^{\mathrm{NO}}$ and Π_i agrees with pans_i.

(b) For some $i \in [\mathsf{n}]$, it holds that $\mathsf{cm}_i \in \mathcal{L}_{\times}$.

(c) For some $i \in [\mathsf{k}]$, it holds that $\mathsf{pcm}_i \in \mathcal{L}_{\times}$.

Extractor. The extractor $\mathbf{E}(\mathbb{i}^{\dagger}, \mathbb{x}^{\dagger}, \mathsf{tr} \| \mathsf{Ans}, \mathsf{w})$ outputs $(\mathsf{w}_{\mathsf{FIOP}} = \bot, \mathsf{w})$.

4. **Opening randomness.** Let k° denote the number of rounds in Open. For simplicity, we assume the prover of Open does not send non-oracle messages. At this stage the transcript has the form

$$\mathsf{tr} = \left((\mathsf{pcm}_i, \alpha_i)_{i \in [\mathsf{k}]}, \mathsf{Ans}, (\Pi_j^{\circ}, \alpha_j^{\circ})_{j \in [j^{\star}]}\right) \text{ for } j^{\star} \in [\mathsf{k}^{\circ}] \,.$$

The prover sends $\Pi_{j^{\star}}^{\circ}$, and the verifier sends $\alpha_{j^{\star}}^{\circ}$.

State function. We set $\mathsf{State}(\mathbb{i}^{\dagger}, \mathbb{x}^{\dagger}, \mathsf{tr} \| \Pi_{j^{\star}}^{\circ} \| \alpha_{j^{\star}}^{\circ}, \mathsf{w}) = 1$ for $\mathsf{w} = \mathsf{w}_{\mathsf{Open}}$ if and only if the following hold:

(a) $\mathbf{V}_{\mathsf{FIOP}}^{\mathsf{Ans}}(x, \alpha_1, \ldots, \alpha_{\mathsf{k}})$ accepts.

(b) $\mathsf{State}_{\mathsf{Open}}(\mathbb{x}^{\circ}, (\Pi_j^{\circ}, \alpha_j^{\circ})_{j \in [j^{\star}]}, \mathsf{w}_{\mathsf{Open}}) = 1$.

Extractor. The extractor $\mathbf{E}(\mathbb{i}^{\dagger}, \mathbb{x}^{\dagger}, \mathsf{tr} \| \Pi_{j^{\star}}^{\circ} \| \alpha_{j^{\star}}^{\circ}, \mathsf{w})$ outputs $\mathbf{E}_{\mathsf{Open}}(\mathbb{x}^{\circ}, (\Pi_j^{\circ}, \alpha_j^{\circ})_{j \in [j^{\star}]}, \mathsf{w}_{\mathsf{Open}})$.

We upper bound the round-by-round knowledge soundness errors of State with respect to $\mathbf{E}$.

1. **Commitment randomness.** We show that

$$\Pr_{\alpha_{i^{\star},j^{\star}}^{\bullet}} \left[\exists \mathsf{w} : \begin{array}{c} \mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| \Pi_{i^{\star},j^{\star}}^{\bullet} \| \alpha_{i^{\star},j^{\star}}^{\bullet}, \mathsf{w})\big) = 0 \\ \wedge\, \mathsf{State}(\mathbb{x}, \mathsf{tr} \| \Pi_{i^{\star},j^{\star}}^{\bullet} \| \alpha_{i^{\star},j^{\star}}^{\bullet}, \mathsf{w}) = 1 \end{array} \right] \leq \varepsilon_{j^{\star}}^{\bullet} \,.$$

This follows from the round-by-round binding of Commit.

2. **FIOP randomness.** We show that

$$\Pr_{\alpha_{i^{\star}}} \left[\exists \mathsf{w} : \begin{array}{c} \mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| \alpha_{i^{\star}}, \mathsf{w})\big) = 0 \\ \wedge\, \mathsf{State}(\mathbb{x}, \mathsf{tr} \| \alpha_{i^{\star}}, \mathsf{w}) = 1 \end{array} \right] \leq \varepsilon_{i^{\star}} \,.$$

There are three cases.

- For some $i \in [i^{\star}]$, there does not exist Π such that $(\mathsf{cm}_i^{\bullet}, \Pi) \in \overline{\mathcal{R}}_{\bullet}^{\mathrm{NO}}$. Then Item 2a cannot hold, and we conclude that the state function never transitions from 0 to 1.
- For some $i \in [i^{\star}]$, there exist distinct Π, Π' such that $(\mathsf{cm}_i^{\bullet}, \Pi), (\mathsf{cm}_i^{\bullet}, \Pi') \in \overline{\mathcal{R}}_{\bullet}^{\mathrm{NO}}$. Then Item 1c or Item 1d must hold, and we conclude that the state function never transitions from 0 to 1.
- There exist a unique choice of oracles $\Pi_1, \ldots, \Pi_{i^{\star}}$ such that $(\mathsf{cm}_i^{\bullet}, \Pi_i) \in \overline{\mathcal{R}}_{\bullet}$ for each $i \in [i^{\star}]$. Then the upper bound follows from the round-by-round binding of FIOP.

3. **Answers.** (The verifier does send any randomness in this round.) Let w be a state function witness where $\mathsf{State}(\mathbb{x}, \mathsf{tr} \| \mathsf{Ans}, \mathsf{w}) = 1$. We show that $\mathsf{State}(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| \mathsf{Ans}, \mathsf{w})) = 1$. This follows from the fact that Ans is consistent with the oracles in w when Item 3a holds.

4. **Inner opening phase.** We show that

$$\Pr_{\alpha_{j^{\star}}^{\circ}} \left[\exists \mathsf{w} : \begin{array}{c} \mathsf{State}\big(\mathbb{x}, \mathsf{tr}, \mathbf{E}(\mathbb{x}, \mathsf{tr} \| \Pi_{j^{\star}}^{\circ} \| \alpha_{j^{\star}}^{\circ}, \mathsf{w})\big) = 0 \\ \wedge\, \mathsf{State}(\mathbb{x}, \mathsf{tr} \| \Pi_{j^{\star}}^{\circ} \| \alpha_{j^{\star}}^{\circ}, \mathsf{w}) = 1 \end{array} \right] \leq \varepsilon_{j^{\star}}^{\circ} \,.$$

This follows from the round-by-round knowledge soundness of Open. For $j^{\star} = 1$, we additionally use the fact that $\mathsf{State}_{\mathsf{Open}}(\mathbb{x}^{\circ}, \emptyset, \mathsf{w}_{\mathsf{Open}}) = 1$ implies that $(\mathbb{x}^{\circ}, \mathsf{w}_{\mathsf{Open}}) \in \overline{\mathcal{R}}_{\circ}^{\mathrm{NO}}$ (Definition 4.5).

□

A.2 Commitment composition

Lemma A.4. *There exists a transformation* T_c *such that the following holds. Let* $\mathcal{O}$ *and* $\mathcal{O}'$ *be oracle classes, and suppose we have:*

- Commit'*, the commitment phase of an interactive* $\mathcal{O}'$*-commitment with output relation* $\mathcal{R}'_\bullet$*, failure language* $\mathcal{L}'_\times$*, and* k *rounds of interaction. Let* $I \subseteq [\mathsf{k}]$ *denote the set of indices* i *where the commitment's* i*-th oracle is in* $\mathcal{O}$*.*
- Commit*, the commitment phase of an interactive* $\mathcal{O}$*-commitment with output relation* $\mathcal{R}_\bullet$ *and failure language* $\mathcal{L}_\times$*.*

Then, $\mathrm{T}_c[\mathsf{Commit}', \mathsf{Commit}]$ *is an interactive* $\mathcal{O}'$*-commitment with output relation* $\mathcal{R}^\dagger_\bullet$ *and failure language* $\mathcal{L}^\dagger_\times$*, defined as follows. Commitments and auxiliary information have the form*

$$\mathsf{cm}^\dagger = \left((\mathsf{cm}_i)_{i\in I}, (\Pi_i)_{i\in[\mathsf{k}]\setminus I}, (\pi_i, \alpha_i)_{i\in[\mathsf{k}]}\right), \quad \mathsf{aux}^\dagger = \left(\mathsf{aux}', (\Pi_i, \mathsf{aux}_i)_{i\in I}\right).$$

- $(\mathcal{R}^\dagger_\bullet)^{\mathrm{YES}}$ *consists of instances* $\mathbb{x} = \mathsf{cm}^\dagger$ *and witnesses* $\mathbb{w} = (\Pi, \mathsf{aux}^\dagger)$ *where:*
  - *For each* $i \in I$*, it holds that* $(\mathsf{cm}_i, (\Pi_i, \mathsf{aux}_i)) \in \mathcal{R}^{\mathrm{YES}}_\bullet$*.*
  - *With* $\mathsf{cm}' = (\Pi_i, \pi_i, \alpha_i)_{i\in[\mathsf{k}]}$*, it holds that* $(\mathsf{cm}', (\Pi, \mathsf{aux}')) \in (\mathcal{R}'_\bullet)^{\mathrm{YES}}$*.*
- $(\overline{\mathcal{R}}^\dagger_\bullet)^{\mathrm{NO}}$ *consists of instances* $\mathbb{x} = \mathsf{cm}^\dagger$ *and witnesses* $\mathbb{w} = \Pi$ *where there exists* $(\Pi_i)_{i\in I}$ *such that the following holds:*
  - *For each* $i \in I$*, it holds that* $(\mathsf{cm}_i, \Pi_i) \in \mathcal{R}^{\mathrm{NO}}_\bullet$*.*
  - *With* $\mathsf{cm}' = (\Pi_i, \pi_i, \alpha_i)_{i\in[\mathsf{k}]}$*, it holds that* $(\mathsf{cm}', \Pi) \in (\overline{\mathcal{R}}'_\bullet)^{\mathrm{NO}}$*.*
- $\mathcal{L}^\dagger_\times$ *consists of instances* $\mathbb{x} = \mathsf{cm}^\dagger$ *where at least one of the following holds:*
  - *For some* $i \in I$*, it holds that* $\mathsf{cm}_i \in \mathcal{L}_\times$*.*
  - *There exists* $(\Pi_i)_{i\in I}$*, where* $(\mathsf{cm}_i, \Pi_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_\bullet$ *for each* $i \in I$*, such that* $(\Pi_i, \pi_i, \alpha_i)_{i\in[\mathsf{k}]} \in \mathcal{L}'_\times$*.*

Moreover, $\mathrm{T}_c[\mathsf{Commit}', \mathsf{Commit}]$ *has the following efficiency measures.*

- *The number of rounds, non-oracle message lengths, randomness lengths, and commit time correspond to:*
  - *An execution of* Commit'*.*
  - $|I|$ *executions of* Commit*.*
- *The commitment oracles consist of:*
  - *All but the* $|I|$ *oracles in* $\mathcal{O}$ *from a commitment produced by* Commit'*.*
  - *Oracles from* $|I|$ *commitments produced by* Commit*.*

The round-by-round binding error is the maximum of the round-by-round binding errors of Commit' *and* Commit*.*

Proof. In Construction A.6 we describe $\mathrm{T}_c[\mathsf{Commit}', \mathsf{Commit}]$, and in Lemma A.7 we analyze its round-by-round binding errors. □

Corollary A.5. *Let* T_p *be the transformation in Lemma A.1 and* T_c *be the transformation in Lemma A.4. Let* $\mathcal{O}$ *and* $\mathcal{O}'$ *be oracle classes, and suppose we have:*

- $\mathsf{FC}' = (\mathsf{Commit}', \mathsf{Open}')$*, an interactive* $\mathcal{O}'$*-commitment.*
- $\mathsf{FC} = (\mathsf{Commit}, \mathsf{Open})$*, an interactive* $\mathcal{O}$*-commitment.*

Then, $\mathrm{T}[\mathsf{FC}', \mathsf{FC}] \coloneqq (\mathrm{T}_c[\mathsf{Commit}', \mathsf{Commit}], \mathrm{T}_p[\mathsf{Open}', \mathsf{FC}])$ *is an interactive* $\mathcal{O}'$*-commitment.*

Proof. Let $\mathcal{R}'_\bullet$ and $\mathcal{L}'_\times$ be the output relation and failure language of Commit'. Let $\mathcal{R}^\dagger$ be as in Lemma A.1, defined with respect to $\mathrm{T}_p[\mathsf{Open}', \mathsf{FC}]$. Let $\mathcal{R}^\dagger_\bullet$ and $\mathcal{L}^\dagger_\times$ be as in Lemma A.4, defined with respect to $\mathrm{T}_c[\mathsf{Commit}', \mathsf{Commit}]$. The statement follows from the fact that $\mathcal{R}^\dagger$ is equivalent to $\mathcal{R}_\circ(\mathcal{R}^\dagger_\bullet, \mathcal{L}^\dagger_\times)$. □

Construction A.6. We describe $\mathrm{T}_c[\mathsf{Commit}', \mathsf{Commit}]$. Let $\mathsf{Commit}' = (\mathbf{C}', \mathcal{R}'_\bullet, \mathcal{L}'_\times)$ and $\mathsf{Commit} = (\mathbf{C}, \mathcal{R}_\bullet, \mathcal{L}_\times)$. Let k and $\mathsf{k}^\bullet$ denote the number of rounds in Commit' and Commit, respectively. Let $\mathsf{r}_1, \ldots, \mathsf{r}_\mathsf{k}$ and $\mathsf{r}^\bullet_1, \ldots, \mathsf{r}^\bullet_{\mathsf{k}^\bullet}$ denote the randomness lengths in each round of Commit' and Commit, respectively. For simplicity, we assume the following:
- The committers $\mathbf{C}'$ and $\mathbf{C}$ do not send non-oracle messages.
- The committer $\mathbf{C}'$ sends an $\mathcal{O}$-oracle in each round.

The construction and security analysis naturally extend to the general setting.

- **Inputs.** In the honest case, the prover receives an oracle $\Pi \in \mathcal{O}'$.
- **Outer commitment.** For $i \in [\mathsf{k}]$:
  1. **Outer oracle.** In the honest case, the prover computes
  $$\Pi_i \coloneqq \mathbf{C}'(\Pi, \alpha_1, \ldots, \alpha_{i-1}) .$$
  2. **Inner commitment.** For $j \in [\mathsf{k}^\bullet]$:
     (a) **Inner oracle.** The prover sends $\Pi^\bullet_{i,j}$. In the honest case,
     $$\Pi^\bullet_{i,j} \coloneqq \mathbf{C}(\Pi_i, \alpha^\bullet_{i,1}, \ldots, \alpha^\bullet_{i,j-1}) .$$
     (b) **Inner randomness.** The verifier samples and sends $\alpha^\bullet_{i,j} \leftarrow \{0,1\}^{\mathsf{r}^\bullet_j}$.

     Set $\mathsf{cm}_i \coloneqq (\Pi^\bullet_{i,j}, \alpha^\bullet_{i,j})_{j \in [\mathsf{k}^\bullet]}$. In the honest case, the prover computes $\mathsf{aux}_i \coloneqq \mathbf{C}(\Pi_i, \alpha^\bullet_{i,1}, \ldots, \alpha^\bullet_{i,\mathsf{k}^\bullet})$.
  3. **Outer randomness.** The verifier samples and sends $\alpha_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.

  In the honest case, the prover computes $\mathsf{aux}' \coloneqq \mathbf{C}'(\Pi, \alpha_1, \ldots, \alpha_\mathsf{k})$.
- **Decision phase.** The commitment is $\mathsf{cm}^\dagger \coloneqq (\mathsf{cm}^\bullet_i, \alpha_i)_{i \in [\mathsf{k}]}$. In the honest case, the auxiliary information is $\mathsf{aux}^\dagger \coloneqq (\mathsf{aux}', (\mathsf{aux}_i)_{i \in [\mathsf{k}]})$.

Lemma A.7. *Suppose that* Commit *has round-by-round binding errors* $(\varepsilon^\bullet_1, \ldots, \varepsilon^\bullet_{\mathsf{k}^\bullet})$ *and* Commit' *has round-by-round binding errors* $(\varepsilon_1, \ldots, \varepsilon_\mathsf{k})$*. Construction A.6 has round-by-round binding errors*

$$\left((\varepsilon^\bullet_j)_{j \in [\mathsf{k}^\bullet]}, \varepsilon_1, \ldots, (\varepsilon^\bullet_j)_{j \in [\mathsf{k}^\bullet]}, \varepsilon_\mathsf{k}\right) .$$

Proof. Let $\mathsf{State}_{\mathsf{Commit}'}$ and $\mathsf{State}_{\mathsf{Commit}}$ be state functions for Commit' and Commit, respectively, with round-by-round errors as in Lemma A.7. We define a state function State for Construction A.6.

1. **Inner randomness.** At this stage the transcript has the form

$$\mathsf{tr} = \left((\mathsf{cm}_i, \alpha_i)_{i\in[i^\star)}, (\Pi^\bullet_{i^\star,j}, \alpha^\bullet_{i^\star,j})_{j\in[j^\star)}\right) \text{ for } i^\star \in [\mathsf{k}], j^\star \in [\mathsf{k}^\bullet]\,.$$

   The prover sends $\Pi^\bullet_{i^\star,j^\star}$, and the verifier sends $\alpha^\bullet_{i^\star,j^\star}$.

   State function. We set $\mathsf{State}(\mathsf{tr}\|\Pi^\bullet_{i^\star,j^\star}\|\alpha^\bullet_{i^\star,j^\star}) = 1$ if and only if at least one of the following hold:

   (a) There exist $\Pi_1, \ldots, \Pi_{i^\star-1}$ with $(\mathsf{cm}_i, \Pi_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_\bullet$ for each $i \in [i^\star)$ such that $\mathsf{State}_{\mathsf{Commit}'}((\Pi_i, \alpha_i)_{i\in[i^\star)}) = 1$.
   (b) For some $i \in [i^\star)$, it holds that $\mathsf{cm}_i \in \mathcal{L}_\times$.
   (c) $\mathsf{State}_{\mathsf{Commit}}((\Pi^\bullet_{i^\star,j}, \alpha^\bullet_{i^\star,j})_{j\in[j^\star]}) = 1$.

2. **Outer randomness.** At this stage the transcript has the form

$$\mathsf{tr} = \left((\mathsf{cm}_i, \alpha_i)_{i\in[i^\star)}, (\Pi^\bullet_{i^\star,j}, \alpha^\bullet_{i^\star,j})_{j\in[\mathsf{k}^\bullet]}\right) \text{ for } i^\star \in [\mathsf{k}]\,.$$

   The verifier sends $\alpha_{i^\star}$.

   State function. We set $\mathsf{State}(\mathsf{tr}\|\alpha_{i^\star}) = 1$ if and only if at least one of the following hold:

   (a) There exist $\Pi_1, \ldots, \Pi_{i^\star}$ with $(\mathsf{cm}_i, \Pi_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_\bullet$ for each $i \in [i^\star]$ such that $\mathsf{State}_{\mathsf{Commit}'}((\Pi_i, \alpha_i)_{i\in[i^\star]}) = 1$.
   (b) For some $i \in [i^\star]$, it holds that $\mathsf{cm}_i \in \mathcal{L}_\times$.

We upper bound the round-by-round binding errors of State.

1. **Inner randomness.** Suppose $\mathsf{State}(\mathsf{tr}) = 0$. We show that

$$\Pr_{\alpha^\bullet_{i^\star,j^\star}}\left[\mathsf{State}(\varkappa, \mathsf{tr}\|\Pi^\bullet_{i^\star,j^\star}\|\alpha^\bullet_{i^\star,j^\star}) = 1\right] \le \varepsilon^\bullet_{j^\star}\,.$$

   This follows from the round-by-round binding of Commit.

2. **Outer randomness.** Suppose $\mathsf{State}(\mathsf{tr}) = 0$. We show that

$$\Pr_{\alpha_{i^\star}}\left[\mathsf{State}(\varkappa, \mathsf{tr}\|\alpha_{i^\star}) = 1\right] \le \varepsilon_{i^\star}\,.$$

   Observe that Item 2b does not hold, since Items 1b and 1c do not hold. There are three cases.

   - For some $i \in [i^\star]$, there does not exist Π such that $(\mathsf{cm}^\bullet_i, \Pi) \in \overline{\mathcal{R}}^{\mathrm{NO}}_\bullet$. Then Item 2a cannot hold, and we conclude that the state function never transitions.
   - For some $i \in [i^\star]$, there exist distinct Π, Π' such that $(\mathsf{cm}^\bullet_i, \Pi), (\mathsf{cm}^\bullet_i, \Pi') \in \overline{\mathcal{R}}^{\mathrm{NO}}_\bullet$. But this is ruled out, since Item 2b does not hold.
   - There exist a unique choice of oracles $\Pi_1, \ldots, \Pi_{i^\star}$ with $(\mathsf{cm}_i, \Pi_i) \in \overline{\mathcal{R}}^{\mathrm{NO}}_\bullet$ for each $i \in [i^\star]$. Then the upper bound follows from the round-by-round binding of Commit'.

□